

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ VIỄN THÔNG
BỘ MÔN MÁY TÍNH – HỆ THỐNG NHÚNG



NGUYỄN ĐĂNG TRÍ

Đề tài đồ án tốt nghiệp:

**DÙNG AI ĐỂ PHÁT HIỆN DỮ LIỆU SAI LỆCH,
THIẾU, BẤT THƯỜNG TRONG CƠ SỞ DỮ LIỆU.
(DATA CLEANING)**

Chuyên ngành Máy Tính - Hệ Thống Nhúng

TP. Hồ Chí Minh, tháng 08 năm 2026

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG
BỘ MÔN MÁY TÍNH - HỆ THỐNG NHÚNG



NGUYỄN ĐĂNG TRÍ

21200241

Đề tài:

DÙNG AI ĐỂ PHÁT HIỆN DỮ LIỆU SAI LỆCH,
THIẾU, BẤT THƯỜNG TRONG CƠ SỞ DỮ LIỆU
(DATA CLEANING)

AI-BASED DETECTION OF MISSING, INVALID, AND
ANOMALOUS DATA IN DATABASES (DATA CLEANING)

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN
NGÀNH KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG
CHUYÊN NGÀNH MÁY TÍNH - HỆ THỐNG NHÚNG

NGƯỜI HƯỚNG DẪN KHOA HỌC

ThS. Lê Đức Trí

TP. Hồ Chí Minh, tháng 08 năm 2026

Lời cam đoan

Em xin cam đoan đồ án tốt nghiệp với đề tài **“Dùng AI phát hiện dữ liệu sai lệch, thiếu, bất thường trong cơ sở dữ liệu (làm sạch dữ liệu)”** là công trình nghiên cứu và thực hiện của cá nhân em dưới sự hướng dẫn của giảng viên hướng dẫn. Các nội dung trình bày trong đồ án được xây dựng dựa trên quá trình tìm hiểu lý thuyết, phân tích bài toán, thiết kế hệ thống, cài đặt chương trình và thực nghiệm trên tập dữ liệu đã nêu trong báo cáo.

Em xin cam đoan rằng mã nguồn, kết quả thực nghiệm, bảng biểu và các phân tích trong báo cáo được thực hiện một cách trung thực trong phạm vi đồ án. Những tài liệu, bài báo, thư viện, framework, dataset và nguồn tham khảo được sử dụng trong quá trình thực hiện đề tài đều được ghi rõ ở phần Các nguồn tham khảo cuối báo cáo. Các nội dung kế thừa từ tài liệu bên ngoài được sử dụng với mục đích học tập, nghiên cứu và được diễn giải lại phù hợp với phạm vi đề tài.

Em xin chịu trách nhiệm trước nhà trường, khoa và giảng viên hướng dẫn về tính trung thực của nội dung báo cáo cũng như kết quả thực hiện đồ án. Nếu phát hiện có bất kỳ sự sao chép, gian lận hoặc sử dụng tài liệu không đúng quy định, em xin hoàn toàn chịu trách nhiệm theo quy định của nhà trường.

TP.HCM, ngày 04 tháng 07 năm 2026

Sinh viên thực hiện

Nguyễn Đăng Trí

Lời cảm ơn

Để hoàn thành đồ án tốt nghiệp với đề tài “Ứng dụng AI hỗ trợ phát hiện và gợi ý làm sạch dữ liệu thiếu, sai luật và bất thường trong cơ sở dữ liệu”, em đã nhận được sự quan tâm, hướng dẫn, động viên và hỗ trợ quý báu từ quý thầy cô, gia đình và bạn bè trong suốt quá trình học tập, nghiên cứu và triển khai đề tài.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn, ThS. Lê Đức Trị, người đã tận tình định hướng, góp ý và hỗ trợ em trong suốt quá trình thực hiện đồ án. Những nhận xét của thầy về phạm vi hệ thống, cách tổ chức nội dung, hướng tiếp cận kỹ thuật và phương pháp trình bày báo cáo đã giúp em điều chỉnh đề tài theo hướng rõ ràng, thực tế và phù hợp hơn với yêu cầu của một đồ án tốt nghiệp. Đặc biệt, các góp ý liên quan đến vai trò của cơ sở dữ liệu, quy trình xử lý dữ liệu và việc ứng dụng AI trong phát hiện dữ liệu bất thường là cơ sở quan trọng để em hoàn thiện hệ thống.

Em xin chân thành cảm ơn quý thầy cô tại Khoa Điện Tử - Viễn Thông, Trường Khoa Học Tự Nhiên đã tận tình giảng dạy và truyền đạt những kiến thức nền tảng, chuyên môn trong suốt thời gian em học tập tại trường. Các kiến thức về lập trình, cơ sở dữ liệu, phân tích thiết kế hệ thống, xử lý dữ liệu và trí tuệ nhân tạo đã giúp em có đủ nền tảng để phân tích bài toán, thiết kế cơ sở dữ liệu, xây dựng quy trình phát hiện lỗi và đánh giá kết quả thực nghiệm.

Em xin chân thành cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ em trong suốt quá trình học tập và hoàn thành đồ án. Mặc dù đã nỗ lực, đồ án khó tránh khỏi thiếu sót. Em rất mong nhận được những ý kiến đóng góp từ quý thầy cô để đề tài được hoàn thiện hơn và giúp em tích lũy thêm kinh nghiệm.

Em xin chân thành cảm ơn.

TP.HCM, ngày 04 tháng 07 năm 2026

Sinh viên thực hiện

Nguyễn Đăng Trí

Tóm tắt đề án

Trong bối cảnh các hệ thống thông tin ngày càng phụ thuộc vào dữ liệu, chất lượng dữ liệu trở thành một yếu tố quan trọng ảnh hưởng trực tiếp đến hoạt động phân tích, báo cáo và ra quyết định. Đối với dữ liệu giao dịch thương mại điện tử, các lỗi như thiếu giá trị, sai miền dữ liệu, sai định dạng hoặc bất thường theo ngữ cảnh có thể làm sai lệch kết quả phân tích, gây khó khăn cho việc khai thác dữ liệu và ảnh hưởng đến độ tin cậy của hệ thống.

Nhằm giải quyết bài toán trên, đề án thực hiện xây dựng một hệ thống hỗ trợ phát hiện và gợi ý làm sạch dữ liệu cho tập dữ liệu giao dịch mua sắm. Hệ thống được thiết kế theo hướng kết hợp giữa kiểm tra dựa trên luật và phương pháp học máy. Các lỗi rõ ràng như thiếu dữ liệu, sai miền giá trị hoặc sai định dạng được phát hiện bằng rule-based validation. Các bản ghi bất thường theo ngữ cảnh được phát hiện bằng mô hình Isolation Forest. Ngoài ra, hệ thống sử dụng phương pháp nearest neighbors để tìm các bản ghi tương tự và đưa ra gợi ý sửa cho một số trường dữ liệu phân loại hoặc dạng văn bản.

Về mặt lưu trữ, đề án sử dụng PostgreSQL để quản lý dữ liệu đã chuẩn hóa, dữ liệu thô, các lỗi được phát hiện, gợi ý sửa lỗi, báo cáo chất lượng dữ liệu và phiên bản dữ liệu sau khi sửa. Pipeline xử lý được xây dựng bằng Python, sử dụng pandas cho xử lý dữ liệu dạng bảng, scikit-learn cho mô hình phát hiện bất thường và SQLAlchemy để kết nối với cơ sở dữ liệu. Hệ thống có thể chạy với nguồn dữ liệu từ CSV hoặc PostgreSQL, sau đó sinh ra các bảng kết quả phục vụ kiểm tra, đánh giá và trình bày.

Cấu trúc báo cáo bao gồm 5 chương. Chương 1 trình bày tổng quan đề tài, lý do chọn đề tài, mục tiêu, phạm vi và hướng tiếp cận. Chương 2 trình bày cơ sở lý thuyết về chất lượng dữ liệu, rule-based validation, anomaly detection, Isolation Forest, nearest neighbors và các công nghệ sử dụng. Chương 3 phân tích và thiết kế hệ thống, bao gồm thiết kế dữ liệu, kiến trúc tổng thể, pipeline xử lý và cơ sở dữ liệu PostgreSQL. Chương 4 trình bày quá trình cài đặt các module chính của hệ thống. Chương 5 trình bày thực nghiệm, kết quả đánh giá, nhận xét, hạn chế và hướng phát triển.

MỤC LỤC

Lời cam đoan	iii
Lời cảm ơn	iv
Tóm tắt đồ án	v
MỤC LỤC	vi
Danh mục từ viết tắt	x
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI	1
1.1. Đặt vấn đề	1
1.2. Lý do chọn đề tài	2
1.3. Mục tiêu của đề tài	3
1.3.1. Câu hỏi nghiên cứu	4
1.3.2. Sản phẩm đầu ra của đề tài	5
1.4. Đối tượng và phạm vi nghiên cứu	6
1.5. Phương pháp thực hiện	6
1.5.1. So sánh các hướng tiếp cận	7
1.6. Đóng góp chính của đề tài	8
1.7. Tiêu chí đánh giá hệ thống	8
1.8. Kết luận chương	9
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ LIÊN QUAN	10
2.1. Chất lượng dữ liệu	10
2.1.1. Vai trò của data profiling	11
2.2. Các loại lỗi dữ liệu thường gặp	12
2.3. Rule-based validation	13
2.3.1. Ưu điểm của cấu hình rule bằng YAML	14
2.4. Phát hiện bất thường dữ liệu	14
2.5. Isolation Forest	15
2.5.1. Cơ sở trực giác của cơ chế cô lập	16
2.5.2. Cấu trúc Isolation Tree	16
2.5.3. Độ dài đường đi và kỳ vọng trên toàn bộ rừng	18
2.5.4. Chuẩn hóa độ dài đường đi và điểm bất thường	19
2.5.5. Cơ chế ensemble và vai trò của lấy mẫu	20
2.5.6. Ý nghĩa của các tham số chính	21

2.5.7. Diễn giải điểm số trong scikit-learn	21
2.5.8. Tiền xử lý dữ liệu số và dữ liệu phân loại	22
2.5.9. Mô hình global và mô hình theo segment	23
2.5.10. Ưu điểm, hạn chế và vai trò trong hệ thống	23
2.6. Nearest Neighbors cho gợi ý sửa dữ liệu	24
2.6.1. Confidence trong recommendation	25
2.7. Feature engineering cho dữ liệu giao dịch	25
2.8. PostgreSQL	27
2.9. Pandas, Scikit-learn và SQLAlchemy	28
2.10. Chỉ số đánh giá	28
2.11. Kết luận chương	29
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	31
3.1. Phân tích bài toán	31
3.1.1. Đầu vào, xử lý và đầu ra	32
3.1.2. Luồng nghiệp vụ của một issue	33
3.2. Dataset sử dụng	33
3.3. Phân loại lỗi dữ liệu trong hệ thống	35
3.4. Yêu cầu chức năng	36
3.5. Yêu cầu phi chức năng	37
3.6. Kiến trúc tổng thể hệ thống	38
3.7. Thiết kế pipeline xử lý dữ liệu	39
3.8. Thiết kế cơ sở dữ liệu PostgreSQL	42
3.8.1. Phân loại các nhóm bảng	42
3.8.2. Mô hình ERD tổng thể	44
3.8.3. Thiết kế khóa chính, khóa ngoại và quan hệ	45
3.8.4. Ràng buộc CHECK và vai trò kiểm soát dữ liệu	47
3.8.5. Thiết kế index hỗ trợ truy vấn	48
3.8.6. Luồng dữ liệu từ staging đến kết quả làm sạch	49
3.9. Thiết kế issue schema	50
3.10. Thiết kế severity và confidence	51
3.11. Thiết kế cấu hình YAML	52
3.12. Thiết kế cơ chế auto-fix có kiểm soát	52

3.13. Tính liên kết giữa các thành phần thiết kế.....	53
3.14. Kết luận chương	54
CHƯƠNG 4. CÀI ĐẶT HỆ THỐNG	55
4.1. Công nghệ sử dụng.....	55
4.2. Cấu trúc thư mục project.....	56
4.3. Cài đặt và khởi tạo cơ sở dữ liệu PostgreSQL	57
4.3.1. Vai trò của file postgresql_schema.sql	57
4.3.2. Chạy schema bằng pgAdmin và psql	61
4.3.3. Vai trò và phạm vi của file littlemigration.sql	62
4.3.4. So sánh hai file SQL	64
4.4. Module import dữ liệu	65
4.5. Module truy cập dữ liệu	67
4.6. Module tiền xử lý và feature engineering	69
4.7. Module phát hiện missing data	70
4.8. Module phát hiện invalid data	71
4.9. Module phát hiện anomaly bằng Isolation Forest	72
4.9.1. Chuẩn bị ma trận đặc trưng	72
4.9.2. Khởi tạo và huấn luyện mô hình	73
4.9.3. Mô hình global và các mô hình theo category	74
4.9.4. Sinh và lưu kết quả phát hiện	74
4.10. Module feature contribution	75
4.11. Module gợi ý sửa bằng nearest neighbors	76
4.12. Module report và output	77
4.13. Giao diện dòng lệnh CLI	78
4.14. Một số vấn đề kỹ thuật đã xử lý	80
4.15. Luồng vận hành end-to-end của hệ thống	81
4.16. Kết luận chương	82
CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ	83
5.1. Mục tiêu thực nghiệm	83
5.2. Môi trường thực nghiệm	84
5.3. Dataset thực nghiệm	84
5.3.1. Phương pháp tạo dữ liệu lỗi	86

5.4. Kịch bản thực nghiệm	86
5.5. Kết quả trên sample dirty	88
5.5.1. Phân tích dòng lỗi tiêu biểu	90
5.6. Kết quả trên full CSV	90
5.6.1. Nhận xét về tỷ lệ anomaly	93
5.7. Kết quả gợi ý sửa lỗi	94
5.8. Kết quả apply fixes	95
5.9. Đánh giá hiệu năng	96
5.10. Nhận xét kết quả thực nghiệm	98
5.10.1. So sánh rule-based only và hybrid approach	98
5.10.2. Phân tích độ nhạy của cấu hình Isolation Forest	99
5.10.3. Các yếu tố ảnh hưởng đến độ tin cậy	100
5.11. Ưu điểm của hệ thống	101
5.12. Hạn chế của hệ thống	101
5.13. Đánh giá tổng hợp theo mục tiêu đề tài	101
5.14. Kết luận chương	103
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	104
1. Kết quả đạt được	104
2. Hạn chế	105
3. Hướng phát triển	105
Phụ lục A. Các file output chính	Error! Bookmark not defined.
Phụ lục B. Khởi tạo và migration schema PostgreSQL bằng psql	Error! Bookmark not defined.
Các nguồn tham khảo	107

Danh mục từ viết tắt

Từ viết tắt	Ý nghĩa
AI	Artificial Intelligence - Trí tuệ nhân tạo
ML	Machine Learning - Học máy
DBMS	Database Management System - Hệ quản trị cơ sở dữ liệu
CSV	Comma-Separated Values
ETL	Extract, Transform, Load
KNN	K-Nearest Neighbors
IF	Isolation Forest
SQL	Structured Query Language
F1-score	Chỉ số trung hòa giữa precision và recall

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

Chương 1 trình bày tổng quan về đề tài, bao gồm lý do chọn đề tài, bài toán cần giải quyết, mục tiêu nghiên cứu, đối tượng và phạm vi thực hiện. Đồng thời, chương này cũng nêu định hướng tiếp cận của hệ thống, ý nghĩa thực tiễn và các tiêu chí đánh giá cơ bản được sử dụng trong quá trình thực nghiệm.

Nội dung chương được xây dựng theo mạch từ vấn đề thực tế đến định hướng giải quyết. Trước hết, chương làm rõ vì sao chất lượng dữ liệu là một vấn đề quan trọng trong các hệ thống thông tin nói chung và dữ liệu giao dịch thương mại điện tử nói riêng. Tiếp theo, chương xác định mục tiêu, phạm vi và sản phẩm đầu ra của đề án. Sau đó, chương trình bày phương pháp thực hiện, cách kết hợp giữa rule-based validation và machine learning, cuối cùng là các tiêu chí dùng để đánh giá hệ thống. Cách sắp xếp này giúp người đọc hiểu được lý do hình thành đề tài trước khi đi vào lý thuyết và thiết kế ở các chương sau.

1.1. Đặt vấn đề

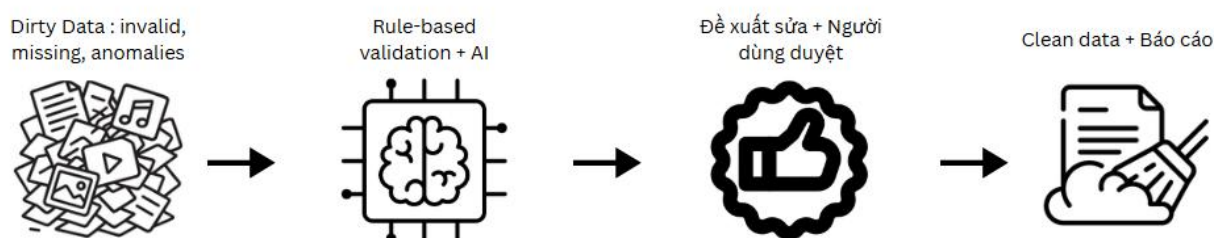
Dữ liệu là một trong những tài sản quan trọng của các hệ thống thông tin hiện đại. Trong các tổ chức, dữ liệu được sử dụng để lập báo cáo, phân tích hành vi người dùng, xây dựng mô hình dự đoán, hỗ trợ ra quyết định và tối ưu hoạt động vận hành. Tuy nhiên, giá trị của dữ liệu chỉ thực sự được khai thác khi dữ liệu có chất lượng đủ tốt. Nếu dữ liệu bị thiếu, sai định dạng, sai miền giá trị hoặc chứa các bản ghi bất thường, kết quả phân tích phía sau có thể bị sai lệch.

Trong lĩnh vực thương mại điện tử, dữ liệu giao dịch thường bao gồm thông tin khách hàng, sản phẩm, số lượng mua, giá trị giao dịch, phương thức thanh toán, thời gian mua hàng và địa điểm mua sắm. Đây là nhóm dữ liệu có tính thực tiễn cao, nhưng cũng dễ xuất hiện lỗi trong quá trình nhập liệu, chuyển đổi, tích hợp hoặc lưu trữ. Ví dụ, một giao dịch có thể bị thiếu phương thức thanh toán, tuổi khách hàng có thể vượt ngoài miền hợp lý, số lượng sản phẩm có thể nhận giá trị âm, hoặc một giao dịch có giá trị quá khác biệt so với các giao dịch cùng nhóm sản phẩm.

Các phương pháp kiểm tra dữ liệu truyền thống thường dựa vào thao tác thủ công hoặc các luật kiểm tra cố định. Cách tiếp cận này phù hợp với những lỗi rõ ràng, nhưng còn hạn chế khi cần phát hiện các điểm bất thường phụ thuộc vào ngữ cảnh dữ liệu. Chẳng hạn, một mức giá có thể hợp lệ về mặt kiểu dữ liệu, không âm và không rỗng,

nhưng vẫn bất thường nếu nó lệch quá xa so với giá trị thường gặp của cùng nhóm sản phẩm. Đây là lý do cần kết hợp giữa kiểm tra dựa trên luật và các phương pháp học máy để nâng cao khả năng phát hiện lỗi dữ liệu.

Từ thực tế đó, đề tài được thực hiện với mục tiêu xây dựng một hệ thống hỗ trợ phát hiện và gợi ý làm sạch dữ liệu trong cơ sở dữ liệu giao dịch thương mại điện tử. Hệ thống không chỉ dừng lại ở việc đọc dữ liệu và kiểm tra lỗi đơn giản, mà còn thiết kế một pipeline xử lý có khả năng chuẩn hóa dữ liệu, phát hiện lỗi, sinh gợi ý sửa, lưu kết quả và tạo dữ liệu đầu ra đã được xử lý.



Hình 1.1. Minh họa bài toán chất lượng dữ liệu

1.2. Lý do chọn đề tài

Lý do đầu tiên để lựa chọn đề tài là tính phổ biến của bài toán chất lượng dữ liệu. Trong hầu hết các hệ thống thông tin, dữ liệu đầu vào không phải lúc nào cũng hoàn chỉnh và chính xác. Dữ liệu có thể bị lỗi do con người nhập sai, do hệ thống tích hợp không đồng nhất, do định dạng dữ liệu thay đổi hoặc do quá trình xử lý trung gian phát sinh sai lệch. Nếu không có cơ chế phát hiện và xử lý phù hợp, các lỗi này có thể tồn tại lâu dài trong cơ sở dữ liệu.

Lý do thứ hai là nhu cầu kết hợp AI vào các quy trình xử lý dữ liệu. AI không nhất thiết thay thế hoàn toàn các luật kiểm tra truyền thống, nhưng có thể hỗ trợ phát hiện những trường hợp khó mô tả bằng luật cố định. Trong phạm vi đề án, các lỗi missing và invalid được xử lý bằng rule-based validation, còn anomaly detection được thực hiện bằng mô hình Isolation Forest. Cách tiếp cận này giúp hệ thống vừa giữ được tính rõ ràng

của luật nghiệp vụ, vừa tận dụng được khả năng phát hiện bất thường của mô hình học máy.

Lý do thứ ba là đề tài có sự kết hợp giữa thiết kế cơ sở dữ liệu, xử lý dữ liệu, lập trình Python, cấu hình hệ thống, học máy và đánh giá thực nghiệm. Nhờ đó, đề án có thể thể hiện được cả năng lực xây dựng hệ thống lẫn năng lực phân tích bài toán dữ liệu.

Ngoài ba lý do trên, đề tài còn có ý nghĩa ở chỗ đặt bài toán AI trong một quy trình dữ liệu hoàn chỉnh. Trong nhiều trường hợp, khi nhắc đến AI, người học dễ tập trung vào mô hình mà bỏ qua các bước nền tảng như chuẩn hóa dữ liệu, thiết kế schema, kiểm tra rule, lưu vết kết quả và đánh giá đầu ra. Tuy nhiên, trong một hệ thống thực tế, mô hình AI chỉ là một thành phần trong toàn bộ pipeline dữ liệu. Nếu dữ liệu đầu vào không được chuẩn hóa hoặc kết quả phát hiện lỗi không được lưu lại rõ ràng, hệ thống sẽ khó vận hành và khó kiểm chứng.

Vì vậy, đề tài được lựa chọn không chỉ để thử nghiệm một thuật toán phát hiện bất thường, mà còn để xây dựng một quy trình xử lý dữ liệu có tổ chức. Quy trình này bắt đầu từ dữ liệu thô, đi qua bước thiết kế cơ sở dữ liệu, tiền xử lý, phát hiện lỗi, gợi ý sửa lỗi, áp dụng sửa lỗi có kiểm soát và cuối cùng là sinh báo cáo. Đây là mạch xuyên suốt của toàn bộ đề án và cũng là lý do các chương sau được sắp xếp theo thứ tự từ tổng quan, lý thuyết, thiết kế, cài đặt đến thực nghiệm.

1.3. Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống AI-assisted data cleaning có khả năng phát hiện dữ liệu thiếu, dữ liệu sai luật và dữ liệu bất thường trong tập dữ liệu giao dịch thương mại điện tử, đồng thời đưa ra gợi ý sửa lỗi và lưu kết quả xử lý vào cơ sở dữ liệu. Các mục tiêu cụ thể bao gồm:

- 1. Chuẩn hóa dữ liệu giao dịch từ tập dữ liệu dạng bảng, bao gồm chuẩn hóa tên cột, kiểu dữ liệu và các trường dẫn xuất.*
- 2. Thiết kế cơ sở dữ liệu PostgreSQL theo hướng tách dữ liệu nghiệp vụ thành các bảng quan hệ gồm khách hàng, sản phẩm, trung tâm mua sắm và giao dịch.*
- 3. Xây dựng pipeline phát hiện lỗi missing, invalid và anomaly.*
- 4. Áp dụng Isolation Forest để phát hiện các bản ghi bất thường theo ngưỡng dữ liệu.*
- 5. Áp dụng nearest neighbors để tìm bản ghi tương tự và gợi ý giá trị phù hợp cho các trường phân loại hoặc văn bản.*

-
6. Sinh các tập dữ liệu đầu ra như *detected_issues*, *fix_recommendations*, *dataset_profile*, *fixed_transactions* và *final_report*.
 7. Thực nghiệm hệ thống trên dữ liệu mẫu và dữ liệu đầy đủ, sau đó đánh giá kết quả bằng các chỉ số thống kê và thời gian xử lý.

Từ các mục tiêu trên, đề án không chỉ hướng tới việc tạo ra một chương trình phát hiện lỗi đơn lẻ, mà hướng tới một quy trình xử lý dữ liệu có thể vận hành theo từng lần chạy. Mỗi lần chạy pipeline cần tạo ra kết quả có thể lưu trữ, kiểm tra lại và so sánh với các lần chạy sau. Đây là điểm quan trọng vì trong thực tế, làm sạch dữ liệu thường không phải là một thao tác duy nhất, mà là một quá trình lặp lại khi dữ liệu mới được bổ sung hoặc khi các quy tắc nghiệp vụ thay đổi.

Đề tài cũng đặt ra yêu cầu phân biệt rõ giữa ba khái niệm: lỗi chắc chắn, điểm nghi vấn và gợi ý sửa. Lỗi chắc chắn là những trường hợp vi phạm luật rõ ràng như thiếu giá trị bắt buộc hoặc giá trị không thuộc miền hợp lệ. Điểm nghi vấn là những bản ghi bất thường theo mô hình học máy, có thể là lỗi nhưng cũng có thể là giao dịch hợp lệ hiếm gặp. Gợi ý sửa là đề xuất của hệ thống dựa trên luật hoặc các bản ghi tương tự, không phải trong mọi trường hợp đều được tự động áp dụng. Việc phân biệt ba khái niệm này giúp hệ thống có tính an toàn hơn khi xử lý dữ liệu.

1.3.1. Câu hỏi nghiên cứu

Để làm rõ phạm vi và hướng triển khai, đề án tập trung trả lời các câu hỏi nghiên cứu sau:

1. Làm thế nào để chuẩn hóa một bảng dữ liệu giao dịch phẳng thành mô hình dữ liệu quan hệ phù hợp cho lưu trữ và phân tích?
2. Làm thế nào để phát hiện đồng thời các lỗi missing, invalid và anomaly trong cùng một pipeline?
3. Khi nào nên dùng rule-based validation và khi nào nên dùng mô hình học máy trong bài toán làm sạch dữ liệu?
4. Làm thế nào để đưa ra gợi ý sửa lỗi dựa trên dữ liệu tham chiếu thay vì sửa cứng theo một giá trị mặc định?
5. Làm thế nào để lưu lại kết quả phát hiện lỗi, gợi ý sửa lỗi và dữ liệu sau xử lý để phục vụ kiểm tra sau này?

8. Hệ thống có thể chạy trên tập dữ liệu gần 100.000 dòng với thời gian chấp nhận được trong phạm vi đồ án hay không?

Những câu hỏi này được sử dụng làm cơ sở để thiết kế hệ thống và đánh giá kết quả thực nghiệm. Trong đó, câu hỏi thứ ba là điểm trọng tâm của đồ án. Nếu chỉ dùng rule-based validation, hệ thống dễ giải thích nhưng khó phát hiện các bất thường theo ngữ cảnh. Nếu chỉ dùng học máy, hệ thống có thể phát hiện bất thường nhưng thiếu tính minh bạch và khó đảm bảo an toàn khi sửa dữ liệu. Vì vậy, đề tài lựa chọn hướng kết hợp, trong đó rule-based validation xử lý các lỗi chắc chắn, còn AI/ML hỗ trợ phát hiện điểm nghi vấn và gợi ý sửa.

1.3.2. Sản phẩm đầu ra của đề tài

Sản phẩm đầu ra của đồ án gồm 2 nhóm: sản phẩm phần mềm và sản phẩm dữ liệu.

Về phần mềm, hệ thống bao gồm mã nguồn Python cho pipeline xử lý dữ liệu, file cấu hình YAML, script import dữ liệu, schema PostgreSQL và giao diện dòng lệnh CLI. Người dùng có thể chạy hệ thống từ CSV để kiểm tra nhanh hoặc chạy từ PostgreSQL để mô phỏng quy trình làm việc gần với cơ sở dữ liệu thực tế.

Về dữ liệu, hệ thống sinh ra các output quan trọng như sau:

Bảng 1.1. Các đầu ra chính của hệ thống

Output	Mục đích sử dụng
detected_issues	Lưu danh sách lỗi và điểm bất thường được phát hiện
fix_recommendations	Lưu gợi ý sửa lỗi tương ứng với từng issue
dataset_profile	Tổng hợp chất lượng dữ liệu theo từng cột
fixed_transactions	Lưu phiên bản dữ liệu giao dịch sau khi áp dụng sửa lỗi phù hợp
final_report	Tóm tắt số lượng lỗi, mức độ nghiêm trọng, confidence và tỷ lệ có thể sửa

Các output này giúp hệ thống có khả năng giải thích và kiểm tra lại. Người dùng không chỉ biết rằng dữ liệu có lỗi, mà còn biết lỗi nằm ở dòng nào, cột nào, do phương pháp nào phát hiện, mức độ nghiêm trọng ra sao và hệ thống đề xuất xử lý như thế nào.

1.4. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là dữ liệu dạng bảng trong bối cảnh giao dịch thương mại điện tử. Tập dữ liệu sử dụng trong đồ án gồm các trường như mã hóa đơn, mã khách hàng, giới tính, tuổi, danh mục sản phẩm, số lượng, giá, phương thức thanh toán, ngày giao dịch và trung tâm mua sắm.

Phạm vi nghiên cứu tập trung vào ba nhóm lỗi chính:

1. Dữ liệu thiếu, ví dụ thiếu tuổi, thiếu giá, thiếu phương thức thanh toán hoặc thiếu ngày giao dịch.
2. Dữ liệu sai luật, ví dụ tuổi vượt miền hợp lý, số lượng nhỏ hơn 1, tháng năm ngoài khoảng 1 đến 12 hoặc giá trị phân loại không thuộc danh sách cho phép.
3. Dữ liệu bất thường, ví dụ giao dịch có đặc trưng khác biệt rõ so với các giao dịch cùng nhóm sản phẩm hoặc cùng ngữ cảnh.

Trong phạm vi đồ án, hệ thống ưu tiên gợi ý và tự động sửa có kiểm soát đối với các lỗi có độ tin cậy cao, đặc biệt là lỗi ở nhóm dữ liệu categorical/text. Với dữ liệu số bất thường, hệ thống chủ yếu ghi nhận, giải thích và khuyến nghị người dùng kiểm tra thủ công, nhằm tránh việc tự động sửa sai các giá trị có thể là giao dịch hợp lệ nhưng hiếm gặp.

Đề tài không đi sâu vào xây dựng một nền tảng quản trị dữ liệu hoàn chỉnh cho doanh nghiệp. Các chức năng như phân quyền người dùng, giao diện duyệt lỗi, workflow phê duyệt nhiều cấp hoặc đồng bộ dữ liệu thời gian thực chưa được triển khai trong phạm vi hiện tại. Tuy nhiên, thiết kế bảng `cleaning_runs`, `detected_issues`, `fix_recommendations` và `cleaning_actions` đã tạo nền tảng để mở rộng các chức năng này trong tương lai.

Một giới hạn khác là dataset không có mã sản phẩm chi tiết ở mức SKU. Vì vậy, bảng `products` trong đồ án đại diện cho danh mục sản phẩm (category) thay vì từng sản phẩm cụ thể. Cách thiết kế này vẫn phù hợp với dữ liệu hiện có, nhưng khi triển khai trong hệ thống thương mại điện tử thật, bảng sản phẩm cần được mở rộng thêm các trường như mã SKU, tên sản phẩm, thương hiệu, nhà cung cấp và trạng thái kinh doanh.

1.5. Phương pháp thực hiện

Đề tài được thực hiện theo hướng xây dựng hệ thống thực nghiệm. Trước hết, dữ liệu đầu vào được khảo sát, chuẩn hóa và tách thành các bảng quan hệ phù hợp với bài toán e-commerce. Sau đó, hệ thống được thiết kế thành pipeline gồm các bước đọc dữ

liệu, chuẩn hóa, sinh feature, kiểm tra dữ liệu bằng luật, xây dựng tập tham chiếu sạch, phát hiện bất thường, phân tích đặc trưng nghi ngờ, sinh gợi ý sửa và xuất báo cáo.

Về mặt thuật toán, hệ thống sử dụng rule-based validation cho các lỗi có thể mô tả rõ bằng điều kiện. Ví dụ, trường gender chỉ chấp nhận Female hoặc Male, trường quantity phải lớn hơn hoặc bằng 1, trường invoice_month phải nằm trong khoảng từ 1 đến 12. Đối với anomaly detection, hệ thống sử dụng Isolation Forest, một phương pháp học không giám sát phù hợp cho phát hiện điểm bất thường trong dữ liệu dạng bảng [1]. Đối với gợi ý sửa dữ liệu phân loại, hệ thống sử dụng nearest neighbors để tìm những bản ghi tương tự và suy ra giá trị hợp lý từ tập tham chiếu [2].

Về mặt công nghệ, hệ thống được xây dựng bằng Python, sử dụng pandas để xử lý dữ liệu dạng bảng, scikit-learn cho thuật toán học máy, PostgreSQL để lưu dữ liệu và SQLAlchemy để kết nối giữa Python với cơ sở dữ liệu [3], [4], [5], [6].

Quy trình thực hiện đề tài có thể chia thành bảy giai đoạn. Giai đoạn đầu tiên là khảo sát dữ liệu, xác định các trường dữ liệu, kiểu dữ liệu và ý nghĩa nghiệp vụ. Giai đoạn thứ hai là chuẩn hóa dữ liệu và thiết kế lại mô hình quan hệ. Giai đoạn thứ ba là xây dựng rule validation cho missing, invalid và cross-field inconsistency. Giai đoạn thứ tư là xây dựng feature cho mô hình anomaly detection. Giai đoạn thứ năm là tích hợp Isolation Forest và phân tích feature contribution. Giai đoạn thứ sáu là xây dựng recommender để gợi ý sửa lỗi. Giai đoạn cuối cùng là thực nghiệm, ghi nhận kết quả và đánh giá.

Trong quá trình triển khai, đề tài ưu tiên tính giải thích của hệ thống. Mỗi issue cần có reason và source_method để người dùng biết lỗi đến từ luật nào hoặc mô hình nào. Điều này đặc biệt quan trọng với anomaly detection, vì mô hình học máy có thể phát hiện bản ghi bất thường nhưng nếu không giải thích được cột nghi ngờ, người dùng sẽ khó tin tưởng kết quả.

1.5.1. So sánh các hướng tiếp cận

Để lựa chọn phương pháp phù hợp, có thể so sánh ba hướng tiếp cận chính trong bài toán làm sạch dữ liệu (Data Cleaning):

Bảng 1.2. So sánh các hướng tiếp cận trong đồ án

Hướng tiếp cận	Ưu điểm	Hạn chế	Vai trò trong đồ án
Kiểm tra thủ công	Linh hoạt, có tri thức con người	Khó lặp lại, dễ thiếu nhất quán và tốn thời gian	Chỉ dùng ở bước review kết quả
Rule-based validation	Dễ hiểu, dễ giải thích, phù hợp lỗi rõ ràng	Khó phát hiện bất thường theo ngữ cảnh	Dùng cho missing, invalid, cross-field
Machine learning	Có thể phát hiện pattern bất thường khó viết rule	Có false positive, cần giải thích và kiểm soát	Dùng cho anomaly detection và hỗ trợ recommendation

Từ bảng so sánh trên, có thể thấy không có phương pháp nào phù hợp tuyệt đối cho mọi loại lỗi dữ liệu. Do đó, hệ thống được thiết kế theo hướng hybrid. Rule-based validation đóng vai trò như lớp kiểm tra chắc chắn ban đầu. Machine learning đóng vai trò như lớp phát hiện các điểm nghi vấn bổ sung. Recommender đóng vai trò hỗ trợ người dùng hoặc hệ thống chọn giá trị sửa phù hợp hơn.

1.6. Đóng góp chính của đề tài

Đồ án có một số đóng góp chính như sau:

1. Xây dựng được pipeline xử lý dữ liệu có thể chạy từ nguồn CSV hoặc PostgreSQL.
2. Mô hình hóa dữ liệu giao dịch e-commerce từ một bảng phẳng thành các bảng quan hệ có ý nghĩa nghiệp vụ.
3. Kết hợp rule-based validation và học máy để phát hiện nhiều nhóm lỗi dữ liệu khác nhau.
4. Thiết kế các bảng lưu vết quá trình làm sạch dữ liệu, giúp hệ thống có khả năng kiểm tra lại kết quả sau mỗi lần chạy.
5. Cung cấp đầu ra gồm báo cáo lỗi, gợi ý sửa lỗi, hồ sơ chất lượng dữ liệu và dữ liệu sau khi được xử lý.
6. Thực nghiệm trên dữ liệu mẫu và dữ liệu đầy đủ, từ đó rút ra nhận xét về kết quả phát hiện lỗi và hiệu năng của hệ thống.

1.7. Tiêu chí đánh giá hệ thống

Hệ thống được đánh giá dựa trên các tiêu chí sau:

1. Tính đúng đắn chức năng: hệ thống phải đọc được dữ liệu, chuẩn hóa dữ liệu, phát hiện lỗi và sinh output đúng định dạng.
2. Khả năng phát hiện lỗi: hệ thống cần phát hiện được các lỗi missing, invalid và anomaly theo thiết kế.
3. Khả năng gợi ý sửa lỗi: với những lỗi có thể sửa, hệ thống cần đưa ra gợi ý hợp lý và lưu lại trong bảng kết quả.
4. Khả năng truy vết: các lỗi, gợi ý và dữ liệu sau xử lý cần được lưu lại để kiểm tra.
5. Hiệu năng: hệ thống cần chạy được trên dữ liệu sample và dữ liệu đầy đủ với thời gian chấp nhận được trong phạm vi đồ án.
6. Tính mở rộng: cấu hình rule, mô hình và nguồn dữ liệu cần được thiết kế sao cho có thể điều chỉnh khi mở rộng sang dataset khác.

1.8. Kết luận chương

Chương 1 đã trình bày tổng quan về đề tài, lý do lựa chọn, mục tiêu, phạm vi và hướng tiếp cận của hệ thống. Có thể thấy bài toán làm sạch dữ liệu không chỉ là một bước tiền xử lý đơn giản, mà là một thành phần quan trọng trong các hệ thống khai thác dữ liệu. Đối với dữ liệu giao dịch thương mại điện tử, các lỗi missing, invalid và anomaly có thể ảnh hưởng trực tiếp đến báo cáo, phân tích hành vi khách hàng và các mô hình dự đoán phía sau.

Từ các phân tích trên, đồ án lựa chọn hướng tiếp cận hybrid, kết hợp rule-based validation và machine learning. Rule-based validation đảm nhiệm các lỗi có thể mô tả rõ bằng luật, còn machine learning hỗ trợ phát hiện các điểm bất thường khó định nghĩa trước. Bên cạnh đó, PostgreSQL được sử dụng để lưu trữ dữ liệu chuẩn hóa và kết quả xử lý, giúp hệ thống có khả năng truy vết thay vì chỉ tạo ra một file dữ liệu đã sửa.

Những nội dung trong Chương 1 là nền tảng để đi vào Chương 2. Nếu Chương 1 trả lời câu hỏi "vì sao cần thực hiện đề tài", thì Chương 2 sẽ trả lời câu hỏi "đề tài dựa trên cơ sở lý thuyết và công nghệ nào". Từ đó, các khái niệm như chất lượng dữ liệu, rule-based validation, anomaly detection, Isolation Forest, nearest neighbors và các chỉ số đánh giá sẽ được trình bày cụ thể hơn.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ LIÊN QUAN

Chương 2 trình bày các kiến thức nền tảng được sử dụng trong đề án. Nội dung chương bao gồm tổng quan về chất lượng dữ liệu, các loại lỗi dữ liệu, rule-based validation, phát hiện bất thường, thuật toán Isolation Forest, nearest neighbors, các chỉ số đánh giá và các công nghệ chính được sử dụng trong quá trình xây dựng hệ thống.

Các nội dung trong chương này được sắp xếp theo đúng thứ tự xuất hiện trong pipeline của hệ thống. Trước hết, báo cáo trình bày khái niệm chất lượng dữ liệu và các loại lỗi thường gặp để làm rõ đối tượng cần xử lý. Sau đó, báo cáo trình bày rule-based validation vì đây là lớp phát hiện lỗi đầu tiên trong hệ thống. Tiếp theo là anomaly detection và Isolation Forest, tương ứng với lớp phát hiện bất thường bằng học máy. Sau đó, nearest neighbors được trình bày như cơ sở cho cơ chế gợi ý sửa lỗi. Cuối cùng, chương giới thiệu các công nghệ triển khai và chỉ số đánh giá. Nhờ cách sắp xếp này, phần lý thuyết không tách rời hệ thống mà gắn trực tiếp với các quyết định thiết kế ở Chương 3.

2.1. Chất lượng dữ liệu

Chất lượng dữ liệu thể hiện mức độ dữ liệu đáp ứng được yêu cầu sử dụng trong một ngữ cảnh cụ thể. Một tập dữ liệu có chất lượng tốt cần phản ánh đúng đối tượng thực tế, đầy đủ các trường cần thiết, nhất quán về định dạng và có thể sử dụng được cho phân tích hoặc vận hành. Trong thực tế, khái niệm chất lượng dữ liệu thường được xem xét qua nhiều chiều như độ đầy đủ, độ chính xác, tính nhất quán, tính hợp lệ, tính duy nhất và tính kịp thời [7].

Trong phạm vi đề án, ba chiều chất lượng dữ liệu được quan tâm nhiều nhất là completeness, validity và plausibility/anomaly. Completeness liên quan đến việc dữ liệu có bị thiếu hay không. Validity liên quan đến việc giá trị có thuộc miền hợp lệ hay không. Plausibility hoặc anomaly liên quan đến việc một giá trị tuy có thể hợp lệ về mặt định dạng nhưng lại bất thường khi so sánh với bối cảnh dữ liệu xung quanh.

Các chiều chất lượng dữ liệu được áp dụng trong đề án có thể mô tả như sau:

Bảng 2.1. Các chiều chất lượng dữ liệu

Chiều chất lượng	Ý nghĩa	Ví dụ trong dataset
Completeness	Dữ liệu có đầy đủ các	Thiếu price, quantity,

Chiều chất lượng	Ý nghĩa	Ví dụ trong dataset
	trường cần thiết hay không	invoice_date
Validity	Giá trị có thuộc miền hợp lệ hay không	payment_method không thuộc Cash/Credit Card/Debit Card
Consistency	Các trường liên quan có mâu thuẫn nhau hay không	unit_price không khớp với price / quantity
Accuracy	Dữ liệu có phản ánh đúng thực tế hay không	Tuổi khách hàng bị nhập sai
Uniqueness	Dữ liệu có bị trùng lặp ngoài mong muốn hay không	Một invoice_no xuất hiện nhiều lần
Timeliness	Dữ liệu có đúng thời điểm và còn giá trị sử dụng hay không	Ngày giao dịch bị parse sai năm
Plausibility	Giá trị có hợp lý trong ngữ cảnh hay không	Giá trị giao dịch quá lệch so với cùng category

Trong các chiều trên, accuracy là chiều khó đánh giá nhất vì cần đối chiếu với thực tế bên ngoài. Ví dụ, hệ thống có thể biết tuổi 150 là không hợp lý theo rule, nhưng nếu tuổi 45 bị nhập thành 46 thì hệ thống rất khó xác định nếu không có nguồn tham chiếu. Vì vậy, trong phạm vi đề án, hệ thống tập trung vào các lỗi có thể phát hiện từ chính dữ liệu và rule cấu hình.

2.1.1. Vai trò của data profiling

Data profiling là quá trình thống kê và mô tả dữ liệu để hiểu cấu trúc, phân phối và chất lượng của từng cột. Một số thông tin thường được quan tâm gồm số dòng, số cột, số giá trị thiếu, tỷ lệ thiếu, số lượng giá trị duy nhất, giá trị nhỏ nhất, giá trị lớn nhất và số lỗi phát hiện được theo từng cột. Các thống kê mô tả này tạo cơ sở ban đầu để nhận biết phân phối, độ phân tán và những quan sát khác biệt cần được phân tích sâu hơn [13].

Trong hệ thống, data profiling được thể hiện qua bảng dataset_profile. Bảng này không trực tiếp sửa lỗi, nhưng có vai trò quan trọng trong việc đánh giá tổng quan chất lượng dữ liệu. Ví dụ, nếu một cột có tỷ lệ thiếu cao, người dùng có thể cần kiểm tra lại nguồn nhập dữ liệu. Nếu một cột có nhiều anomaly, người dùng có thể cần xem lại feature hoặc rule liên quan đến cột đó.

Data profiling cũng giúp so sánh chất lượng dữ liệu giữa các lần chạy pipeline. Khi hệ thống được mở rộng thêm bằng `cleaning_runs`, mỗi lần chạy có thể gắn với một `run_id`. Khi đó, người dùng có thể theo dõi số lỗi giảm hay tăng theo thời gian, từ đó đánh giá hiệu quả của quá trình làm sạch dữ liệu.

2.2. Các loại lỗi dữ liệu thường gặp

Dữ liệu thiếu là loại lỗi xuất hiện khi một trường bắt buộc không có giá trị. Ví dụ, một giao dịch thiếu `invoice_date` sẽ gây khó khăn cho việc phân tích theo thời gian; một giao dịch thiếu `price` hoặc `quantity` sẽ ảnh hưởng đến việc tính toán giá trị giao dịch.

Dữ liệu sai luật là loại lỗi xuất hiện khi giá trị không thỏa mãn điều kiện đã định nghĩa. Ví dụ, `quantity` phải lớn hơn hoặc bằng 1, `age` không nên vượt quá miền hợp lý, `payment_method` chỉ nên thuộc các giá trị như `Cash`, `Credit Card` hoặc `Debit Card`.

Dữ liệu mâu thuẫn liên trường là loại lỗi xuất hiện khi từng trường riêng lẻ có thể hợp lệ, nhưng khi xét cùng nhau lại không hợp lý. Ví dụ, `unit_price` được tính từ `price / quantity`; nếu `price` hoặc `quantity` thay đổi nhưng `unit_price` không tương ứng, bản ghi có thể bị nghi ngờ.

Dữ liệu bất thường là loại lỗi hoặc điểm nghi vấn xuất hiện khi một bản ghi khác biệt đáng kể so với các bản ghi còn lại. Khác với `invalid data`, `anomaly data` không nhất thiết sai tuyệt đối. Một giao dịch có giá rất cao có thể là dữ liệu sai, nhưng cũng có thể là một giao dịch thật có giá trị lớn. Vì vậy, hệ thống cần đánh dấu và đưa ra lý do để người dùng xem xét thay vì tự động sửa trong mọi trường hợp.

Trong hệ thống, ba nhóm lỗi này được xử lý theo thứ tự có chủ đích. `Missing` và `invalid` được phát hiện trước vì đây là các lỗi chắc chắn hơn và có thể ảnh hưởng đến bước huấn luyện mô hình. Sau đó, hệ thống tạo `clean reference set` bằng cách loại các dòng đã có lỗi rõ ràng. Tập tham chiếu này được dùng để huấn luyện `anomaly detector` và `recommender`. Cách làm này giúp mô hình học từ dữ liệu đáng tin cậy hơn, hạn chế việc các dòng lỗi rõ ràng ảnh hưởng đến kết quả phát hiện bất thường.

Một điểm cần lưu ý là cùng một bản ghi có thể có nhiều issue. Ví dụ, một giao dịch có thể vừa thiếu `quantity`, vừa có `category invalid`, vừa bị xem là `anomaly` do tổ hợp các feature khác thường. Vì vậy, hệ thống không thiết kế mỗi dòng chỉ có một nhãn lỗi duy nhất, mà cho phép một dòng có nhiều issue ở nhiều cột khác nhau. Cách biểu diễn này phù hợp hơn với bài toán làm sạch dữ liệu thực tế.

2.3. Rule-based validation

Rule-based validation là phương pháp kiểm tra dữ liệu dựa trên các luật được định nghĩa trước. Các luật này có thể bao gồm ràng buộc bắt buộc, kiểu dữ liệu, miền giá trị, tập giá trị cho phép hoặc ràng buộc giữa các trường. Đây là phương pháp có tính minh bạch cao vì mỗi lỗi phát hiện được đều có thể giải thích bằng một điều kiện cụ thể.

Trong đồ án, rule-based validation được sử dụng cho các nhóm lỗi rõ ràng như missing và invalid. Các luật được cấu hình trong file YAML thay vì viết cố định trong code. Cách làm này giúp hệ thống dễ điều chỉnh khi thay đổi dataset hoặc mở rộng sang lĩnh vực khác. Ví dụ, danh sách giá trị hợp lệ của category hoặc shopping_mall có thể được cập nhật trong file cấu hình mà không cần sửa logic xử lý chính.

Ưu điểm của rule-based validation là đơn giản, dễ giải thích và phù hợp với ràng buộc nghiệp vụ. Tuy nhiên, hạn chế của phương pháp này là khó bao quát các trường hợp bất thường theo ngữ cảnh. Đối với những lỗi phụ thuộc vào phân phối dữ liệu hoặc quan hệ giữa nhiều đặc trưng, cần kết hợp thêm các phương pháp thống kê hoặc học máy.

Trong đồ án, rule-based validation được chia thành ba nhóm:

7. Missing rule: kiểm tra cột bắt buộc có bị thiếu hoặc rỗng hay không.
8. Invalid rule: kiểm tra kiểu dữ liệu, miền giá trị số và tập giá trị hợp lệ.
9. Cross-field rule: kiểm tra tính nhất quán giữa các trường có quan hệ logic.

Ví dụ với cross-field rule, hệ thống kiểm tra unit_price có nhất quán với price và quantity hay không. Về mặt công thức:

```
expected_price = quantity * unit_price
```

Nếu sai lệch giữa price và expected_price vượt quá ngưỡng dung sai, hệ thống tạo issue cho cột unit_price. Ngưỡng dung sai được dùng để tránh đánh dấu lỗi do sai số làm tròn nhỏ trong tính toán số thực.

Đối với thời gian, hệ thống kiểm tra các feature dẫn xuất từ invoice_date như invoice_year, invoice_month, invoice_day, day_of_week và is_weekend. Nếu các trường này không khớp với ngày giao dịch sau khi parse, hệ thống đánh dấu invalid ở trường tương ứng. Đây là một ví dụ cho thấy làm sạch dữ liệu không chỉ kiểm tra từng cột độc lập mà còn cần kiểm tra quan hệ giữa các cột.

2.3.1. Ưu điểm của cấu hình rule bằng YAML

Thay vì viết toàn bộ rule cố định trong code, hệ thống sử dụng file YAML để mô tả các cột, kiểu dữ liệu, miền giá trị và cờ sử dụng cho detection/recommendation. Cách làm này có ba ưu điểm chính.

Thứ nhất, cấu hình tách khỏi logic xử lý. Khi thay đổi danh sách `allowed_values` của `category` hoặc `payment_method`, người dùng chỉ cần sửa YAML mà không cần sửa code detector.

Thứ hai, YAML giúp báo cáo và bảo vệ dễ hơn. Người đọc có thể nhìn vào file cấu hình để hiểu hệ thống đang kiểm tra những cột nào, cột nào bắt buộc, cột nào được dùng cho machine learning.

Thứ ba, cấu hình giúp hệ thống có khả năng mở rộng. Nếu sau này áp dụng pipeline cho dataset khác, có thể tạo file cấu hình mới với `table_name`, `id_column`, `rules` và `source_query` khác.

2.4. Phát hiện bất thường dữ liệu

Phát hiện bất thường là bài toán xác định các điểm dữ liệu có hành vi khác biệt so với phần lớn dữ liệu còn lại. Trong dữ liệu giao dịch, bất thường có thể đến từ giá trị quá lớn, tổ hợp đặc trưng hiếm gặp hoặc sự sai lệch so với nhóm dữ liệu cùng ngữ cảnh. Bài toán này thường khó hơn kiểm tra rule vì không phải mọi bất thường đều là lỗi.

Các phương pháp anomaly detection có thể được chia thành nhiều nhóm, bao gồm phương pháp thống kê, phương pháp dựa trên khoảng cách, phương pháp dựa trên mật độ và phương pháp học máy. Cách phân loại này phù hợp với nền tảng chung về trí tuệ nhân tạo, học máy và học không giám sát được trình bày trong các tài liệu chuyên ngành [10], [11], [12], [15]. Trong phạm vi đồ án, Isolation Forest được lựa chọn vì phù hợp với dữ liệu dạng bảng, có thể hoạt động theo hướng không giám sát, có cơ sở thuật toán rõ ràng và đã được hỗ trợ sẵn trong scikit-learn [1], [8].

Điểm khác biệt giữa anomaly detection và rule-based validation nằm ở bản chất của tiêu chí đánh giá. Rule-based validation dựa trên các điều kiện xác định trước, ví dụ `quantity >= 1` hoặc `payment_method` thuộc danh sách hợp lệ. Ngược lại, anomaly detection dựa trên sự khác biệt của một bản ghi so với phân phối dữ liệu. Do đó, một bản ghi không vi phạm bất kỳ rule nào vẫn có thể bị xem là bất thường nếu tổ hợp các đặc trưng của nó hiếm gặp.

Trong bối cảnh làm sạch dữ liệu, anomaly detection nên được xem như lớp phát hiện điểm nghi vấn. Lớp này giúp mở rộng khả năng quan sát của hệ thống nhưng không thay thế hoàn toàn vai trò của con người hoặc luật nghiệp vụ. Đây là lý do hệ thống của đồ án không tự động sửa tất cả anomaly, mà chỉ lưu issue, phân tích cột nghi ngờ và đưa ra recommendation trong những trường hợp phù hợp.

Trong dữ liệu giao dịch, anomaly có thể xuất hiện ở nhiều dạng:

1. Point anomaly: một giao dịch có giá trị khác biệt rõ rệt so với toàn bộ dữ liệu.
2. Contextual anomaly: một giao dịch chỉ bất thường khi xét trong ngữ cảnh cụ thể, ví dụ category hoặc shopping mall.
3. Collective anomaly: một nhóm giao dịch có mẫu hành vi bất thường khi xét cùng nhau.

Đồ án tập trung chủ yếu vào point anomaly và contextual anomaly. Việc cấu hình `anomaly_segment_column: category` cho phép hệ thống huấn luyện thêm các mô hình theo từng nhóm category nếu nhóm đó đủ lớn. Nhờ vậy, một giao dịch được đánh giá không chỉ theo toàn bộ dataset mà còn theo nhóm sản phẩm tương ứng. Điều này phù hợp với dữ liệu thương mại điện tử vì các category có phân phối giá và số lượng mua khác nhau.

Một khó khăn của anomaly detection là lựa chọn tỷ lệ bất thường kỳ vọng. Trong cấu hình hiện tại, tham số `contamination` được đặt là 0.08, tức mô hình dự kiến một tỷ lệ nhất định bản ghi có thể là outlier. Tham số này không nên hiểu là tỷ lệ lỗi chắc chắn trong dữ liệu, mà là tỷ lệ cảnh báo của mô hình. Nếu đặt quá cao, hệ thống có thể phát hiện nhiều false positive; nếu đặt quá thấp, hệ thống có thể bỏ sót một số bất thường.

2.5. Isolation Forest

Isolation Forest là thuật toán phát hiện bất thường theo hướng học không giám sát do Liu, Ting và Zhou đề xuất. Khác với nhiều phương pháp phải ước lượng phân phối của dữ liệu bình thường hoặc tính khoảng cách giữa các điểm, Isolation Forest tiếp cận bài toán bằng cách trực tiếp cô lập từng quan sát. Cơ sở của cách tiếp cận này là các điểm bất thường thường xuất hiện với tần suất thấp và có giá trị đặc trưng khác biệt, vì vậy chúng có xu hướng bị tách khỏi phần dữ liệu còn lại sau ít phép phân hoạch ngẫu nhiên hơn [8].

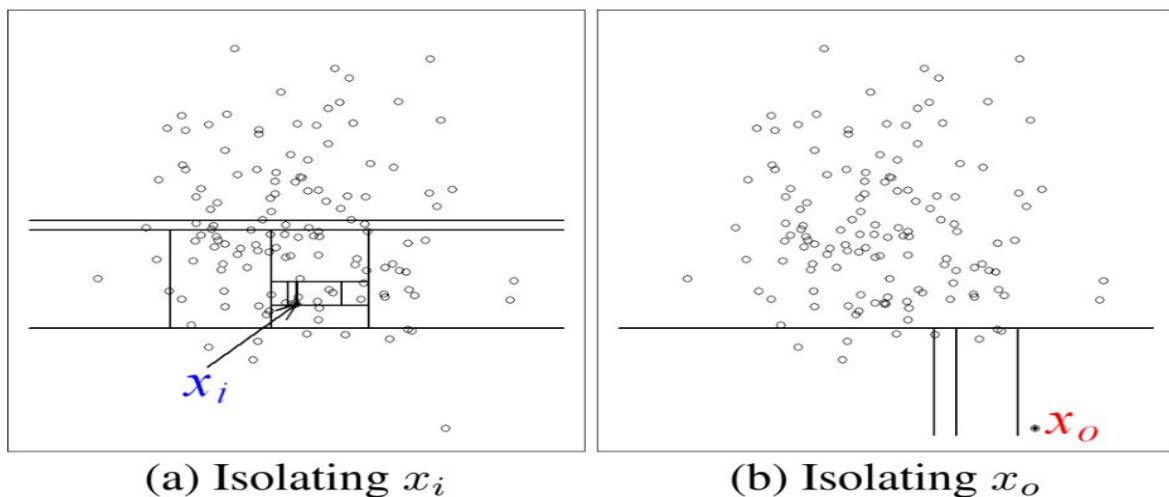
Trong bài toán làm sạch dữ liệu, đặc điểm trên có ý nghĩa thực tiễn vì nhãn lỗi thường không đầy đủ. Hệ thống có thể xác định các bản ghi đáng nghi ngay cả khi chưa

có tập nhãn đủ lớn để huấn luyện mô hình phân loại. Tuy nhiên, kết quả của Isolation Forest chỉ phản ánh mức độ khác biệt về mặt phân phối, không chứng minh một bản ghi chắc chắn sai. Vì vậy, đồ án sử dụng mô hình như lớp phát hiện bổ sung cho rule-based validation và chuyển các anomaly thành đối tượng cần ưu tiên kiểm tra, thay vì tự động xem chúng là lỗi nghiệp vụ [10].

2.5.1. Cơ sở trực giác của cơ chế cô lập

Giả sử tập dữ liệu gồm một vùng quan sát dày đặc và một số điểm nằm xa hoặc nằm trong vùng thưa. Nếu liên tục chọn ngẫu nhiên một đặc trưng rồi chọn ngẫu nhiên một điểm chia trong miền giá trị của đặc trưng đó, một điểm nằm tách biệt thường nhanh chóng rơi vào nhánh chỉ chứa chính nó. Ngược lại, một điểm thuộc vùng dữ liệu dày đặc cần nhiều phép chia hơn vì tại mỗi bước nó vẫn đi cùng nhiều quan sát tương tự.

Do đó, Isolation Forest không hỏi trực tiếp khoảng cách từ x đến một tâm cụm là bao nhiêu, mà hỏi cần bao nhiêu quyết định phân hoạch để cô lập x . Đại lượng này biến quá trình phát hiện bất thường thành bài toán đo độ sâu trên cây. Cách diễn giải bằng số bước cô lập cũng phù hợp với yêu cầu của báo cáo: đường đi càng ngắn thì bằng chứng về sự khác biệt của bản ghi càng mạnh, nhưng quyết định cuối cùng vẫn cần được đặt trong ngữ cảnh nghiệp vụ.



Hình 2.1. Minh họa cơ chế cô lập điểm bình thường và điểm bất thường
(trích từ tài liệu *Isolation_Forest.pdf*)

2.5.2. Cấu trúc Isolation Tree

Thành phần cơ sở của Isolation Forest là Isolation Tree, viết tắt là iTree. Với một tập mẫu S , tại mỗi nút trong cây, thuật toán chọn ngẫu nhiên một đặc trưng q trong tập đặc trưng khả dụng. Sau đó, một điểm chia p được chọn ngẫu nhiên trong khoảng từ giá

trị nhỏ nhất đến giá trị lớn nhất của q trên tập dữ liệu tại nút hiện thời. Các quan sát có giá trị nhỏ hơn p được chuyển sang nhánh trái, các quan sát còn lại được chuyển sang nhánh phải.

Quá trình phân hoạch được thực hiện đệ quy. Một nút trở thành nút ngoài khi chỉ còn tối đa một quan sát, khi mọi quan sát không thể tiếp tục tách theo các đặc trưng đang xét, hoặc khi cây đạt giới hạn chiều cao l . Trong thuật toán gốc, giới hạn thường được đặt theo biểu thức $l = \lceil \log_2 \psi \rceil$, với ψ là kích thước mẫu dùng để xây dựng cây. Giới hạn này kiểm soát chi phí tính toán và ngăn cây tiếp tục phát triển quá sâu đối với các vùng dữ liệu dày đặc [8].

```
Đầu vào: tập mẫu  $S$ , độ sâu hiện tại  $e$ , giới hạn  $h\_max$ 

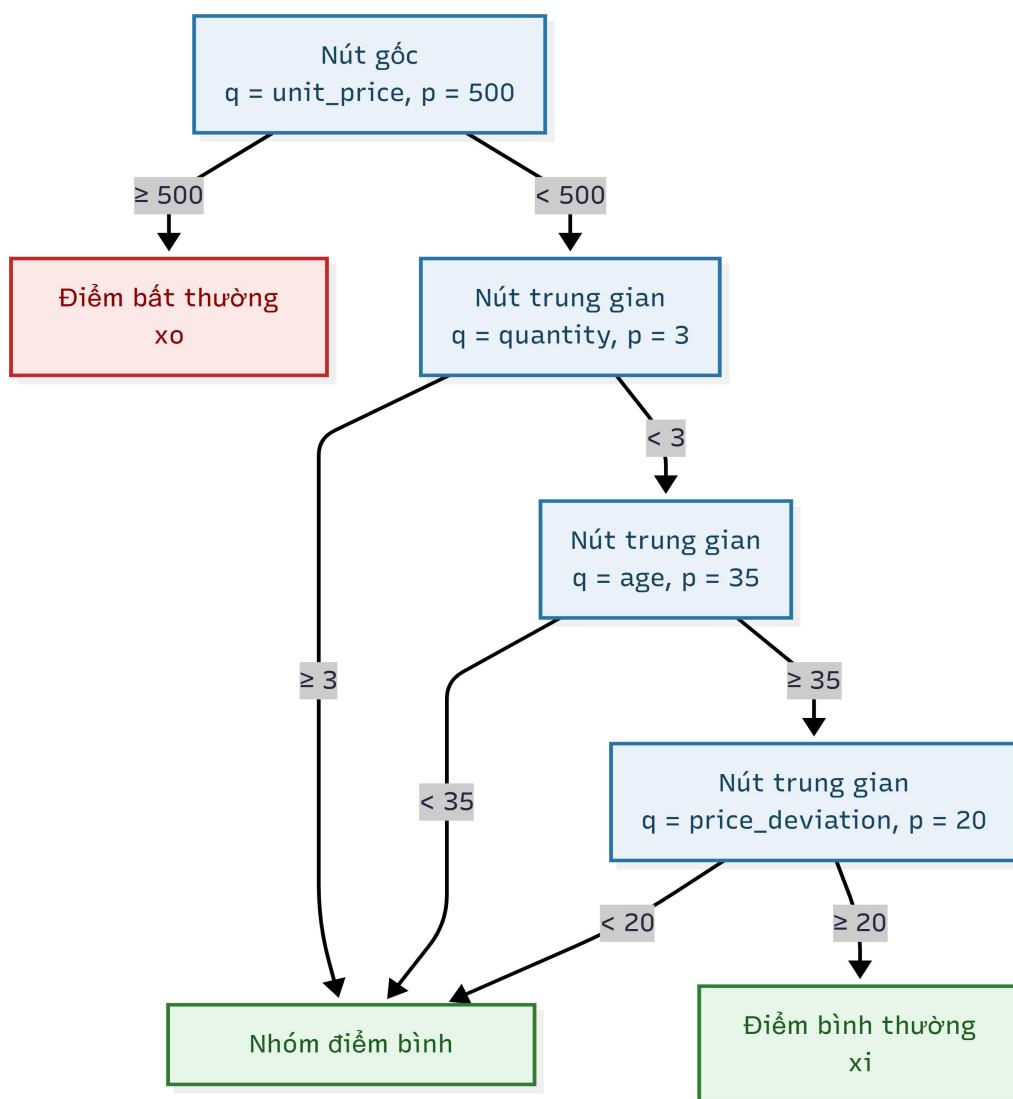
Nếu  $e \geq h\_max$  hoặc  $|S| \leq 1$ :
    Trả về nút lá chứa  $|S|$ 

Chọn ngẫu nhiên đặc trưng  $q$ 
Chọn ngẫu nhiên ngưỡng chia  $p \in [\min(Sq), \max(Sq)]$ 

Sleft  =  $\{x \in S \mid xq < p\}$ 
Sright =  $\{x \in S \mid xq \geq p\}$ 

Trả về nút trong:
    ( $q, p,$ 
      $iTree(Sleft, e + 1, h\_max),$ 
      $iTree(Sright, e + 1, h\_max)$ )
```

Điểm quan trọng là q và p đều được chọn ngẫu nhiên. Một cây đơn lẻ vì vậy có thể tạo ra phép phân hoạch chưa đại diện tốt cho cấu trúc dữ liệu. Isolation Forest khắc phục hạn chế này bằng cách xây dựng nhiều $iTree$ độc lập và tổng hợp độ dài đường đi trên toàn bộ rừng.



Hình 2.2. Cấu trúc một Isolation Tree và độ dài đường đi

2.5.3. Độ dài đường đi và kỳ vọng trên toàn bộ rừng

Với một cây thứ i , độ dài đường đi $h_i(x)$ được xác định bằng số cạnh mà bản ghi x đi qua từ nút gốc đến nút ngoài. Khi x được cô lập ở độ sâu nhỏ, $h_i(x)$ có giá trị thấp. Nếu cây dừng tại một nút ngoài còn chứa nhiều hơn một quan sát do đã đạt giới hạn chiều cao, độ dài đường đi được hiệu chỉnh thêm bằng $c(|S|)$, trong đó $|S|$ là số quan sát còn lại tại nút. Hiệu chỉnh này tránh việc coi mọi quan sát trong một lá chưa được cô lập hoàn toàn là có cùng độ sâu thực tế.

Đối với rừng gồm t cây, độ dài đường đi của x được lấy trung bình để giảm ảnh hưởng của một phép chia ngẫu nhiên riêng lẻ:

$$E[h(x)] = (1/t) \times \sum_{i=1}^t h_i(x)$$

Giá trị $E[h(x)]$ nhỏ cho thấy x thường được cô lập sớm trong nhiều cây khác nhau. Ngược lại, nếu chỉ một số ít cây cô lập x sớm nhưng phần lớn cây vẫn giữ x trong vùng

dày đặc, giá trị trung bình sẽ không quá thấp. Cơ chế tổng hợp này giúp kết quả ổn định hơn so với việc dựa trên một cây duy nhất.

2.5.4. Chuẩn hóa độ dài đường đi và điểm bất thường

Độ dài đường đi phụ thuộc vào số mẫu dùng để xây dựng cây, vì vậy không thể so sánh trực tiếp giữa các mô hình có kích thước mẫu khác nhau. Isolation Forest chuẩn hóa $E[h(x)]$ bằng độ dài đường đi trung bình kỳ vọng của một cây tìm kiếm nhị phân không thành công. Với $n > 2$, đại lượng chuẩn hóa được xác định như sau:

$$c(n) = 2H(n-1) - 2(n-1)/n$$

Trong đó $H(k)$ là số điều hòa bậc k , $H(k) = \sum_{j=1}^k 1/j$ và có thể xấp xỉ bởi $\ln(k) + \gamma$, với γ là hằng số Euler–Mascheroni. Đối với $n = 2$, $c(n) = 1$; đối với $n \leq 1$, $c(n) = 0$. Việc sử dụng $c(n)$ đưa độ dài đường đi về cùng một thang đo tương đối.

Điểm bất thường theo công thức gốc được tính bởi:

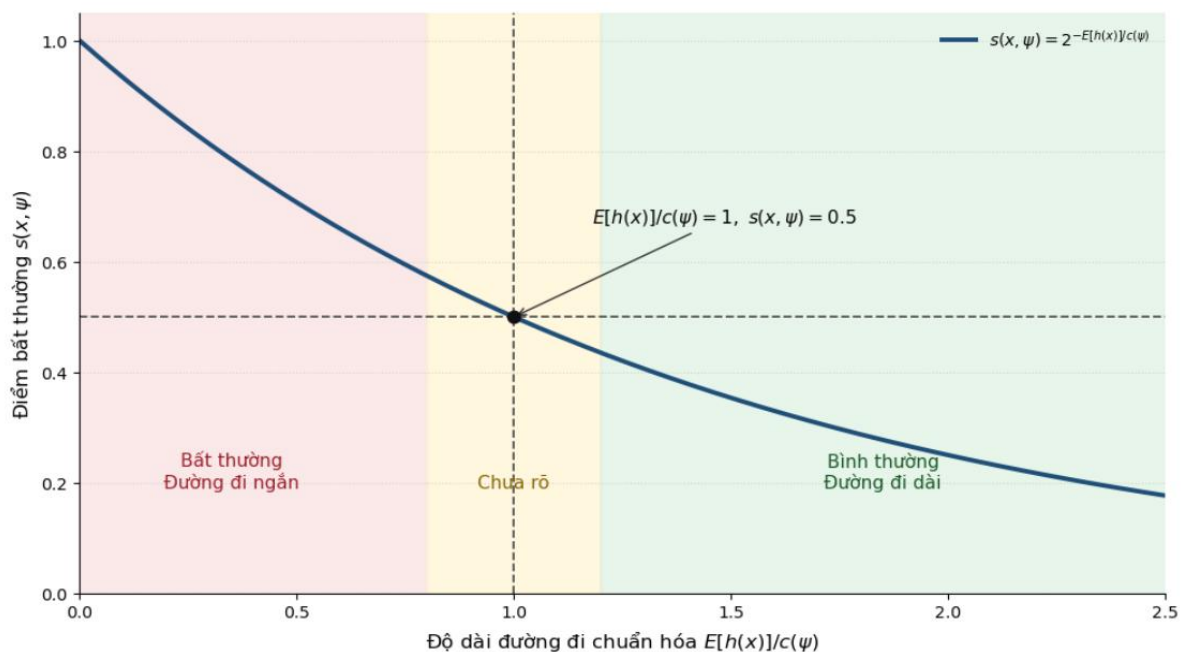
$$s(x, \psi) = 2^{(-E[h(x)] / c(\psi))}$$

Nếu $E[h(x)]$ nhỏ hơn đáng kể so với $c(\psi)$, giá trị $s(x, \psi)$ tiến gần 1 và x có xu hướng là bất thường. Nếu $E[h(x)]$ xấp xỉ $c(\psi)$, điểm số gần 0,5 và kết luận chưa rõ ràng. Nếu $E[h(x)]$ lớn hơn $c(\psi)$, điểm số tiến về 0 và x có xu hướng thuộc vùng bình thường. Cần lưu ý rằng $s(x, \psi)$ là điểm xếp hạng bất thường, không phải xác suất bản ghi bị sai và không thể diễn giải, chẳng hạn, điểm 0,8 tương đương xác suất lỗi 80%.

Bảng 2.2. Ký hiệu sử dụng trong mô hình Isolation Forest

Ký hiệu	Ý nghĩa
x	Bản ghi dữ liệu cần được đánh giá mức độ bất thường.
S	Tập mẫu dùng để xây dựng một Isolation Tree.
ψ	Kích thước mẫu dùng để huấn luyện mỗi cây, tương ứng với tham số <code>max_samples</code> .
t	Số lượng cây trong rừng cô lập, tương ứng với tham số <code>n_estimators</code> .
q, p	Đặc trưng q và điểm chia p được lựa chọn ngẫu nhiên tại một nút của cây.
$h_i(x)$	Độ dài đường đi của x trong cây thứ i .

Ký hiệu	Ý nghĩa
$E[h(x)]$	Độ dài đường đi trung bình của x trên toàn bộ rừng.
$H(k)$	Số điều hòa bậc k, được dùng khi xấp xỉ độ dài đường đi kỳ vọng.
$c(n)$	Độ dài đường đi trung bình kỳ vọng của cây tìm kiếm nhị phân với n mẫu.
$s(x, \psi)$	Điểm bất thường theo công thức gốc của Isolation Forest.
l	Giới hạn chiều cao của cây, thường được đặt bằng $\lceil \log_2 \psi \rceil$.



Hình 2.3. Quan hệ giữa độ dài đường đi chuẩn hóa và điểm bất thường

2.5.5. Cơ chế ensemble và vai trò của lấy mẫu

Isolation Forest sử dụng một tập hợp t cây để giảm phương sai do lựa chọn ngẫu nhiên đặc trưng và điểm chia. Khi t tăng, thứ hạng bất thường thường ổn định hơn, nhưng thời gian huấn luyện và suy luận cũng tăng tương ứng. Sau một mức nhất định, việc tăng thêm số cây chỉ đem lại cải thiện nhỏ, vì vậy $n_estimators$ cần được lựa chọn dựa trên sự cân bằng giữa độ ổn định và chi phí tính toán.

Lấy mẫu con không chỉ là một kỹ thuật rút ngắn thời gian mà còn là một phần quan trọng của ý tưởng Isolation Forest. Khi tập huấn luyện chứa nhiều quan sát bình thường bao quanh một cụm bất thường, hiện tượng masking có thể làm các bất thường khó bị cô lập. Ngược lại, phép phân hoạch không phù hợp có thể khiến một số điểm bình

thường bị đánh dấu nhầm, gọi là swamping. Các tập mẫu con nhỏ và độc lập có thể giảm mức độ hai hiện tượng này, đồng thời tạo thêm tính đa dạng giữa các cây [8].

Với t cây, kích thước mẫu ψ và n bản ghi cần đánh giá, độ phức tạp huấn luyện thường được mô tả ở mức $O(t\psi \log \psi)$, còn độ phức tạp chấm điểm xấp xỉ $O(nt \log \psi)$. Biểu thức này cho thấy chi phí tăng tuyến tính theo số cây và số bản ghi, trong khi độ sâu cây tăng theo logarit của kích thước mẫu. Đây là lý do Isolation Forest phù hợp với dữ liệu dạng bảng có quy mô vừa và lớn.

2.5.6. Ý nghĩa của các tham số chính

Tham số `n_estimators` quy định số iTree trong ensemble. Giá trị quá thấp có thể làm điểm số nhảy với ngẫu nhiên; giá trị lớn hơn giúp ổn định kết quả nhưng kéo dài thời gian xử lý. Tham số `max_samples` quyết định ψ , tức số quan sát dùng để xây dựng mỗi cây. Giá trị nhỏ làm cây nông hơn và tăng tính đa dạng do lấy mẫu; giá trị bằng toàn bộ tập fit sử dụng nhiều thông tin hơn trong từng cây nhưng giảm tác dụng của subsampling và có thể làm tăng chi phí.

Tham số `contamination` biểu diễn tỷ lệ bất thường giả định dùng để xác định ngưỡng quyết định. Tham số này không dạy mô hình đâu là lỗi đúng, cũng không đảm bảo tỷ lệ anomaly đầu ra chính xác tuyệt đối bằng giá trị cấu hình trong mọi trường hợp. Nó chủ yếu điều chỉnh vị trí ngưỡng trên phân phối điểm số: contamination càng lớn thì ngưỡng càng rộng và số bản ghi bị gắn nhãn bất thường thường càng nhiều.

Tham số `max_features` quy định số đặc trưng được xét khi xây dựng cây; `random_state` kiểm soát nguồn ngẫu nhiên để có thể tái lập thí nghiệm. `random_state` cố định không làm mô hình chính xác hơn, nhưng giúp bảo đảm rằng cùng dữ liệu và cùng cấu hình sẽ tạo ra kết quả có thể đối chiếu. Khi đánh giá độ ổn định, nên chạy thêm nhiều seed thay vì chỉ dựa vào một giá trị duy nhất.

2.5.7. Diễn giải điểm số trong scikit-learn

Cần phân biệt chiều của công thức điểm số trong bài báo gốc với giao diện của scikit-learn. Theo công thức gốc, $s(x, \psi)$ càng lớn thì bản ghi càng bất thường. Trong scikit-learn, `score_samples` trả về điểm mà giá trị thấp hơn thể hiện quan sát bất thường hơn. Hàm `decision_function` lấy `score_samples` trừ đi `offset_` của mô hình; giá trị âm nằm về phía outlier, còn giá trị không âm nằm về phía inlier [1].

Hàm predict quy đổi ngưỡng trên thành nhãn -1 cho outlier và +1 cho inlier. Vì vậy, khi lưu kết quả vào cơ sở dữ liệu hoặc tệp đầu ra, hệ thống cần ghi rõ chiều của từng cột điểm số. Việc gọi chung mọi đại lượng là anomaly score mà không nêu chiều lớn–nhỏ có thể dẫn đến diễn giải ngược kết quả.

Trong đồ án, raw_score lưu kết quả score_samples, decision_score lưu kết quả decision_function và is_anomaly được đặt bằng 1 khi predict trả về -1. Cách lưu ba trường này cho phép vừa giữ điểm liên tục để xếp hạng ưu tiên, vừa có nhãn nhị phân để tích hợp với detected_issues. Tuy nhiên, các điểm số chỉ có ý nghĩa so sánh trong cùng mô hình hoặc cùng segment; không nên mặc định so sánh tuyệt đối giữa hai mô hình được huấn luyện trên các phân phối khác nhau.

2.5.8. Tiền xử lý dữ liệu số và dữ liệu phân loại

Isolation Forest trong scikit-learn nhận ma trận đặc trưng số, trong khi dữ liệu giao dịch của đồ án gồm cả cột số và cột phân loại. Vì vậy, các giá trị thiếu của cột số được điền bằng trung vị; các giá trị thiếu của cột phân loại được điền bằng giá trị xuất hiện nhiều nhất. Sau đó, cột phân loại được chuyển sang dạng one-hot để mỗi mức giá trị trở thành một đặc trưng nhị phân.

Các cột số được chuẩn hóa bằng StandardScaler trong pipeline hiện tại. Về lý thuyết, cây quyết định không phụ thuộc khoảng cách Euclid và thường không bắt buộc phải chuẩn hóa thang đo. Việc giữ StandardScaler chủ yếu giúp pipeline tiền xử lý thống nhất và thuận lợi khi ma trận đặc trưng được tái sử dụng ở các module khác. Báo cáo không xem chuẩn hóa là điều kiện bắt buộc để Isolation Forest hoạt động.

One-hot encoding giúp thuật toán xử lý dữ liệu phân loại nhưng cũng làm tăng số chiều. Cột có quá nhiều mức giá trị có thể tạo ra nhiều đặc trưng nhị phân hơn và nhận nhiều cơ hội được chọn trong quá trình phân hoạch. Do đó, cần kiểm soát cột định danh, cột có cardinality cao và các đặc trưng ít ý nghĩa. Trong cấu hình hệ thống, chỉ những cột có use_for_detection=true mới được đưa vào mô hình; cột định danh được loại khỏi ma trận đặc trưng.

Bên cạnh dữ liệu gốc, hệ thống sử dụng các đặc trưng dẫn xuất như unit_price, age_group, invoice_year, invoice_month, invoice_day, day_of_week, is_weekend, quantity_band và price_deviation_from_category. Các đặc trưng này giúp mô hình quan sát không chỉ giá trị đơn lẻ mà còn quan hệ theo thời gian, nhóm tuổi, mức số lượng và

độ lệch giá trong từng category. Chất lượng feature engineering vì vậy ảnh hưởng trực tiếp đến chất lượng cảnh báo.

2.5.9. Mô hình global và mô hình theo segment

Một điểm thiết kế quan trọng trong hệ thống là anomaly detector luôn huấn luyện một mô hình global trên toàn bộ clean reference set. Mô hình này cung cấp góc nhìn tổng thể và được dùng cho các bản ghi không thuộc segment đã biết hoặc thuộc nhóm không đủ dữ liệu để huấn luyện mô hình riêng.

Ngoài mô hình global, hệ thống có thể huấn luyện các mô hình theo segment. Trong bộ dữ liệu e-commerce, `anomaly_segment_column` được đặt là category. Cách tiếp cận này giúp phát hiện bất thường theo ngữ cảnh: mức giá cao có thể bình thường trong nhóm Technology nhưng bất thường trong nhóm Food & Beverage. Nếu chỉ dùng mô hình global, sự khác biệt tự nhiên giữa các nhóm có thể làm tăng cảnh báo sai.

Mô hình segment chỉ được tạo khi nhóm có ít nhất `min_anomaly_segment_size` bản ghi; cấu hình hiện tại đặt ngưỡng này bằng 500. Nhóm nhỏ hơn được chuyển về mô hình global để tránh huấn luyện một rừng trên lượng dữ liệu quá ít. Kết quả đầu ra lưu `anomaly_model_segment` để truy vết bản ghi đã được đánh giá bởi mô hình nào.

Điểm số từ các mô hình segment không nhất thiết có cùng phân phối vì mỗi mô hình được fit trên dữ liệu và bộ tiền xử lý riêng. Vì vậy, hệ thống ưu tiên sử dụng nhãn và thứ hạng trong từng segment, thay vì so sánh trực tiếp `raw_score` giữa các category. Đây là giới hạn quan trọng cần được lưu ý khi xây dựng dashboard hoặc sắp xếp mức độ ưu tiên trên toàn bộ dữ liệu.

2.5.10. Ưu điểm, hạn chế và vai trò trong hệ thống

Ưu điểm chính của Isolation Forest là không yêu cầu nhãn đầy đủ, có khả năng mở rộng tốt trên dữ liệu dạng bảng và tạo ra điểm liên tục để xếp hạng bản ghi. Cơ chế phân hoạch ngẫu nhiên cũng tránh phải tính toán bộ ma trận khoảng cách giữa các quan sát, nhờ đó phù hợp hơn với dữ liệu giao dịch có số dòng lớn so với nhiều phương pháp dựa trên khoảng cách.

Tuy nhiên, mô hình nhạy với lựa chọn đặc trưng, tỷ lệ contamination và cấu trúc phân phối. Dữ liệu nhiều chiều, nhiều cột nhiễu hoặc nhiều mức phân loại có thể làm phép chia ngẫu nhiên kém hiệu quả. Một bản ghi hiếm nhưng hợp lệ vẫn có thể bị gán nhãn anomaly; ngược lại, một nhóm lỗi lặp lại đủ nhiều có thể trở thành một cụm và khó

bị cô lập. Khi phân phối dữ liệu thay đổi theo thời gian, mô hình cũng cần được fit lại trên tập tham chiếu mới.

Isolation Forest chỉ xác định mức độ bất thường ở cấp bản ghi, không trực tiếp kết luận cột nào là nguyên nhân và không cung cấp quan hệ nhân quả. Vì vậy, hệ thống bổ sung FeatureContributionAnalyzer để so sánh anomaly với các bản ghi bình thường tương tự, đồng thời sử dụng rule-based validation cho những điều kiện nghiệp vụ có thể mô tả rõ. Sự kết hợp này giúp duy trì tính giải thích: luật xử lý lỗi chắc chắn, Isolation Forest mở rộng phạm vi phát hiện, còn recommender hỗ trợ bước xem xét và sửa dữ liệu.

2.6. Nearest Neighbors cho gợi ý sửa dữ liệu

Nearest Neighbors là phương pháp tìm các điểm dữ liệu gần nhất với một điểm cần xét dựa trên một độ đo khoảng cách hoặc độ tương tự. Trong scikit-learn, NearestNeighbors được mô tả là một learner không giám sát dùng để thực hiện tìm kiếm láng giềng [2].

Trong đồ án, nearest neighbors được sử dụng để hỗ trợ gợi ý sửa dữ liệu categorical/text. Ý tưởng chính là nếu một bản ghi bị thiếu hoặc sai giá trị ở một cột nào đó, hệ thống có thể tìm các bản ghi tương tự dựa trên những cột còn lại, sau đó suy luận giá trị gợi ý từ nhóm láng giềng gần nhất. Ví dụ, nếu một bản ghi có nhiều đặc trưng giống với các giao dịch cùng nhóm sản phẩm, cùng khoảng giá và cùng ngữ cảnh thời gian, hệ thống có thể đề xuất giá trị categorical phù hợp hơn.

Phương pháp này có ưu điểm là dễ hiểu và phù hợp với dữ liệu có cấu trúc. Tuy nhiên, với dữ liệu số bất thường, việc tự động thay thế bằng giá trị trung bình hoặc giá trị của láng giềng có thể làm mất thông tin quan trọng. Vì vậy, hệ thống chỉ áp dụng gợi ý sửa ở mức có kiểm soát và ưu tiên review thủ công với anomaly numeric.

Trong hệ thống, dữ liệu đầu vào cho nearest neighbors là dữ liệu hỗn hợp gồm numeric và categorical. Vì vậy, hệ thống không chỉ dùng trực tiếp một khoảng cách Euclidean thông thường. Với dữ liệu numeric, khoảng cách được chuẩn hóa theo khoảng giá trị của cột:

```
distance_numeric = |x_ref - x_row| / range(column)
```

Với dữ liệu categorical, hệ thống dùng khoảng cách dạng match/mismatch:

```
distance_categorical = 0 nếu hai giá trị giống nhau  
distance_categorical = 1 nếu hai giá trị khác nhau
```

Khoảng cách tổng hợp được tính bằng trung bình khoảng cách của các cột được sử dụng. Nếu một cột đang bị lỗi, cột đó được loại khỏi quá trình tính khoảng cách để tránh việc giá trị sai làm ảnh hưởng đến việc tìm láng giềng. Đây là điểm quan trọng trong thiết kế recommender.

Ví dụ, nếu `payment_method` của một giao dịch bị sai, hệ thống không dùng `payment_method` để tìm các dòng tương tự. Thay vào đó, hệ thống dùng các feature còn lại như `category`, `unit_price`, `quantity_band`, `age_group` hoặc các feature phù hợp khác. Sau khi tìm được các dòng gần nhất, hệ thống lấy mode của `payment_method` trong nhóm láng giềng để đề xuất giá trị sửa.

2.6.1. Confidence trong recommendation

Confidence của gợi ý sửa được tính khác nhau tùy loại dữ liệu. Với categorical, confidence được tính dựa trên tỷ lệ láng giềng có cùng giá trị được đề xuất:

```
text
confidence = số láng giềng có giá trị đề xuất / tổng số láng giềng hợp lệ
```

Nếu 4 trong 5 láng giềng gần nhất có `payment_method` = Cash, confidence của gợi ý Cash là 0.8. Với numeric missing, hệ thống lấy median của các giá trị láng giềng và ước lượng confidence dựa trên hệ số biến thiên. Nếu các giá trị láng giềng gần nhau, confidence cao hơn; nếu phân tán mạnh, confidence thấp hơn.

Với invalid categorical, hệ thống sử dụng so khớp chuỗi gần đúng. Nếu giá trị sai gần giống một giá trị hợp lệ, hệ thống đề xuất giá trị hợp lệ gần nhất. Ví dụ, nếu người dùng nhập sai chính tả một payment method, phương pháp này có thể đưa ra gợi ý hợp lý. Nếu không tìm được chuỗi gần đúng, hệ thống chọn giá trị đầu tiên trong danh sách allowed values với confidence thấp hơn, xem như một gợi ý mặc định cần kiểm tra.

2.7. Feature engineering cho dữ liệu giao dịch

Feature engineering là quá trình tạo ra các đặc trưng mới từ dữ liệu gốc nhằm hỗ trợ mô hình hoặc pipeline xử lý hiệu quả hơn về ngữ cảnh dữ liệu. Trong đồ án, từ các trường gốc như `price`, `quantity`, `invoice_date`, `category` và `shopping_mall`, hệ thống sinh thêm nhiều feature phục vụ kiểm tra và phát hiện bất thường.

Một số feature quan trọng gồm:

Bảng 2.3. Các đặc trưng được xây dựng cho dữ liệu giao dịch

Feature	Ý nghĩa
unit_price	Đơn giá, được tính từ price / quantity
age_group	Nhóm tuổi khách hàng
invoice_year	Năm giao dịch
invoice_month	Tháng giao dịch
invoice_day	Ngày trong tháng
day_of_week	Thứ trong tuần
is_weekend	Giao dịch có rơi vào cuối tuần hay không
base_unit_price_by_category	Đơn giá tham chiếu theo danh mục sản phẩm
price_deviation_from_category	Độ lệch giá so với mức tham chiếu của category
quantity_band	Nhóm số lượng mua

Các feature này giúp hệ thống phân biệt tốt hơn giữa lỗi rõ ràng và bất thường theo ngữ cảnh. Ví dụ, cùng một mức giá có thể bình thường với nhóm sản phẩm Technology nhưng bất thường với nhóm Food & Beverage. Vì vậy, việc xét thêm category và độ lệch giá theo category giúp kết quả có ý nghĩa hơn.

Trong hệ thống, feature engineering được dùng cho hai mục đích. Mục đích thứ nhất là hỗ trợ kiểm tra dữ liệu bằng rule. Ví dụ, invoice_month, invoice_day, day_of_week và is_weekend có thể được kiểm tra ngược lại với invoice_date để phát hiện sai lệch. Mục đích thứ hai là hỗ trợ mô hình học máy. Các mô hình như Isolation Forest không hiểu trực tiếp ý nghĩa nghiệp vụ nếu chỉ nhìn vào các cột gốc, do đó việc tạo feature giúp mô hình có thêm tín hiệu.

Một số feature có vai trò đặc biệt:

1. unit_price: giúp tách ảnh hưởng của số lượng ra khỏi tổng giá. Nếu chỉ dùng price, giao dịch mua nhiều sản phẩm có thể bị xem là bất thường dù đơn giá bình thường.
2. price_deviation_from_category: giúp đánh giá giá theo bối cảnh category. Đây là feature có ý nghĩa cao trong phát hiện bất thường giá.

-
3. `quantity_band`: gom số lượng thành nhóm `single/small/bulk`, giúp dữ liệu categorical phản ánh hành vi mua.
 4. `age_group`: giúp mô hình xét nhóm tuổi thay vì chỉ dùng tuổi liên tục.
 5. `mall_popularity_score`: phản ánh mức độ phổ biến của shopping mall, hỗ trợ phân tích theo địa điểm.

Quá trình feature engineering cũng cần thận trọng vì feature sai có thể kéo theo lỗi dây chuyền. Ví dụ, nếu `invoice_date` bị parse sai, các feature `invoice_year`, `invoice_month`, `day_of_week` cũng sai. Vì vậy, hệ thống có bước parse ngày riêng và kiểm tra lại các thành phần ngày trong cross-field validation.

2.8. PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ đối tượng mã nguồn mở, hỗ trợ SQL và nhiều tính năng hiện đại của hệ quản trị cơ sở dữ liệu [3]. Trong đồ án, PostgreSQL được sử dụng để lưu trữ dữ liệu nghiệp vụ đã chuẩn hóa, dữ liệu thô trong bảng staging và các bảng phục vụ quá trình làm sạch dữ liệu.

Điểm quan trọng trong thiết kế là AI không chạy trực tiếp trong database. PostgreSQL đóng vai trò là nơi lưu dữ liệu chuẩn và kết quả. Pipeline Python đọc dữ liệu từ PostgreSQL hoặc CSV, thực hiện xử lý bên ngoài, sau đó ghi lại kết quả vào các bảng output. Cách tiếp cận này giúp tách biệt trách nhiệm giữa tầng lưu trữ và tầng xử lý dữ liệu, đồng thời giúp hệ thống dễ kiểm tra và mở rộng.

Trong báo cáo này, PostgreSQL không chỉ được xem như nơi lưu bảng giao dịch. Database còn là một phần của kiến trúc data cleaning. Các bảng như `detected_issues`, `fix_recommendations`, `dataset_profile` và `fixed_transactions` giúp lưu lại toàn bộ kết quả xử lý. Nhờ vậy, sau khi pipeline chạy xong, người dùng có thể truy vấn lại dữ liệu lỗi, xem recommendation hoặc kiểm tra dữ liệu sau khi fix mà không cần phụ thuộc vào log terminal.

Việc thiết kế database theo nhiều nhóm bảng cũng giúp phân biệt rõ dữ liệu nguồn, dữ liệu nghiệp vụ và dữ liệu phục vụ kiểm soát chất lượng. Đây là điểm khác biệt giữa một chương trình xử lý file đơn giản và một hệ thống hỗ trợ làm sạch dữ liệu có khả năng mở rộng.

2.9. Pandas, Scikit-learn và SQLAlchemy

Pandas hay pandas là thư viện Python cung cấp các cấu trúc dữ liệu linh hoạt cho dữ liệu dạng bảng, đặc biệt phù hợp với dữ liệu có cột và hàng như bảng SQL hoặc file Excel/CSV [4]. Trong hệ thống, pandas được sử dụng để đọc dữ liệu, chuẩn hóa cột, xử lý kiểu dữ liệu, tính feature và sinh các bảng output.

Scikit-learn là thư viện học máy được sử dụng để triển khai Isolation Forest và K-nearest-neighbors. Việc sử dụng scikit-learn giúp hệ thống tận dụng các thuật toán đã được chuẩn hóa, có tài liệu rõ ràng và phù hợp với các bài toán học máy truyền thống.

SQLAlchemy là bộ công cụ làm việc với cơ sở dữ liệu trong Python, cung cấp các thành phần để kết nối, thực thi SQL và tích hợp dữ liệu giữa ứng dụng Python với hệ quản trị cơ sở dữ liệu [6]. Trong đề án, SQLAlchemy hỗ trợ luồng đọc dữ liệu từ PostgreSQL và ghi kết quả xử lý trở lại database.

2.10. Chỉ số đánh giá

Đối với lỗi có ground truth, hệ thống có thể được đánh giá bằng precision, recall và F1-score. Precision cho biết trong số các lỗi hệ thống phát hiện, có bao nhiêu lỗi là đúng. Recall cho biết trong số các lỗi thực sự tồn tại, hệ thống phát hiện được bao nhiêu. F1-score là trung bình điều hòa giữa precision và recall, thường được sử dụng khi cần cân bằng giữa hai chỉ số. Việc diễn giải các chỉ số này phải gắn với cách xây dựng mẫu, nhãn chuẩn và phạm vi dữ liệu đánh giá để tránh đưa ra kết luận vượt quá bằng chứng thực nghiệm [13].

Đối với anomaly detection, việc đánh giá khó hơn vì không phải mọi anomaly đều có nhãn đúng/sai rõ ràng. Trong phạm vi đề án, kết quả anomaly được đánh giá kết hợp giữa thống kê số lượng, kiểm tra theo cột, phân tích ví dụ cụ thể và nhận xét về khả năng sinh false positive.

Ngoài chất lượng phát hiện lỗi, hệ thống còn được đánh giá bằng runtime. Thời gian chạy giúp xác định pipeline có thể xử lý dữ liệu đủ lớn hay không và stage nào là bottleneck chính.

Các chỉ số precision, recall và F1-score được định nghĩa như sau:

$\begin{aligned}\text{Precision} &= \text{TP} / (\text{TP} + \text{FP}) \\ \text{Recall} &= \text{TP} / (\text{TP} + \text{FN}) \\ \text{F1-score} &= 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})\end{aligned}$

Bảng 2.4. Các chỉ số đánh giá hệ thống

Ký hiệu	Ý nghĩa
TP	True Positive - lỗi được phát hiện đúng
FP	False Positive - hệ thống báo lỗi nhưng thực tế không lỗi
FN	False Negative - lỗi thực tế nhưng hệ thống bỏ sót
TN	True Negative - dữ liệu đúng và hệ thống không báo lỗi

Trong bài toán missing và invalid, việc xác định TP/FP/FN tương đối rõ nếu có ground truth. Tuy nhiên, trong anomaly detection, ranh giới đúng/sai phức tạp hơn vì một điểm bất thường có thể là một giao dịch hợp lệ nhưng hiếm. Do đó, báo cáo cần trình bày rõ cách dùng labels. Nếu labels chưa hoàn chỉnh, kết quả precision/recall/F1 chỉ nên tính trên phần lỗi có nhãn chắc chắn, không nên diễn giải như đánh giá toàn bộ dataset.

Ngoài các chỉ số định lượng, hệ thống cũng cần được đánh giá định tính thông qua các case study. Ví dụ, chọn một bản ghi bị thiếu nhiều trường hoặc có category invalid, sau đó trình bày dữ liệu gốc, issue được phát hiện, gợi ý sửa và kết quả sau fix. Cách trình bày này giúp người đọc hiểu rõ cơ chế hoạt động của pipeline hơn là chỉ nhìn vào số lượng issue.

2.11. Kết luận chương

Chương 2 đã trình bày các kiến thức nền tảng và công nghệ chính được sử dụng trong đồ án. Chất lượng dữ liệu là cơ sở để xác định các nhóm lỗi cần xử lý. Rule-based validation phù hợp với các lỗi rõ ràng, có thể mô tả bằng điều kiện. Isolation Forest hỗ trợ phát hiện bất thường theo ngữ cảnh khi dữ liệu không có nhãn đầy đủ. Nearest neighbors hỗ trợ tìm các bản ghi tương tự để đưa ra gợi ý sửa lỗi. PostgreSQL, pandas, scikit-learn và SQLAlchemy tạo thành nền tảng công nghệ để triển khai toàn bộ pipeline.

Điểm quan trọng của chương này là làm rõ sự liên kết giữa lý thuyết và hệ thống. Mỗi khái niệm được trình bày đều tương ứng với một thành phần trong pipeline: data profiling tương ứng với dataset_profile, rule-based validation tương ứng với các validator, Isolation Forest tương ứng với anomaly detector, nearest neighbors tương ứng với recommender, còn precision/recall/F1 và runtime tương ứng với phần đánh giá ở Chương 5.

Từ cơ sở lý thuyết đã trình bày, Chương 3 sẽ chuyển sang phân tích và thiết kế hệ thống cụ thể. Nếu Chương 2 cung cấp "công cụ tư duy", thì Chương 3 sẽ trình bày cách các công cụ đó được tổ chức thành một kiến trúc có thể vận hành, có input, output, module xử lý, cơ sở dữ liệu và cơ chế truy vết rõ ràng.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương 3 trình bày quá trình phân tích bài toán và thiết kế hệ thống. Nội dung bao gồm mô tả dataset, phân loại lỗi dữ liệu, yêu cầu chức năng, yêu cầu phi chức năng, kiến trúc tổng thể, thiết kế pipeline xử lý và thiết kế cơ sở dữ liệu PostgreSQL.

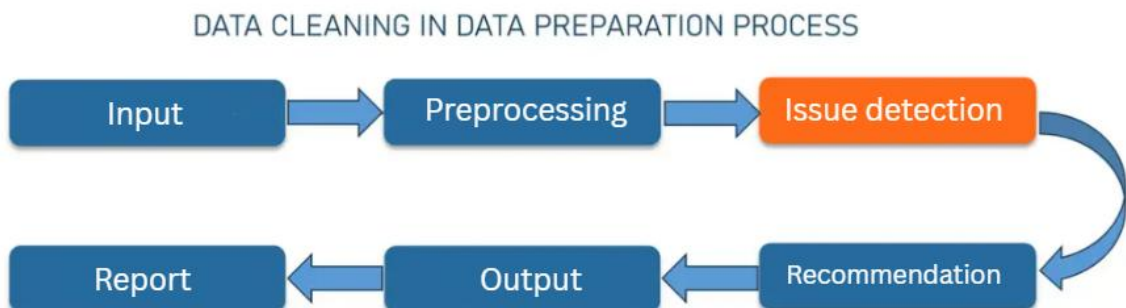
Chương này đóng vai trò chuyển tiếp từ phần lý thuyết sang phần cài đặt. Các khái niệm ở Chương 2 sẽ được cụ thể hóa thành những quyết định thiết kế: dữ liệu được tổ chức thành bảng nào, pipeline gồm những bước nào, detector và recommender trao đổi dữ liệu ra sao, issue được biểu diễn như thế nào và kết quả cuối cùng được lưu ở đâu. Vì vậy, khi trình bày Chương 3, trọng tâm không chỉ là liệt kê thành phần hệ thống, mà là giải thích lý do các thành phần đó được đặt cạnh nhau và cách chúng phối hợp để giải quyết bài toán làm sạch dữ liệu.

3.1. Phân tích bài toán

Bài toán đặt ra là xây dựng một hệ thống hỗ trợ làm sạch dữ liệu giao dịch thương mại điện tử. Đầu vào của hệ thống là dữ liệu dạng bảng, có thể đến từ file CSV hoặc từ PostgreSQL. Đầu ra của hệ thống là danh sách lỗi phát hiện được, gợi ý sửa lỗi, hồ sơ chất lượng dữ liệu, báo cáo tổng hợp và dữ liệu sau khi được áp dụng các sửa lỗi phù hợp.

Về bản chất, hệ thống cần giải quyết ba yêu cầu chính. Thứ nhất, hệ thống phải chuẩn hóa dữ liệu để các bước xử lý phía sau có định dạng thống nhất. Thứ hai, hệ thống phải phát hiện được các loại lỗi dữ liệu khác nhau. Thứ ba, hệ thống phải lưu lại kết quả theo cách có thể kiểm tra, truy vết và sử dụng lại.

Luồng xử lý tổng quát của hệ thống có thể mô tả như sau:



Hình 3.1. Luồng xử lý tổng quan của hệ thống

Về mặt phân tích, bài toán có bốn đặc điểm cần lưu ý:

Thứ nhất, dữ liệu đầu vào là dữ liệu dạng bảng có cả numeric, categorical và date. Điều này yêu cầu hệ thống phải có bước phân loại kiểu dữ liệu và tiền xử lý riêng cho từng nhóm. Không thể áp dụng cùng một cách xử lý cho tuổi, giá, phương thức thanh toán và ngày giao dịch.

Thứ hai, dữ liệu vừa có lỗi chắc chắn vừa có lỗi nghi vấn. Lỗi chắc chắn có thể phát hiện bằng rule, còn lỗi nghi vấn cần mô hình thống kê hoặc học máy. Vì vậy, pipeline cần thiết kế theo nhiều lớp thay vì chỉ dùng một detector duy nhất.

Thứ ba, dữ liệu sau khi phát hiện lỗi cần có khả năng truy vết. Một hệ thống data cleaning thực tế không nên chỉ trả về một file đã sửa, vì người dùng cần biết giá trị nào đã bị thay đổi và thay đổi dựa trên cơ sở nào. Do đó, issue và recommendation được lưu thành bảng riêng.

Thứ tư, hệ thống phải hỗ trợ cả vận hành thử nghiệm và vận hành gắn với database thật. Chạy từ CSV giúp demo nhanh và tái lập dễ, còn chạy từ PostgreSQL giúp thể hiện đúng vai trò của cơ sở dữ liệu trong hệ thống.

3.1.1. Đầu vào, xử lý và đầu ra

Có thể mô tả bài toán theo ba thành phần chính:

Bảng 3.1. Mô hình đầu vào, xử lý và đầu ra của hệ thống

Thành phần	Mô tả
Input	Dữ liệu giao dịch từ CSV hoặc PostgreSQL
Processing	Chuẩn hóa, feature engineering, rule validation, anomaly detection, recommendation
Output	Detected issues, fix recommendations, dataset profile, final report, fixed transactions

Input của hệ thống không chỉ là dữ liệu thô, mà còn bao gồm file cấu hình. File cấu hình quy định cột nào bắt buộc, cột nào dùng cho detection, cột nào dùng cho recommendation, miền giá trị nào hợp lệ và tham số mô hình như thế nào. Nhờ đó, pipeline có thể tách phần logic xử lý chung khỏi phần quy tắc phụ thuộc dataset.

Output của hệ thống được thiết kế theo hướng phục vụ cả máy và người. Các file CSV hoặc bảng PostgreSQL có thể được đọc lại bởi chương trình khác, đồng thời người

dùng cũng có thể mở trực tiếp để kiểm tra. Điều này phù hợp với phạm vi đề án vì hệ thống chưa có dashboard, nhưng vẫn cần output để quan sát.

3.1.2. Luồng nghiệp vụ của một issue

Một issue trong hệ thống đi qua các trạng thái logic sau:

1. Được phát hiện bởi detector hoặc validator.
2. Được chuẩn hóa thành `IssueRecord`.
3. Được bổ sung gợi ý sửa nếu có thể.
4. Được ghi vào `detected_issues`.
5. Được rút gọn thành `recommendation` và ghi vào `fix_recommendations`.
6. Nếu đủ điều kiện, được `FixService` áp dụng vào dữ liệu đầu ra.
7. Dữ liệu sau sửa được ghi vào `fixed_transactions`.

Luồng này giúp hệ thống không bị trộn lẫn giữa phát hiện lỗi và sửa lỗi. Detector chỉ chịu trách nhiệm phát hiện và giải thích. Recommender chịu trách nhiệm đề xuất. `FixService` chịu trách nhiệm quyết định có áp dụng đề xuất hay không theo chế độ `conservative`, `auto` hoặc `interactive`.

3.2. Dataset sử dụng

Dataset sử dụng trong đề án là Customer Shopping Dataset - Retail Sales Data trên Kaggle [9]. Dữ liệu mô phỏng các giao dịch mua sắm với các thông tin như mã hóa đơn, mã khách hàng, giới tính, tuổi, danh mục sản phẩm, số lượng, giá, phương thức thanh toán, ngày hóa đơn và trung tâm mua sắm.

Việc lựa chọn dataset này phù hợp với mục tiêu của đề án vì dữ liệu có đủ các nhóm cột quan trọng cho một bài toán data cleaning dạng bảng. Các cột như `age`, `quantity`, `price` là dữ liệu số; các cột như `gender`, `category`, `payment_method`, `shopping_mall` là dữ liệu phân loại; `invoice_date` là dữ liệu thời gian. Sự kết hợp này tạo điều kiện để hệ thống kiểm tra nhiều loại lỗi khác nhau, đồng thời cũng phù hợp để áp dụng các kỹ thuật feature engineering và anomaly detection.

Dataset ban đầu được cung cấp dưới dạng bảng phẳng. Nếu chỉ xử lý trực tiếp bảng phẳng, hệ thống vẫn có thể phát hiện một số lỗi dữ liệu, nhưng khó thể hiện được tư duy thiết kế cơ sở dữ liệu. Vì vậy, đề án chủ động chuyển đổi dataset thành mô hình quan hệ với các bảng dimension và fact. Quyết định này giúp đề tài có chiều sâu hơn ở phần database design và gần hơn với cách tổ chức dữ liệu trong các hệ thống thực tế.

Các cột gốc của dataset gồm:

Bảng 3.2. Các cột dữ liệu gốc của bộ dữ liệu

Cột gốc	Ý nghĩa
INVOICE_NO	Mã hóa đơn/giao dịch
CUSTOM_ID	Mã khách hàng
GENDER	Giới tính khách hàng
AGE	Tuổi khách hàng
CATEGORY	Danh mục sản phẩm
QUANTITY	Số lượng mua
PRICE	Giá trị giao dịch
PAYMENT_METHOD	Phương thức thanh toán
INVOICE_DATE	Ngày giao dịch
SHOPPING_MALL	Trung tâm mua sắm

Sau khi chuẩn hóa, hệ thống sử dụng tên cột dạng snake_case:

Bảng 3.3. Các cột dữ liệu sau khi chuẩn hóa

Cột chuẩn hóa	Ý nghĩa
invoice_no	Mã hóa đơn
customer_id	Mã khách hàng
gender	Giới tính
age	Tuổi
category	Danh mục sản phẩm
quantity	Số lượng
price	Giá
payment_method	Phương thức thanh toán
invoice_date	Ngày giao dịch
shopping_mall	Trung tâm mua sắm

Dataset đầy đủ sau khi xử lý có 99.457 giao dịch. Trong thiết kế hiện tại, bảng products đại diện ở mức category, chưa phải mức SKU sản phẩm chi tiết. Đây là giới hạn được chấp nhận trong phạm vi đề án vì dataset gốc không có mã sản phẩm chi tiết.

```

1 INVOICE_NO,CUSTOM_ID,GENDER,AGE,CATEGORY,QUANTITY,PRICE,PAYMENT_METHOD,INVOICE_DATE,SHOPPING_MALL
2 I138884,C241288,Female,28,Clothing,5,1500.4,Credit Card,5/8/2022,Kanyon
3 I317333,C111565,Male,21,Shoes,3,1800.51,Debit Card,12/12/2021,Forum Istanbul
4 I127801,C266599,Male,20,Clothing,1,300.08,Cash,9/11/2021,Metrocity
5 I173702,C988172,Female,66,Shoes,5,3000.85,Credit Card,16/05/2021,Metropol AVM
6 I337046,C189076,Female,53,Books,4,60.6,Cash,24/10/2021,Kanyon
7 I227836,C657758,Female,28,Clothing,5,1500.4,Credit Card,24/05/2022,Forum Istanbul
8 I121056,C151197,Female,49,Cosmetics,1,40.66,Cash,13/03/2022,Istinye Park
9 I293112,C176086,Female,32,Clothing,2,600.16,Credit Card,13/01/2021,Mall of Istanbul
10 I293455,C159642,Male,69,Clothing,3,900.24,Credit Card,4/11/2021,Metrocity
11 I326945,C283361,Female,60,Clothing,2,600.16,Credit Card,22/08/2021,Kanyon
12 I306368,C240286,Female,36,Food & Beverage,2,10.46,Cash,25/12/2022,Metrocity
13 I139207,C191708,Female,29,Books,1,15.15,Credit Card,28/10/2022,Emaar Square Mall
14 I640508,C225330,Female,67,Toys,4,143.36,Debit Card,31/07/2022,Metrocity
15 I179802,C312861,Male,25,Clothing,2,600.16,Cash,17/11/2022,Cevahir AVM
16 I336189,C555402,Female,67,Clothing,2,600.16,Credit Card,3/6/2022,Kanyon
17 I688768,C362288,Male,24,Shoes,5,3000.85,Credit Card,7/11/2021,Viaport Outlet
18 I294687,C300786,Male,65,Books,2,30.3,Debit Card,16/01/2021,Metrocity
19 I195744,C330667,Female,42,Food & Beverage,3,15.69,Credit Card,5/1/2022,Zorlu Center
20 I993048,C218149,Female,46,Clothing,2,600.16,Cash,26/07/2021,Metropol AVM

```

Hình 3.2. Ảnh mẫu dataset đầu vào

3.3. Phân loại lỗi dữ liệu trong hệ thống

Hệ thống phân loại lỗi thành ba nhóm chính: missing, invalid và anomaly.

Missing data là lỗi khi một cột bắt buộc không có giá trị. Các cột như invoice_no, customer_id, gender, age, category, quantity, price, payment_method, invoice_date và shopping_mall được xem là các trường quan trọng. Nếu những trường này bị thiếu, dữ liệu có thể không đủ điều kiện để phân tích.

Invalid data là lỗi khi giá trị không nằm trong miền hợp lệ. Ví dụ, gender chỉ chấp nhận Female hoặc Male; payment_method chỉ chấp nhận Cash, Credit Card, Debit Card; quantity phải lớn hơn hoặc bằng 1; invoice_month phải nằm trong khoảng từ 1 đến 12.

Anomaly data là những bản ghi có đặc trưng khác biệt so với phần lớn dữ liệu. Những bản ghi này được phát hiện bằng mô hình Isolation Forest. Vì anomaly không luôn đồng nghĩa với dữ liệu sai, hệ thống đánh dấu, ghi lý do và sinh gợi ý kiểm tra thay vì mặc định sửa tất cả.

Bảng sau mô tả chi tiết hơn cách hệ thống phát hiện từng nhóm lỗi:

Bảng 3.4. Phân loại nhóm lỗi và phương pháp phát hiện

Nhóm lỗi	Điều kiện phát hiện	Phương pháp	Ví dụ
Missing	Bị NaN, rỗng hoặc blank space	Rule-based	Thiếu price

Nhóm lỗi	Điều kiện phát hiện	Phương pháp	Ví dụ
Invalid numeric	Không chuyển được sang số hoặc vượt min/max	Rule-based	quantity = -5
Invalid categorical	Giá trị không thuộc allowed values	Rule-based	gender = Unknown
Invalid date	Không parse được ngày	Rule-based	invoice_date = abc
Cross-field invalid	Các trường liên quan không nhất quán	Rule-based	unit_price không khớp price / quantity
Anomaly	Bản ghi bị Isolation Forest đánh dấu bất thường	ML-based	Tổ hợp category, price, mall khác thường

Việc tách cross-field invalid thành một nhóm riêng có ý nghĩa quan trọng. Nhiều lỗi dữ liệu không thể phát hiện nếu chỉ nhìn từng cột độc lập. Ví dụ, price = 100, quantity = 2, unit_price = 60 đều là các giá trị số hợp lệ nếu xét riêng, nhưng khi xét công thức price = quantity * unit_price thì bản ghi có mâu thuẫn.

3.4. Yêu cầu chức năng

Các yêu cầu chức năng chính của hệ thống gồm:

1. Đọc dữ liệu từ CSV hoặc PostgreSQL.
2. Chuẩn hóa tên cột, kiểu dữ liệu và các trường ngày tháng.
3. Sinh feature phục vụ phát hiện lỗi và gợi ý sửa.
4. Phát hiện missing data dựa trên cấu hình required fields.
5. Phát hiện invalid data dựa trên miền giá trị và điều kiện ràng buộc.
6. Phát hiện anomaly data bằng Isolation Forest.
7. Phân tích feature nghi ngờ gây ra anomaly.
8. Gợi ý sửa dữ liệu categorical/text bằng nearest neighbors.
9. Sinh các file hoặc bảng output gồm lỗi, gợi ý, profile, report và fixed data.
10. Hỗ trợ apply fixes có kiểm soát.
11. Hỗ trợ đo thời gian chạy từng stage bằng tùy chọn runtime profiling.

Các yêu cầu chức năng có thể mô tả dưới dạng use case như sau:

Bảng 3.5. Các ca sử dụng của hệ thống

Use case	Tác nhân	Mô tả
UC01 - Chạy pipeline từ CSV	Người dùng kỹ thuật	Người dùng cung cấp file CSV, config và thư mục output để chạy toàn bộ pipeline
UC02 - Chạy pipeline từ PostgreSQL	Người dùng kỹ thuật	Người dùng cung cấp connection URI để hệ thống đọc dữ liệu từ database
UC03 - Sinh dirty data	Người dùng kiểm thử	Hệ thống tạo dữ liệu lỗi và labels để phục vụ kiểm tra detector
UC04 - Phát hiện lỗi	Pipeline	Hệ thống phát hiện missing, invalid, cross-field và anomaly
UC05 - Gợi ý sửa lỗi	Pipeline	Hệ thống sinh suggested value và confidence cho issue
UC06 - Apply fixes	Pipeline	Hệ thống áp dụng gợi ý đủ điều kiện vào bản dữ liệu đầu ra
UC07 - Sinh báo cáo	Pipeline	Hệ thống tạo final report và dataset profile
UC08 - Ghi output	Pipeline	Hệ thống ghi kết quả ra CSV hoặc PostgreSQL

Trong các use case trên, UC04, UC05 và UC06 là phần thể hiện rõ nhất bản chất AI-assisted data cleaning. UC04 phát hiện vấn đề, UC05 đề xuất hướng xử lý, UC06 áp dụng sửa lỗi với cơ chế kiểm soát.

3.5. Yêu cầu phi chức năng

Hệ thống cần đáp ứng các yêu cầu phi chức năng sau:

1. Dễ cấu hình: các rule và tham số mô hình được đặt trong file YAML.
2. Có khả năng truy vết: lỗi phát hiện và gợi ý sửa được lưu thành bảng riêng.
3. Có khả năng mở rộng: có thể thay đổi nguồn dữ liệu hoặc mở rộng rule.
4. An toàn khi sửa dữ liệu: không tự động sửa các lỗi rủi ro cao nếu thiếu độ tin cậy.
5. Hiệu năng chấp nhận được: pipeline phải chạy được trên tập dữ liệu gần 100.000 dòng.
6. Dễ trình bày: output được thiết kế thành các bảng rõ ràng, phù hợp cho báo cáo và bảo vệ.

Ngoài ra, hệ thống cần đảm bảo tính tái lập. Các tham số như `random_state` trong mô hình được cấu hình để kết quả có thể lặp lại trong cùng điều kiện dữ liệu. Đây là yêu cầu quan trọng trong môi trường học thuật, vì kết quả thực nghiệm cần có khả năng kiểm chứng.

Hệ thống cũng cần đảm bảo tính bảo toàn dữ liệu. Khi apply fixes, dữ liệu gốc không bị ghi đè trực tiếp. Thay vào đó, hệ thống tạo bảng hoặc file `fixed_transactions`. Cách làm này giúp người dùng có thể so sánh dữ liệu trước và sau khi sửa, đồng thời tránh rủi ro mất dữ liệu gốc.

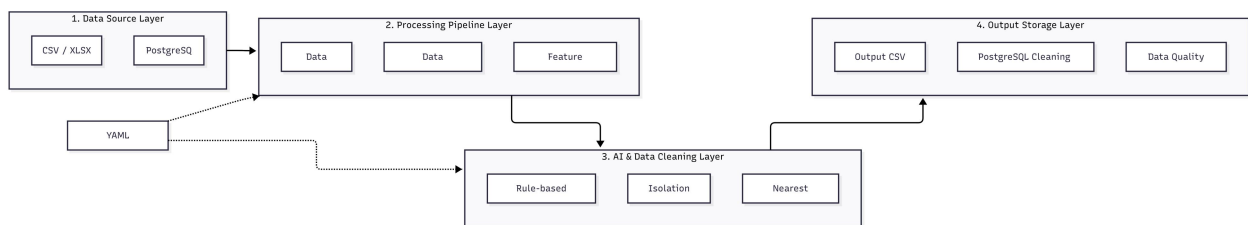
Về khả năng bảo trì, hệ thống được chia thành nhiều module độc lập. Detector, recommender, data source và report service có trách nhiệm riêng. Khi cần thay đổi thuật toán anomaly detection hoặc thêm recommender mới, có thể mở rộng từng module mà không cần sửa toàn bộ pipeline.

3.6. Kiến trúc tổng thể hệ thống

Kiến trúc hệ thống được chia thành các lớp chính:

1. Lớp dữ liệu đầu vào: CSV, Excel hoặc PostgreSQL.
2. Lớp tiền xử lý: chuẩn hóa cột, parse ngày, ép kiểu dữ liệu, sinh feature.
3. Lớp phát hiện lỗi bằng luật: missing detector, invalid detector, cross-field validator.
4. Lớp phát hiện bất thường bằng AI/ML: Isolation Forest và feature contribution analysis.
5. Lớp gợi ý sửa: nearest-neighbor recommendation.
6. Lớp báo cáo và lưu trữ: ghi output ra CSV hoặc PostgreSQL.

Sơ đồ logic của hệ thống:



Hình 3.3. Kiến trúc tổng thể hệ thống

Kiến trúc tổng thể được thiết kế theo hướng phân lớp. Mỗi lớp có trách nhiệm rõ ràng, giúp hệ thống dễ kiểm tra và mở rộng.

Lớp Data Source chịu trách nhiệm đọc dữ liệu. Với CSV, lớp này đọc file đầu vào và ghi các file output vào thư mục đích. Với PostgreSQL, lớp này đọc dữ liệu thông qua

source_query và ghi kết quả vào các bảng đã định nghĩa trong schema. Việc tách data source giúp pipeline xử lý không cần quan tâm dữ liệu đến từ đâu.

Lớp Preprocessing chịu trách nhiệm chuẩn hóa dữ liệu trước khi phát hiện lỗi. Đây là lớp rất quan trọng vì dữ liệu thực tế có thể có tên cột viết hoa, khoảng trắng, định dạng ngày khác nhau hoặc kiểu dữ liệu chưa đúng. Nếu bỏ qua bước này, detector và model phía sau dễ hoạt động sai.

Lớp Validation và Detection chịu trách nhiệm phát hiện lỗi. Rule-based validation xử lý missing, invalid và cross-field inconsistency. Anomaly detector xử lý các điểm bất thường bằng Isolation Forest. Feature contribution analyzer giúp giải thích cột nghi ngờ gây anomaly.

Lớp Recommendation và Fixing chịu trách nhiệm sinh gợi ý và áp dụng sửa lỗi. Missing recommender, invalid recommender và anomaly recommender có cách hoạt động khác nhau tùy loại issue. FixService là lớp cuối cùng quyết định có ghi suggested value vào dữ liệu đầu ra hay không.

Lớp Output chịu trách nhiệm sinh báo cáo và lưu kết quả. Output có thể là CSV hoặc bảng PostgreSQL, tùy nguồn dữ liệu được chọn khi chạy CLI.

Luồng dữ liệu trong kiến trúc này đi theo một chiều rõ ràng. Dữ liệu đầu vào được đọc từ source, sau đó được chuẩn hóa trước khi đưa vào validator và model. Kết quả phát hiện lỗi được đưa sang recommender để bổ sung suggested value. Sau đó report service tạo các bảng tổng hợp, còn fix service tạo dữ liệu sau xử lý nếu người dùng bật chế độ apply fixes. Nhờ luồng một chiều này, hệ thống dễ debug hơn vì mỗi bước có input và output rõ ràng.

Kiến trúc cũng tránh trộn lẫn trách nhiệm giữa các module. AnomalyDetector không trực tiếp sửa dữ liệu. Recommender không quyết định ghi dữ liệu vào PostgreSQL. PostgresDataSource không can thiệp vào logic phát hiện lỗi. Sự tách biệt này làm cho hệ thống dễ bảo trì và giúp báo cáo có thể giải thích từng phần một cách mạch lạc.

3.7. Thiết kế pipeline xử lý dữ liệu

Pipeline xử lý dữ liệu gồm nhiều bước nối tiếp nhau.

Bước đầu tiên là data loading. Nếu nguồn là CSV, hệ thống đọc file bằng pandas. Nếu nguồn là PostgreSQL, hệ thống sử dụng query trong file cấu hình để join các bảng nghiệp vụ và lấy dữ liệu đầu vào.

Bước thứ hai là normalization. Hệ thống chuẩn hóa tên cột, ép kiểu dữ liệu, parse ngày giao dịch và xử lý một số định dạng không thống nhất.

Bước thứ ba là feature engineering. Hệ thống sinh các feature như `unit_price`, `age_group`, `invoice_year`, `invoice_month`, `day_of_week`, `is_weekend`, `quantity_band` và `price_deviation_from_category`.

Bước thứ tư là rule-based validation. Các detector sẽ kiểm tra missing và invalid dựa trên cấu hình YAML.

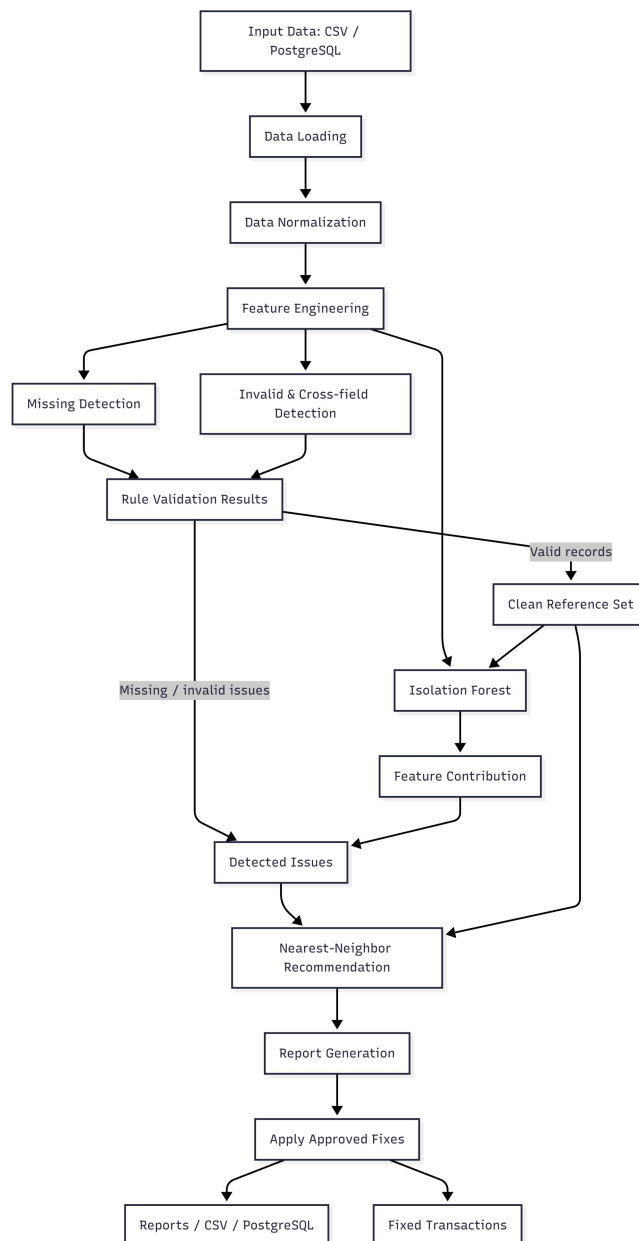
Bước thứ năm là clean reference set construction. Hệ thống loại bỏ những bản ghi có lỗi rõ ràng để xây dựng tập tham chiếu sạch hơn cho các bước gợi ý sửa và phát hiện anomaly.

Bước thứ sáu là anomaly detection. Isolation Forest được huấn luyện trên các feature phù hợp và trả về điểm bất thường cho từng bản ghi.

Bước thứ bảy là feature contribution analysis. Hệ thống phân tích các cột có khả năng đóng góp vào việc bản ghi bị xem là bất thường, giúp kết quả có tính giải thích tốt hơn.

Bước thứ tám là recommendation. Với các lỗi có thể gợi ý sửa, hệ thống tìm bản ghi tương tự bằng nearest neighbors và đề xuất giá trị phù hợp.

Bước cuối cùng là report generation và apply fixes. Hệ thống sinh các bảng output và tạo phiên bản dữ liệu đã sửa đổi với các lỗi đủ điều kiện.



Hình 3.4. Pipeline chi tiết

Điểm đáng chú ý là anomaly detector được fit sau khi đã loại các dòng có missing/invalid/cross-field issue. Đây là quyết định thiết kế nhằm tránh để các lỗi rõ ràng làm nhiễu tập huấn luyện của mô hình. Nếu mô hình học trên dữ liệu có nhiều lỗi rõ ràng, nó có thể xem các mẫu lỗi đó như một phần của phân phối bình thường, làm giảm chất lượng phát hiện anomaly.

Một điểm khác là hệ thống không tạo một anomaly issue duy nhất cho cả dòng. Thay vào đó, sau khi biết dòng nào bất thường, hệ thống dùng feature contribution để xác định cột nghi ngờ và tạo issue theo cột. Cách làm này giúp output cụ thể hơn, vì người dùng biết cần kiểm tra cột nào trong bản ghi.

3.8. Thiết kế cơ sở dữ liệu PostgreSQL

Cơ sở dữ liệu PostgreSQL được thiết kế nhằm bảo đảm ba mục tiêu chính. Mục tiêu thứ nhất là chuyển tập dữ liệu giao dịch dạng bảng phẳng thành mô hình quan hệ có cấu trúc, trong đó thông tin khách hàng, nhóm sản phẩm, trung tâm mua sắm và giao dịch được quản lý theo trách nhiệm riêng. Mục tiêu thứ hai là bảo toàn dữ liệu đầu vào để có thể truy vết từ dữ liệu thô đến dữ liệu đã chuẩn hóa. Mục tiêu thứ ba là lưu được các sản phẩm trung gian và đầu ra của pipeline data cleaning, bao gồm lỗi phát hiện, gợi ý sửa, lịch sử hành động, hồ sơ chất lượng dữ liệu và phiên bản giao dịch sau sửa.

Thiết kế này cho thấy PostgreSQL không chỉ là nơi đặt bảng transactions. Database tham gia vào toàn bộ vòng đời dữ liệu: tiếp nhận dữ liệu thô, lưu dữ liệu nghiệp vụ đã chuẩn hóa, cung cấp dữ liệu cho pipeline Python, nhận lại kết quả phát hiện lỗi và bảo toàn phiên bản dữ liệu sau xử lý. Vì vậy, schema phải đồng thời đáp ứng yêu cầu toàn vẹn dữ liệu nghiệp vụ và yêu cầu truy vết của quá trình làm sạch dữ liệu.

File `sql/postgresql_schema.sql` hiện thực hóa thiết kế trên bằng các lệnh `CREATE TABLE` và `CREATE INDEX`. File này không chứa lệnh `CREATE DATABASE`; do đó, database đích, ví dụ `doantn`, phải được tạo trước khi chạy script. Sau khi kết nối vào database đã tồn tại, script mới tạo các bảng, khóa, ràng buộc và index trong schema mặc định của phiên kết nối

3.8.1. Phân loại các nhóm bảng

Đối chiếu trực tiếp với `postgresql_schema.sql`, các bảng được phân thành bốn nhóm như sau:

Bảng 3.6. Phân nhóm các bảng trong schema PostgreSQL

Nhóm bảng	Các bảng thực tế trong schema	Vai trò trong hệ thống
Bảng nghiệp vụ	customers, products, shopping_malls, transactions	Lưu dữ liệu giao dịch đã chuẩn hóa và các thực thể tham chiếu
Bảng staging	staging_customer_shopping_raw	Giữ dữ liệu gần với nguồn, ưu tiên khả năng tiếp nhận và truy vết
Bảng quản lý phiên chạy	cleaning_runs	Lưu thông tin mô tả một lần chạy pipeline

Nhóm bảng	Các bảng thực tế trong schema	Vai trò trong hệ thống
Bảng phục vụ data cleaning	detected_issues, fix_recommendations, cleaning_actions, dataset_profile, fixed_transactions	Lưu kết quả phát hiện, đề xuất, hành động, thống kê chất lượng và dữ liệu sau sửa

Nhóm bảng nghiệp vụ sử dụng mô hình gần với star schema. transactions đóng vai trò bảng fact trung tâm, trong khi customers, products và shopping_malls cung cấp các chiều phân tích về khách hàng, danh mục sản phẩm và địa điểm mua sắm. Cách tổ chức này giảm lặp lại các thuộc tính mô tả trong từng giao dịch, đồng thời giúp truy vấn phân tích có cấu trúc rõ ràng hơn.

Bảng customers lưu thông tin khách hàng như mã, giới tính, tuổi, nhóm tuổi và phân khúc. Bảng products biểu diễn các danh mục sản phẩm đã chuẩn hóa, kèm đơn giá tham chiếu và nhóm giá. Bảng shopping_malls quản lý tên, phân hạng và mức độ phổ biến của trung tâm mua sắm. Bảng transactions liên kết ba bảng trên, đồng thời lưu thông tin giao dịch và các đặc trưng dẫn xuất phục vụ phân tích.

Bảng staging_customer_shopping_raw lưu dữ liệu nguồn trước khi làm sạch. Các trường nghiệp vụ chủ yếu sử dụng kiểu TEXT để tiếp nhận cả dữ liệu sai định dạng hoặc ngoài miền hợp lệ. Hai trường source_file và imported_at dùng để truy vết nguồn và thời điểm nạp dữ liệu.

Bảng cleaning_runs quản lý từng lần thực thi pipeline, bao gồm chế độ chạy, nguồn dữ liệu, thời gian, trạng thái và ghi chú. Trường run_id là điểm liên kết các lỗi và hồ sơ dữ liệu thuộc cùng một phiên chạy, qua đó hỗ trợ theo dõi và so sánh chất lượng dữ liệu theo thời gian.

Nhóm data cleaning phản ánh từng giai đoạn sau khi pipeline đọc dữ liệu. detected_issues lưu lỗi hoặc điểm nghi vấn ở mức dòng và cột. fix_recommendations lưu giá trị đề xuất tương ứng với issue. cleaning_actions lưu dấu vết hành động sửa, bao gồm giá trị cũ, giá trị mới, trạng thái và đối tượng thực hiện. dataset_profile lưu thống kê theo cột. fixed_transactions lưu phiên bản giao dịch sau khi áp dụng các sửa đổi đủ điều kiện.

final_report không phải là bảng được tạo bởi postgresql_schema.sql. Trong phạm vi schema SQL hiện tại, báo cáo tổng hợp được xem là sản phẩm đầu ra do tăng

Python tạo ra, thường ở dạng `final_report.csv`. Vì vậy, không nên liệt kê `final_report` như một bảng chính thức của schema PostgreSQL. Nếu hệ thống ghi một bảng cùng tên bảng cơ chế `to_sql` tại thời điểm chạy, đó là bảng được tạo động bởi mã ứng dụng, không phải thành phần được định nghĩa và bảo vệ bằng script schema.

3.8.2. Mô hình ERD tổng thể

Xét theo vai trò trong vòng đời dữ liệu, ERD của hệ thống có thể được đọc theo ba lớp logic. Lớp thứ nhất là lớp tiếp nhận dữ liệu nguồn, được đại diện bởi `staging_customer_shopping_raw`. Lớp thứ hai là lớp dữ liệu nghiệp vụ thương mại điện tử, gồm `customers`, `products`, `shopping_malls` và `transactions`. Lớp thứ ba là lớp quản lý và xử lý chất lượng dữ liệu, gồm `cleaning_runs`, `detected_issues`, `fix_recommendations`, `cleaning_actions`, `dataset_profile` và `fixed_transactions`. Cách phân lớp này giúp người đọc nhận biết rõ sự chuyển tiếp từ dữ liệu thô sang dữ liệu có cấu trúc, sau đó đến các kết quả phát hiện và sửa lỗi.

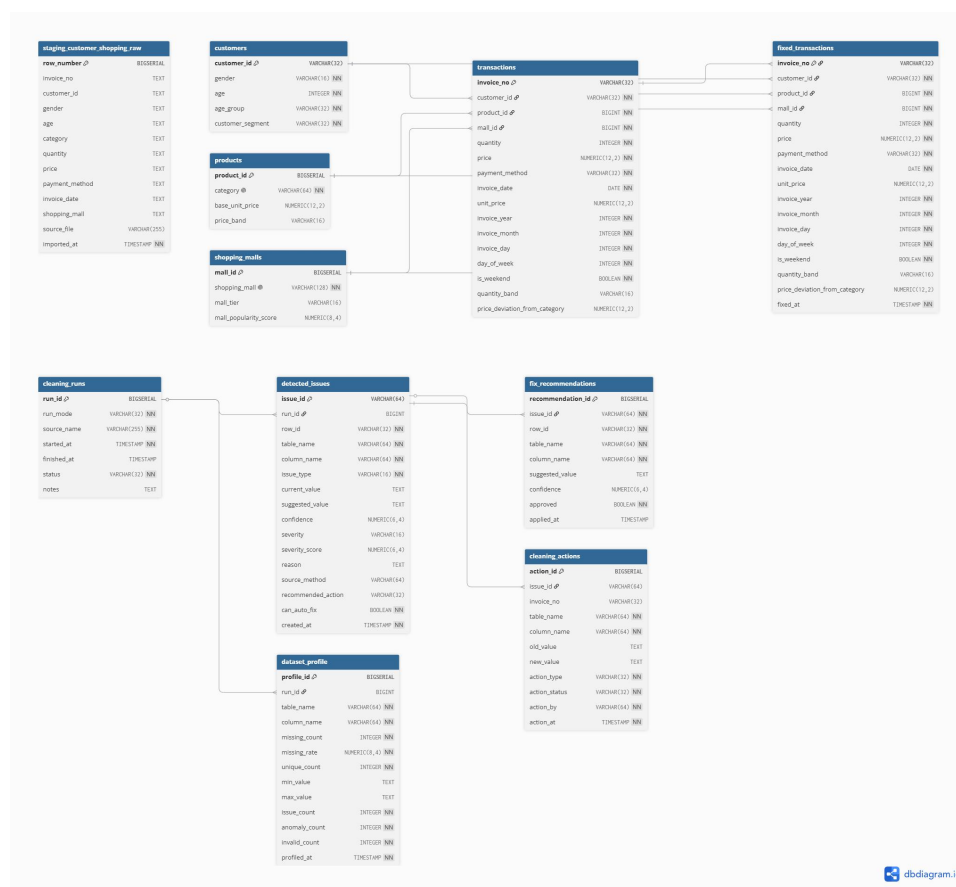
Trong lớp nghiệp vụ, `transactions` là bảng fact trung tâm. Mỗi giao dịch tham chiếu đúng một khách hàng, một danh mục sản phẩm và một trung tâm mua sắm; ngược lại, mỗi bản ghi trong ba bảng dimension có thể liên quan đến nhiều giao dịch. Mô hình này cho phép các thuộc tính mô tả được quản lý tập trung, hạn chế lặp dữ liệu và hỗ trợ truy vấn phân tích theo nhiều chiều.

Trong lớp data cleaning, `cleaning_runs` quản lý ngữ cảnh của từng lần thực thi pipeline. Một phiên chạy có thể phát sinh nhiều issue và nhiều bản ghi thống kê trong `dataset_profile`. Mỗi issue có thể dẫn đến một hoặc nhiều khuyến nghị sửa và có thể được liên kết với các hành động xử lý thực tế. Sau khi những sửa đổi đủ điều kiện được áp dụng, `fixed_transactions` lưu phiên bản giao dịch đã xử lý. Do `fixed_transactions.invoice_no` đồng thời là khóa chính và khóa ngoại đến `transactions.invoice_no`, một giao dịch gốc có tối đa một phiên bản tương ứng trong bảng này ở thiết kế hiện tại.

Trong nhóm dữ liệu nghiệp vụ, mỗi khách hàng, danh mục sản phẩm và trung tâm mua sắm có thể liên quan đến nhiều giao dịch thông qua các khóa ngoại `customer_id`, `product_id` và `mall_id` trong bảng `transactions`. Mỗi giao dịch chỉ có tối đa một phiên bản sau sửa trong `fixed_transactions`, do `invoice_no` của bảng này đồng thời là khóa chính và khóa ngoại.

Trong nhóm làm sạch dữ liệu, một phiên chạy trong `cleaning_runs` có thể tạo nhiều bản ghi `detected_issues` và `dataset_profile`. Mỗi issue có thể phát sinh nhiều `fix_recommendations` và `cleaning_actions`.

Bảng `staging_customer_shopping_raw` không có khóa ngoại đến các bảng nghiệp vụ vì dữ liệu tại đây có thể chưa hợp lệ. Việc chuyển dữ liệu từ staging sang các bảng nghiệp vụ được thực hiện bởi tầng import và transform. `final_report` cũng không xuất hiện trong ERD vì đây là tệp báo cáo do ứng dụng Python sinh ra, không phải bảng được định nghĩa trong `postgresql_schema.sql`.



Hình 3.5. ERD tổng thể của PostgreSQL

3.8.3. Thiết kế khóa chính, khóa ngoại và quan hệ

Khóa chính được lựa chọn theo hai hướng. Với thực thể đã có mã nghiệp vụ ổn định, schema sử dụng khóa tự nhiên. Cụ thể, `customers.customer_id` là khóa chính của khách hàng và `transactions.invoice_no` là khóa chính của giao dịch. Với thực thể được tạo trong quá trình chuẩn hóa, schema sử dụng khóa thay thế tự tăng: `products.product_id` và `shopping_malls.mall_id` có kiểu `BIGSERIAL`. Đồng thời,

products.category và shopping_malls.shopping_mall được đặt ràng buộc UNIQUE để ngăn tạo nhiều dòng cho cùng một danh mục hoặc cùng một trung tâm mua sắm.

Trong nhóm bảng quản lý và data cleaning, cleaning_runs.run_id, fix_recommendations.recommendation_id, cleaning_actions.action_id và dataset_profile.profile_id sử dụng BIGSERIAL. Riêng detected_issues.issue_id dùng VARCHAR(64) để phù hợp với mã issue do ứng dụng sinh ra. fixed_transactions.invoice_no vừa là khóa chính, vừa tham chiếu đến giao dịch gốc, qua đó bảo đảm mỗi giao dịch chỉ có tối đa một dòng tương ứng trong bảng dữ liệu sau sửa ở trạng thái schema hiện tại.

Các quan hệ khóa ngoại thực tế gồm:

Bảng 3.7. Các quan hệ khóa ngoại trong schema PostgreSQL

Bảng con và cột khóa ngoại	Bảng cha và cột được tham chiếu	Ý nghĩa
transactions.customer_id	customers.customer_id	Mỗi giao dịch thuộc một khách hàng đã tồn tại
transactions.product_id	products.product_id	Mỗi giao dịch thuộc một danh mục sản phẩm đã chuẩn hóa
transactions.mall_id	shopping_malls.mall_id	Mỗi giao dịch phát sinh tại một trung tâm mua sắm đã tồn tại
detected_issues.run_id	cleaning_runs.run_id	Gắn issue với phiên chạy, nhưng cột hiện cho phép NULL
fix_recommendations.issue_id	detected_issues.issue_id	Mỗi recommendation phải tham chiếu một issue hợp lệ
cleaning_actions.issue_id	detected_issues.issue_id	Gắn hành động với issue; cột hiện cho phép NULL
dataset_profile.run_id	cleaning_runs.run_id	Gắn thông kê chất lượng với phiên chạy; cột hiện cho phép NULL
fixed_transactions.invoice_no	transactions.invoice_no	Liên kết dữ liệu sau sửa với giao dịch gốc

Bảng con và cột khóa ngoại	Bảng cha và cột được tham chiếu	Ý nghĩa
fixed_transactions.customer_id	customers.customer_id	Bảo toàn khách hàng hợp lệ sau khi sửa
fixed_transactions.product_id	products.product_id	Bảo toàn danh mục hợp lệ sau khi sửa
fixed_transactions.mall_id	shopping_malls.mall_id	Bảo toàn trung tâm mua sắm hợp lệ sau khi sửa

Các khóa ngoại không khai báo ON DELETE hoặc ON UPDATE, vì vậy PostgreSQL sử dụng hành vi mặc định. Thiết kế này ưu tiên ngăn việc xóa bản ghi cha khi vẫn còn bản ghi con tham chiếu. Khi cần reset bảng cleaning, ứng dụng hoặc migration phải xóa bảng con trước bảng cha; đây là lý do `littlemigration.sql` xóa `cleaning_actions` và `fix_recommendations` trước `detected_issues`.

Một số giới hạn cần được hiểu đúng. `cleaning_actions.invoice_no` hiện chỉ là cột `VARCHAR(32)`, không được khai báo khóa ngoại đến `transactions.invoice_no`. Bảng staging cũng không có khóa ngoại vì dữ liệu trong đó chưa được xác nhận hợp lệ. Ngoài ra, `fixed_transactions` chưa có `run_id`, nên schema hiện tại phù hợp với việc lưu một phiên bản dữ liệu sau sửa hơn là lưu đồng thời lịch sử kết quả của nhiều phiên chạy.

3.8.4. Ràng buộc CHECK và vai trò kiểm soát dữ liệu

Các ràng buộc CHECK đưa một phần quy tắc chất lượng dữ liệu xuống tầng lưu trữ. Chúng ngăn dữ liệu rõ ràng không hợp lệ được ghi vào bảng nghiệp vụ hoặc bảng sau sửa, kể cả khi dữ liệu không đi qua đầy đủ pipeline Python.

Bảng 3.8. Các ràng buộc CHECK kiểm soát miền dữ liệu

Bảng và cột	Ràng buộc trong schema	Ý nghĩa kiểm soát
customers.age	CHECK (age BETWEEN 0 AND 120)	Chỉ chấp nhận tuổi trong miền hợp lý từ 0 đến 120
transactions.quantity	CHECK (quantity >= 1)	Giao dịch phải có ít nhất một đơn vị sản phẩm
transactions.price	CHECK (price >= 0)	Không cho phép giá trị giao dịch âm
transactions.invoice_month	CHECK (invoice_month	Giới hạn tháng trong miền lịch hợp lệ

Bảng và cột	Ràng buộc trong schema	Ý nghĩa kiểm soát
	BETWEEN 1 AND 12)	
transactions.invoice_day	CHECK (invoice_day BETWEEN 1 AND 31)	Giới hạn ngày trong miền tổng quát từ 1 đến 31
transactions.day_of_week	CHECK (day_of_week BETWEEN 0 AND 6)	Chỉ chấp nhận bảy giá trị biểu diễn thứ trong tuần
fixed_transactions.quantity, price, invoice_month, invoice_day, day_of_week	Lặp lại các CHECK tương ứng của transactions	Bảo đảm dữ liệu sau sửa không phá vỡ miền hợp lệ của giao dịch

Các ràng buộc này là lớp bảo vệ cuối cùng nhưng không thay thế toàn bộ data cleaning. Chẳng hạn, invoice_day BETWEEN 1 AND 31 không thể phát hiện ngày 31 trong một tháng chỉ có 30 ngày và cũng không kiểm tra sự thống nhất giữa invoice_date, invoice_month, invoice_day và day_of_week. Tương tự, price >= 0 chỉ phát hiện giá âm, không phát hiện mức giá dương nhưng bất thường so với danh mục sản phẩm. Những trường hợp phụ thuộc ngữ cảnh vẫn cần cross-field validation, feature engineering và anomaly detection trong pipeline.

Việc lặp lại các CHECK quan trọng ở fixed_transactions có ý nghĩa đặc biệt. Một recommendation chỉ nên được xem là áp dụng thành công khi giá trị sau sửa vẫn thỏa mãn ràng buộc cơ sở dữ liệu. Nếu gợi ý làm cho quantity nhỏ hơn 1 hoặc tháng nằm ngoài 1-12, PostgreSQL sẽ từ chối thao tác ghi, qua đó ngăn một lỗi mới được tạo ra trong quá trình làm sạch.

3.8.5. Thiết kế index hỗ trợ truy vấn

Ngoài index được PostgreSQL tạo ngầm cho khóa chính và ràng buộc UNIQUE, file schema tạo thêm các index phục vụ những hướng truy vấn thường gặp. Trên bảng transactions, ba index theo customer_id, product_id và mall_id hỗ trợ join từ bảng fact đến các bảng dimension, đồng thời tăng tốc các truy vấn lọc giao dịch theo khách hàng, danh mục hoặc trung tâm mua sắm. Index idx_transactions_invoice_date hỗ trợ truy vấn theo thời gian, còn idx_transactions_payment_method hỗ trợ thống kê hoặc lọc theo phương thức thanh toán.

Trên bảng `detected_issues`, `idx_detected_issues_run_id` hỗ trợ lấy toàn bộ issue thuộc một phiên chạy, trong khi `idx_detected_issues_row_id` hỗ trợ truy vết các lỗi liên quan đến một giao dịch cụ thể. Hai hướng truy vấn này phù hợp với nhu cầu của dashboard hoặc quy trình review: người dùng có thể xem tổng thể một lần chạy hoặc đi sâu vào lịch sử lỗi của một dòng dữ liệu.

Index làm tăng tốc độ đọc nhưng cũng làm tăng dung lượng lưu trữ và chi phí cập nhật khi chèn, sửa hoặc xóa dữ liệu. Vì vậy, schema chỉ tạo index cho các cột có vai trò join, lọc hoặc truy vết rõ ràng. File hiện tại không tạo index riêng cho mọi khóa ngoại trong nhóm cleaning; nếu dữ liệu cleaning tăng lớn và xuất hiện truy vấn chậm, cần đo bằng `EXPLAIN ANALYZE` trước khi bổ sung index thay vì tạo index theo cảm tính.

3.8.6. Luồng dữ liệu từ staging đến kết quả làm sạch

Luồng dữ liệu theo thiết kế bắt đầu tại `staging_customer_shopping_raw`. Dữ liệu từ Excel hoặc CSV được nạp vào staging với kiểu dữ liệu ít ràng buộc để bảo toàn hình thức ban đầu. Ở bước này, `source_file` và `imported_at` cung cấp thông tin truy vết. Staging không được dùng trực tiếp làm dữ liệu nghiệp vụ cuối cùng vì các cột số và ngày vẫn đang ở dạng văn bản.

Tiếp theo, tầng transform chuẩn hóa tên cột, ép kiểu, parse ngày, tạo feature và tách dữ liệu thành `customers`, `products`, `shopping_malls` và `transactions`. Các bảng dimension phải được ghi trước `transactions` để các khóa ngoại `customer_id`, `product_id` và `mall_id` có đối tượng tham chiếu hợp lệ. Khi dữ liệu đã vào nhóm bảng nghiệp vụ, các ràng buộc `NOT NULL`, `UNIQUE`, khóa ngoại và `CHECK` tạo lớp kiểm soát chất lượng ở mức database.

Pipeline Python đọc dữ liệu nghiệp vụ thông qua query join hoặc đọc trực tiếp từ CSV. Theo thiết kế quản lý phiên chạy, một dòng trong `cleaning_runs` được tạo để ghi nhận nguồn và trạng thái xử lý. Các detector sinh issue vào `detected_issues`; recommender sinh đề xuất vào `fix_recommendations`; quá trình profiling ghi thống kê vào `dataset_profile`; các hành động được duyệt có thể lưu trong `cleaning_actions`; cuối cùng, dữ liệu sau sửa được ghi vào `fixed_transactions`.

Luồng tổng quát quá trình được mô tả như sau:

Excel/CSV nguồn

- > staging_customer_shopping_raw
- > chuẩn hóa và tách bảng
- > customers + products + shopping_malls
- > transactions
- > pipeline rule-based + AI/ML
- > detected_issues + fix_recommendations + dataset_profile
- > cleaning_actions
- > fixed_transactions

Sự phân lớp này giúp dữ liệu sau sửa không ghi đè dữ liệu nghiệp vụ gốc. Người dùng có thể đối chiếu transactions với fixed_transactions, xem issue và recommendation liên quan, đồng thời quay lại staging khi cần xác minh giá trị ban đầu. Đây là cơ sở để database hỗ trợ audit và human-in-the-loop thay vì chỉ đóng vai trò kho chứa đầu vào.

Luồng staging và quản lý phiên chạy trên phản ánh kiến trúc dữ liệu đích của hệ thống. Trong phạm vi phiên bản mã nguồn hiện tại, scripts/import_ecommerce_data.py thực hiện chuẩn hóa rồi xuất bốn bảng nghiệp vụ thành CSV; bước tự động nạp staging_customer_shopping_raw, ghi dữ liệu nghiệp vụ vào PostgreSQL, tạo dòng trong cleaning_runs và gán run_id cho detected_issues hoặc dataset_profile chưa được tích hợp vào cùng một lệnh pipeline. Vì vậy, báo cáo phân biệt rõ giữa schema đã được thiết kế để hỗ trợ truy vết phiên chạy và mức độ tự động hóa đã hoàn thành trong chương trình Python. Việc nạp PostgreSQL có thể được thực hiện như một bước triển khai riêng, nhưng không được xem là chức năng tự động đã được kiểm chứng nếu chưa có log thực nghiệm tương ứng.

3.9. Thiết kế issue schema

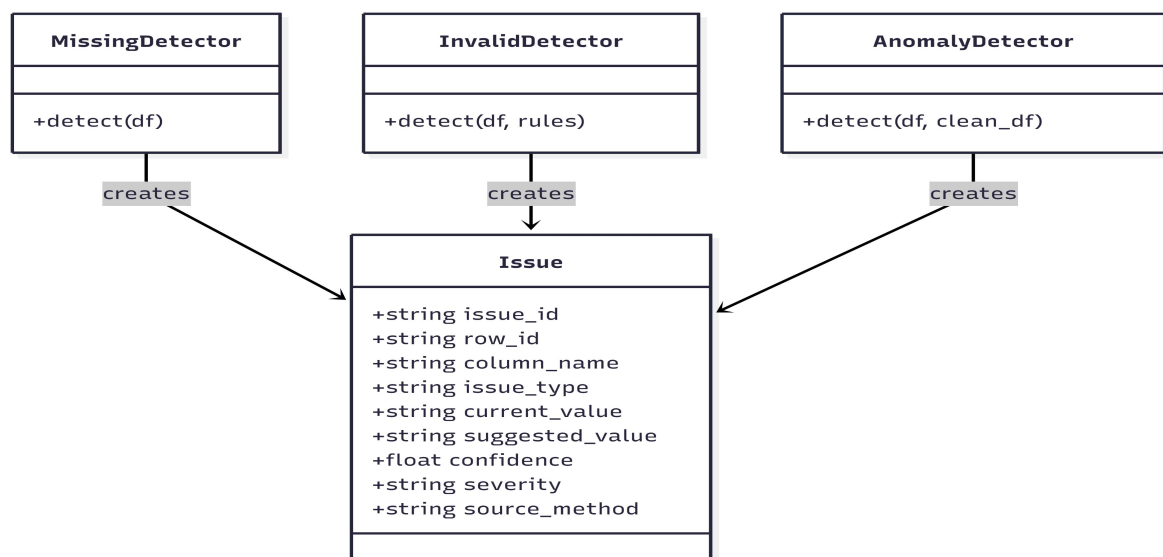
Mỗi lỗi được phát hiện trong hệ thống được biểu diễn dưới dạng một issue. Các trường chính của issue gồm:

Bảng 3.9. Cấu trúc của một bản ghi issue

Trường	Ý nghĩa
row_id	Mã dòng hoặc mã giao dịch liên quan
table_name	Bảng chứa dữ liệu
column_name	Cột phát sinh lỗi
issue_type	Loại lỗi: missing, invalid, anomaly

Trường	Ý nghĩa
current_value	Giá trị hiện tại
suggested_value	Giá trị gợi ý nếu có
confidence	Độ tin cậy
severity	Mức độ nghiêm trọng
source_method	Phương pháp phát hiện lỗi
recommended_action	Hành động khuyến nghị
can_auto_fix	Có thể tự động sửa hay không

Thiết kế issue schema giúp hệ thống thống nhất cách biểu diễn lỗi từ nhiều detector khác nhau. Nhờ đó, missing detector, invalid detector và anomaly detector đều có thể trả kết quả về cùng một định dạng.



Hình 3.6. Cấu trúc issue schema

3.10. Thiết kế severity và confidence

Severity thể hiện mức độ nghiêm trọng của lỗi. Các lỗi ảnh hưởng trực tiếp đến tính toán giao dịch như thiếu price, thiếu quantity hoặc tuổi vượt miền hợp lý thường được đánh giá ở mức cao hơn. Các lỗi categorical có thể vẫn ảnh hưởng đến phân tích nhóm, nhưng tùy trường hợp có thể được đánh giá ở mức medium.

Confidence thể hiện độ tin cậy của phát hiện hoặc gợi ý sửa. Với rule-based validation, confidence thường cao vì lỗi được xác định bằng điều kiện rõ ràng. Với

anomaly detection, confidence cần được hiểu cẩn trọng hơn vì mô hình chỉ chỉ ra mức độ khác biệt thống kê, không khẳng định chắc chắn dữ liệu sai.

3.11. Thiết kế cấu hình YAML

File cấu hình YAML đóng vai trò trung tâm trong việc mô tả rule, kiểu dữ liệu và tham số mô hình. Một số nội dung chính trong cấu hình gồm:

1. Tên bảng và cột định danh.
2. Query đọc dữ liệu từ PostgreSQL.
3. Tham số mô hình như contamination, số lượng cây, random state.
4. Danh sách cột, kiểu dữ liệu, cờ required và miền giá trị hợp lệ.
5. Cột dùng cho detection và recommendation.

Việc đưa cấu hình ra YAML giúp hệ thống linh hoạt hơn. Khi cần thay đổi miền giá trị hợp lệ hoặc thêm cột mới, người dùng có thể cập nhật file cấu hình thay vì sửa trực tiếp trong code.

Ví dụ, cấu hình cho cột `payment_method` gồm kiểu dữ liệu categorical, required là true và allowed values là Cash, Credit Card, Debit Card. Từ cấu hình này, MissingDetector biết rằng cột này không được thiếu, InvalidDetector biết rằng giá trị phải thuộc tập cho phép, còn recommender biết có thể dùng cột này trong quá trình tìm bản ghi tương tự nếu `use_for_recommendation` được bật.

Các cột như `invoice_no` và `customer_id` được đánh dấu không dùng cho detection/recommendation vì chúng là định danh. Nếu đưa các cột ID vào mô hình, khoảng cách hoặc one-hot encoding có thể bị nhiễu, vì ID thường không mang ý nghĩa tương đồng nghiệp vụ giữa các giao dịch.

File YAML cũng chứa `source_query` dùng khi đọc dữ liệu từ PostgreSQL. Query này join transactions, customers, products và shopping_malls để tạo dataframe đầu vào cho pipeline. Nhờ đó, pipeline luôn xử lý cùng một cấu trúc dữ liệu dù nguồn ban đầu là database đã chuẩn hóa.

3.12. Thiết kế cơ chế auto-fix có kiểm soát

Auto-fix trong hệ thống được thiết kế theo hướng thận trọng. Hệ thống chỉ áp dụng sửa tự động khi lỗi có thể sửa và đủ điều kiện tin cậy. Với các lỗi categorical/text, gợi ý từ nearest neighbors có thể được dùng để sửa nếu confidence phù hợp. Với dữ liệu

số bất thường, hệ thống không tự động thay bằng một giá trị khác nếu không có cơ sở chắc chắn.

Cách thiết kế này giúp giảm nguy cơ làm sai lệch dữ liệu gốc. Trong bài toán data cleaning, sửa sai đôi khi nguy hiểm hơn không sửa, vì một giá trị hiếm gặp vẫn có thể phản ánh một sự kiện thật. Vì vậy, hệ thống ưu tiên ghi nhận, giải thích và lưu lại hành động sửa để có thể audit sau này.

Hệ thống hiện hỗ trợ ba chế độ fix:

Bảng 3.10. Các chế độ áp dụng sửa lỗi

Chế độ	Ý nghĩa
conservative	Chỉ áp dụng fix khi confidence đạt ngưỡng cấu hình
auto	Áp dụng các fix có thể auto-fix mà không hỏi thêm
interactive	Hỏi người dùng xác nhận từng fix

Trong báo cáo và demo, chế độ conservative là phù hợp nhất vì cân bằng giữa tự động hóa và an toàn dữ liệu. Với chế độ này, hệ thống dùng `conservative_confidence_threshold` để quyết định có áp dụng suggested value hay không. Nếu confidence thấp hơn ngưỡng, issue vẫn được ghi nhận nhưng không làm thay đổi dữ liệu đầu ra.

Một nguyên tắc quan trọng là dữ liệu gốc không bị ghi đè. Kết quả sau fix được ghi ra `fixed_transactions`, còn dữ liệu gốc trong `transactions` hoặc file input vẫn được giữ lại. Điều này giúp người dùng có thể so sánh, rollback hoặc đánh giá lại nếu phát hiện gợi ý sửa chưa phù hợp.

3.13. Tính liên kết giữa các thành phần thiết kế

Sau khi phân tích từng thành phần riêng lẻ, có thể thấy thiết kế hệ thống được xây dựng theo một chuỗi liên kết logic. Cơ sở dữ liệu cung cấp dữ liệu đầu vào có cấu trúc. File cấu hình YAML mô tả cách hiểu các cột trong dữ liệu. Preprocessing biến dữ liệu về dạng thống nhất và sinh thêm feature. Validators phát hiện các lỗi chắc chắn. Clean reference set được tạo từ dữ liệu không có lỗi rõ ràng. Anomaly detector học từ clean reference set để phát hiện bản ghi bất thường. Feature contribution analyzer xác định cột

ngghi ngờ. Recommenders đưa ra gợi ý sửa lỗi. Report service và data source lưu kết quả ra output.

Điểm quan trọng là mỗi thành phần không hoạt động độc lập mà tạo dữ liệu đầu vào cho thành phần tiếp theo. Ví dụ, nếu preprocessing không chuẩn hóa `invoice_date`, cross-field validator sẽ không kiểm tra đúng các feature thời gian. Nếu clean reference set chứa nhiều missing/invalid row, anomaly detector có thể học sai phân phối bình thường. Nếu feature contribution không xác định cột nghi ngờ, anomaly issue sẽ khó giải thích. Nếu recommendation không có confidence, FixService không thể quyết định có nên apply hay không.

Như vậy, thiết kế hệ thống không chỉ là tập hợp nhiều module, mà là một pipeline có thứ tự và lý do rõ ràng. Đây là cơ sở để chuyển sang Chương 4, nơi từng module thiết kế sẽ được cài đặt thành mã nguồn cụ thể.

3.14. Kết luận chương

Chương 3 đã trình bày phân tích bài toán và thiết kế hệ thống. Hệ thống được thiết kế theo hướng pipeline rõ ràng, tách biệt giữa nguồn dữ liệu, tiền xử lý, phát hiện lỗi, gợi ý sửa và lưu kết quả. Cơ sở dữ liệu PostgreSQL được tổ chức thành các nhóm bảng nghiệp vụ, staging và data cleaning, giúp hệ thống có khả năng truy vết và mở rộng. Đây là nền tảng để triển khai các module cụ thể ở chương tiếp theo.

CHƯƠNG 4. CÀI ĐẶT HỆ THỐNG

Chương 4 trình bày quá trình cài đặt hệ thống, bao gồm môi trường phát triển, cấu trúc thư mục, các module chính và một số vấn đề kỹ thuật đã xử lý trong quá trình triển khai.

Nếu Chương 3 tập trung vào thiết kế, Chương 4 trình bày cách thiết kế đó được hiện thực hóa thành mã nguồn. Nội dung chương được sắp xếp theo đúng luồng vận hành của hệ thống: công nghệ sử dụng, cấu trúc project, import dữ liệu, truy cập dữ liệu, tiền xử lý, phát hiện lỗi, phát hiện anomaly, gợi ý sửa, sinh báo cáo, apply fixes và CLI. Cách sắp xếp này giúp người đọc có thể đi từ mức tổng quan đến từng module cụ thể mà không bị tách rời khỏi pipeline đã thiết kế ở chương trước.

4.1. Công nghệ sử dụng

Hệ thống được xây dựng bằng các công nghệ chính sau:

Bảng 4.1. Các công nghệ sử dụng trong hệ thống

Công nghệ	Vai trò
Python	Ngôn ngữ lập trình chính
pandas	Xử lý dữ liệu dạng bảng
NumPy	Hỗ trợ tính toán số
scikit-learn	Cài đặt Isolation Forest và nearest neighbors
PostgreSQL	Lưu trữ dữ liệu và kết quả
SQLAlchemy	Kết nối Python với PostgreSQL
YAML	Cấu hình rule và tham số pipeline
PowerShell/CLI	Chạy pipeline và kiểm thử

Các công nghệ trên được lựa chọn theo tiêu chí đơn giản, phù hợp với dữ liệu dạng bảng và dễ triển khai trong phạm vi đồ án. Python là lựa chọn phù hợp vì có cú pháp rõ ràng, hệ sinh thái thư viện phong phú và khả năng ứng dụng thuận lợi cho xử lý dữ liệu [14]. pandas hỗ trợ thao tác với dữ liệu bảng giống với cách làm việc trên Excel hoặc SQL table. scikit-learn cung cấp các thuật toán học máy ổn định, trong đó có Isolation Forest và NearestNeighbors. PostgreSQL đóng vai trò lưu trữ dữ liệu có cấu

trúc và kết quả cleaning. SQLAlchemy giúp kết nối giữa Python và PostgreSQL mà không phụ thuộc quá nhiều vào chi tiết của driver.

Một ưu điểm của stack công nghệ này là tính tái lập. Người dùng có thể cài đặt dependencies từ requirements.txt, chạy CLI bằng PowerShell và kiểm tra output trong thư mục outputs. Điều này phù hợp với bối cảnh bảo vệ đồ án, khi hệ thống cần có khả năng chạy demo trên máy cá nhân hoặc máy của giảng viên.

4.2. Cấu trúc thư mục project

Cấu trúc project được tổ chức theo các nhóm chức năng:

Bảng 4.2. Cấu trúc thư mục chính của project

Thư mục/tập tin	Vai trò
app/cli.py	Entry point chạy pipeline
app/services/pipeline_service.py	Điều phối luồng phát hiện lỗi và gợi ý sửa
app/validators/	Các detector rule-based
app/detectors/	Module anomaly detection và feature contribution
app/recommenders/	Module sinh gợi ý sửa lỗi
app/data_access/	Đọc/ghi dữ liệu từ CSV hoặc PostgreSQL
configs/ecommerce_config.yaml	Cấu hình chính cho dataset e-commerce
scripts/import_ecommerce_data.py	Script chuẩn hóa và tách dataset thành bảng
sql/postgresql_schema.sql	Script khởi tạo toàn bộ schema PostgreSQL
sql/littlemigration.sql	Script reset một nhóm bảng phục vụ data cleaning
outputs/	Kết quả chạy pipeline
docs/	Tài liệu hướng dẫn và báo cáo

Cấu trúc thư mục được thiết kế theo hướng tách trách nhiệm. Các module trong validators chỉ xử lý rule-based detection, các module trong detectors xử lý anomaly detection và feature contribution, các module trong recommenders xử lý gợi ý sửa lỗi, còn các service trong services điều phối luồng nghiệp vụ. Cách tổ chức này giúp mã nguồn dễ đọc hơn so với việc đặt toàn bộ logic trong một file duy nhất.

Trong `data_access`, hệ thống định nghĩa các nguồn dữ liệu khác nhau. `CSVDataSource` phù hợp với chế độ chạy từ file, còn `PostgresDataSource` phù hợp với chế độ chạy từ database. Cả hai cùng cung cấp các thao tác đọc và ghi, giúp phần pipeline phía trên không cần biết output cuối cùng sẽ được ghi ra CSV hay PostgreSQL.

Thư mục `configs` chứa file YAML. Đây là nơi mô tả quy tắc của dataset e-commerce, bao gồm `table_name`, `id_column`, `source_query`, tham số mô hình và rule của từng cột. Việc đặt cấu hình riêng giúp hệ thống có thể mở rộng bằng cách thêm file config mới cho dataset khác.

4.3. Cài đặt và khởi tạo cơ sở dữ liệu PostgreSQL

Sau khi cấu trúc database được phân tích ở Chương 3, bước cài đặt cần chuyển mô hình bảng và quan hệ thành các đối tượng thực tế trong PostgreSQL. Project sử dụng hai file SQL với phạm vi khác nhau: `sql/postgresql_schema.sql` để tạo toàn bộ schema trên một database mới hoặc database chưa có các bảng của hệ thống, và `sql/littlemigration.sql` để reset một nhóm bảng data cleaning khi cần đồng bộ lại cấu trúc trong quá trình phát triển.

Hai file không thay thế cho nhau. `postgresql_schema.sql` là điểm khởi đầu của một môi trường PostgreSQL mới, còn `littlemigration.sql` là thao tác cập nhật có tính phá hủy dữ liệu đối với một phần schema đã tồn tại. Việc phân biệt rõ hai mục đích này giúp tránh sử dụng migration như một script khởi tạo hoàn chỉnh hoặc vô tình xóa dữ liệu cleaning khi chỉ muốn xem cấu trúc bảng.

4.3.1. Vai trò của file `postgresql_schema.sql`

File `postgresql_schema.sql` chứa toàn bộ lệnh tạo các bảng được phân tích ở Mục 3.8, các khóa chính, khóa ngoại, ràng buộc NOT NULL, UNIQUE, CHECK và các index được lựa chọn. Script được dùng sau khi người quản trị đã tạo database đích và kết nối đúng vào database đó. Nói cách khác, file khởi tạo schema của hệ thống bên trong database, nhưng không tự tạo database vì không có lệnh `CREATE DATABASE`.

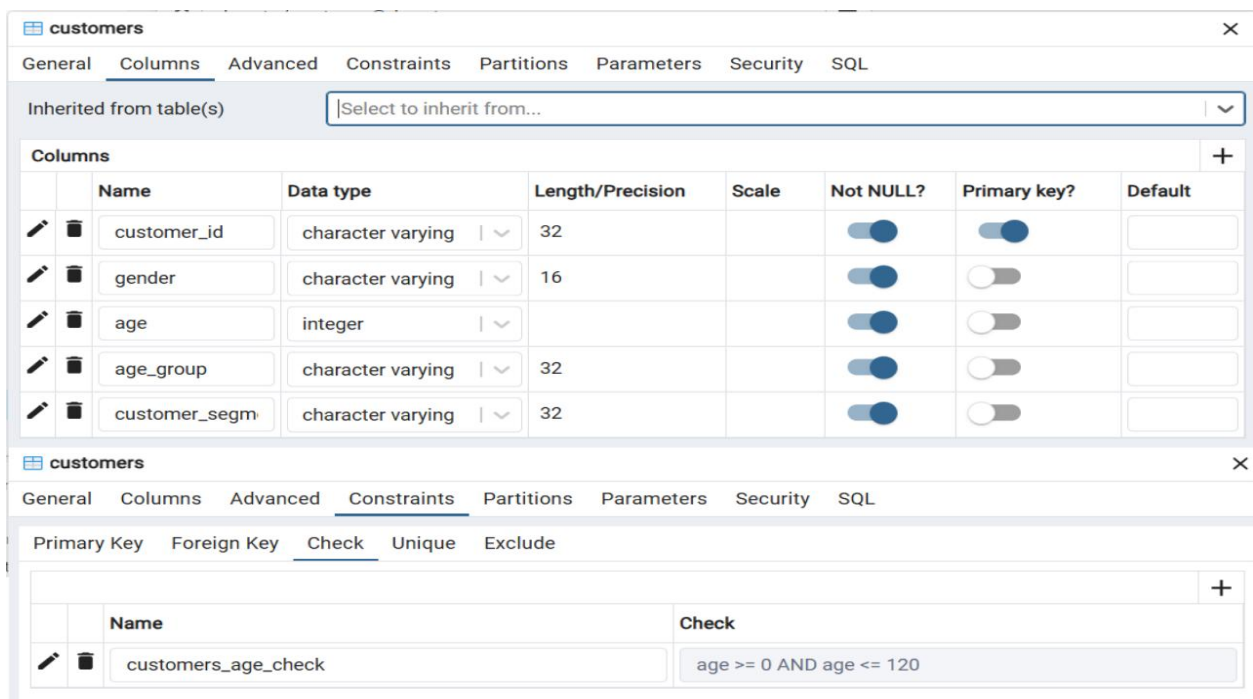
Trình tự các lệnh trong file có chủ đích. Các bảng `dimension customers`, `products` và `shopping_malls` được tạo trước `transactions` để các khóa ngoại của bảng giao dịch có thể tham chiếu đến đối tượng đã tồn tại. Bảng `cleaning_runs` được tạo trước `detected_issues` và `dataset_profile`; `detected_issues` được tạo trước

`fix_recommendations` và `cleaning_actions`. Thứ tự này phù hợp với đồ thị phụ thuộc khóa ngoại và giúp script có thể chạy tuần tự trên database chưa có schema.

Đoạn SQL tạo bảng khách hàng như sau:

```
CREATE TABLE customers (  
  customer_id VARCHAR(32) PRIMARY KEY,  
  gender VARCHAR(16) NOT NULL,  
  age INTEGER NOT NULL CHECK (age BETWEEN 0 AND 120),  
  age_group VARCHAR(32) NOT NULL,  
  customer_segment VARCHAR(32) NOT NULL  
);
```

Trong bảng này, `customer_id` là mã nghiệp vụ nên được dùng trực tiếp làm khóa chính thay vì tạo một mã tự tăng khác. Các trường `gender`, `age`, `age_group` và `customer_segment` đều bắt buộc, phản ánh yêu cầu rằng một khách hàng đã được đưa vào tầng nghiệp vụ phải có đủ thông tin cần thiết cho phân tích. Ràng buộc tuổi từ 0 đến 120 ngăn dữ liệu ngoài miền hợp lý được ghi vào bảng chuẩn, nhưng việc đánh giá tuổi có phù hợp với một giao dịch cụ thể hay không vẫn thuộc trách nhiệm của pipeline.



Hình 4.1. Cấu trúc bảng customers trong pgAdmin

Query Query History

```

1  SELECT
2      customer_id,
3      gender,
4      age,
5      age_group,
6      customer_segment
7  FROM public.customers
8  ORDER BY customer_id
9  LIMIT 10;

```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	customer_id [PK] character varying (32)	gender character varying (16)	age integer	age_group character varying (32)	customer_segment character varying (32)
1	C100004	Male	61	56_plus	senior
2	C100005	Male	34	26_35	adult
3	C100006	Male	44	36_45	adult
4	C100012	Male	25	18_25	young_adult
5	C100019	Female	21	18_25	young_adult
6	C100025	Male	55	46_55	senior
7	C100028	Female	21	18_25	young_adult
8	C100030	Male	46	46_55	senior
9	C100034	Male	60	56_plus	senior
10	C100041	Female	50	46_55	senior

Hình 4.2. Dữ liệu mẫu trong bảng customers

Bảng transactions là bảng trung tâm và có cấu trúc phức tạp hơn:

```

CREATE TABLE transactions (
    invoice_no VARCHAR(32) PRIMARY KEY,
    customer_id VARCHAR(32) NOT NULL REFERENCES customers(customer_id),
    product_id BIGINT NOT NULL REFERENCES products(product_id),
    mall_id BIGINT NOT NULL REFERENCES shopping_malls(mall_id),
    quantity INTEGER NOT NULL CHECK (quantity >= 1),

```

```

price NUMERIC(12, 2) NOT NULL CHECK (price >= 0),
payment_method VARCHAR(32) NOT NULL,
invoice_date DATE NOT NULL,
unit_price NUMERIC(12, 2),
invoice_year INTEGER NOT NULL,
invoice_month INTEGER NOT NULL CHECK (invoice_month BETWEEN 1 AND 12),
invoice_day INTEGER NOT NULL CHECK (invoice_day BETWEEN 1 AND 31),
day_of_week INTEGER NOT NULL CHECK (day_of_week BETWEEN 0 AND 6),
is_weekend BOOLEAN NOT NULL,
quantity_band VARCHAR(16),
price_deviation_from_category NUMERIC(12, 2)
);

```

`invoice_no` là khóa chính, giúp mỗi giao dịch được định danh duy nhất và là cơ sở đối chiếu với issue cũng như dữ liệu sau sửa. Ba dòng khai báo REFERENCES tạo các khóa ngoại từ giao dịch đến khách hàng, sản phẩm và trung tâm mua sắm:

```

customer_id VARCHAR(32) NOT NULL REFERENCES customers(customer_id),
product_id BIGINT NOT NULL REFERENCES products(product_id),
mall_id BIGINT NOT NULL REFERENCES shopping_malls(mall_id),

```

Các khóa ngoại bảo đảm một giao dịch không thể tham chiếu khách hàng, danh mục hoặc trung tâm mua sắm chưa tồn tại. Điều này cũng quyết định thứ tự import: dữ liệu dimension phải được ghi trước dữ liệu giao dịch. Các trường thời gian dẫn xuất được lưu cùng `invoice_date` để hỗ trợ truy vấn và mô hình học máy, nhưng phải được cross-field validator kiểm tra để tránh trường hợp ngày gốc và thành phần dẫn xuất không nhất quán.

Trong nhóm data cleaning, bảng `detected_issues` là điểm nối giữa kết quả phát hiện và các bước xử lý sau:

```

CREATE TABLE detected_issues (
  issue_id VARCHAR(64) PRIMARY KEY,
  run_id BIGINT REFERENCES cleaning_runs(run_id),
  row_id VARCHAR(32) NOT NULL,
  table_name VARCHAR(64) NOT NULL,
  column_name VARCHAR(64) NOT NULL,
  issue_type VARCHAR(16) NOT NULL,
  current_value TEXT,
  suggested_value TEXT,
  confidence NUMERIC(6, 4),
  severity VARCHAR(16),
  severity_score NUMERIC(6, 4),
  reason TEXT,
  source_method VARCHAR(64),
  recommended_action VARCHAR(32),
  can_auto_fix BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

Thiết kế này thống nhất đầu ra của missing detector, invalid detector, cross-field validator và anomaly detector. `row_id`, `table_name` và `column_name` xác định vị trí dữ liệu cần xem xét; `issue_type`, `reason` và `source_method` giải thích nguyên nhân; `confidence`,

severity và severity_score hỗ trợ ưu tiên review; suggested_value, recommended_action và can_auto_fix phục vụ bước gợi ý và áp dụng sửa. run_id cho phép liên kết issue với phiên chạy, nhưng hiện được phép NULL, phù hợp với giai đoạn cài đặt mà run tracking chưa được tích hợp đầy đủ.

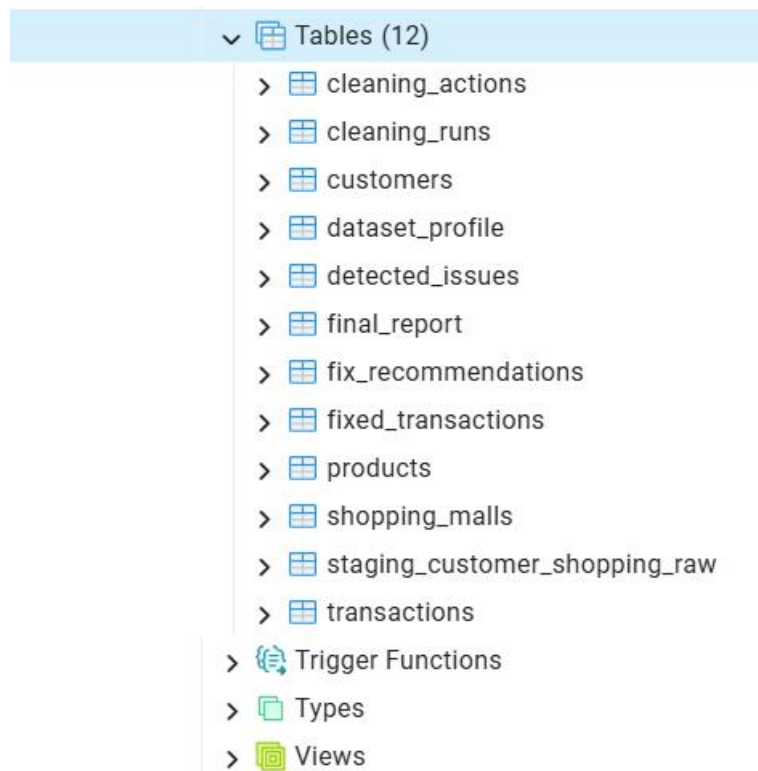
Một ví dụ index tiêu biểu trong schema là:

```
CREATE INDEX idx_detected_issues_run_id
ON detected_issues (run_id);
```

Index này giúp truy vấn tất cả issue của một phiên chạy mà không phải quét toàn bộ bảng detected_issues. Cùng nguyên tắc đó, schema có index theo row_id để truy vết lỗi của một giao dịch và các index trên transactions để hỗ trợ join hoặc lọc theo khách hàng, sản phẩm, trung tâm mua sắm, ngày giao dịch và phương thức thanh toán.

4.3.2. Chạy schema bằng pgAdmin và psql

Để chạy bằng pgAdmin, trước hết người dùng tạo database doantn nếu database chưa tồn tại. Sau đó, trong cây Browser, nhấp phải database doantn và chọn **Query Tool**. Tại cửa sổ Query Tool, chọn **Open File**, mở sql/postgresql_schema.sql, kiểm tra tên database đang hiển thị trên thanh kết nối rồi chọn **Execute Script**. Khi script hoàn tất, cần xem tab **Messages** để bảo đảm không có lỗi và chọn **Refresh** tại nhánh **Schemas > public > Tables** để tải lại danh sách bảng.



Hình 4.3. Danh sách bảng PostgreSQL trong pgAdmin

Nếu sử dụng dòng lệnh, người dùng mở PowerShell tại thư mục gốc project và chạy:

```
psql -U postgres -d doantn -f sql/postgresql_schema.sql
```

Tham số -U xác định người dùng PostgreSQL, -d xác định database đích và -f yêu cầu psql đọc lệnh từ file. Để script dừng ngay khi gặp lỗi thay vì tiếp tục chạy các câu lệnh sau, có thể dùng:

```
psql -v ON_ERROR_STOP=1 -U postgres -d doantn -f sql/postgresql_schema.sql
```

File schema hiện không sử dụng CREATE TABLE IF NOT EXISTS. Vì vậy, nó phù hợp nhất với database mới hoặc schema chưa có các bảng cùng tên. Nếu chạy lại trên schema đã được khởi tạo, PostgreSQL sẽ báo lỗi đối tượng đã tồn tại. Trong trường hợp cần bảo toàn dữ liệu, người dùng phải dùng migration có kiểm soát hoặc sao lưu rồi xử lý từng thay đổi, không nên xóa toàn bộ bảng chỉ để chạy lại schema.

4.3.3. Vai trò và phạm vi của file `littlemigration.sql`

`littlemigration.sql` là script migration nhỏ dùng trong quá trình phát triển để cập nhật hoặc khởi tạo lại một nhóm bảng phục vụ data cleaning. File này không tạo bảng nghiệp vụ, bảng staging, `cleaning_runs` hoặc `fixed_transactions`; do đó, nó không phải script tạo database hoàn chỉnh và không thể thay thế `postgresql_schema.sql`.

Phần đầu migration thực hiện:

```
DROP TABLE IF EXISTS cleaning_actions;  
DROP TABLE IF EXISTS fix_recommendations;  
DROP TABLE IF EXISTS recommendations;  
DROP TABLE IF EXISTS dataset_profile;  
DROP TABLE IF EXISTS detected_issues;
```

Thứ tự xóa phản ánh quan hệ phụ thuộc. `cleaning_actions` và `fix_recommendations` có khóa ngoại đến `detected_issues`, nên phải được xóa trước bảng `issue`. `dataset_profile` độc lập với `detected_issues` nhưng cùng thuộc nhóm cần reset. Dòng `DROP TABLE IF EXISTS recommendations` dùng để dọn tên bảng cũ nếu phiên bản trước của hệ thống từng ghi recommendation vào bảng `recommendations`; schema hiện tại sử dụng tên chính thức là `fix_recommendations`.

Sau bước xóa, script tạo lại `detected_issues`, hai index của bảng này, `fix_recommendations`, `cleaning_actions` và `dataset_profile`. Các định nghĩa được đồng bộ với nhóm bảng tương ứng trong `postgresql_schema.sql`. do `detected_issues.run_id` và `dataset_profile.run_id` tham chiếu `cleaning_runs(run_id)`, bảng `cleaning_runs` phải tồn tại trước khi migration chạy. Điều kiện này thường được đáp ứng khi database đã từng được khởi tạo bằng schema đầy đủ.

Migration có tính phá hủy dữ liệu. `DROP TABLE IF EXISTS` xóa cả cấu trúc lẫn toàn bộ dữ liệu hiện có trong `cleaning_actions`, `fix_recommendations`, `dataset_profile` và `detected_issues`. Dữ liệu trong bảng cũ `recommendations` cũng bị xóa nếu bảng đó còn tồn tại. Script không chứa bước sao chép dữ liệu sang bảng tạm, không có câu lệnh phục hồi và cũng không tự tạo bản sao lưu. Vì vậy, migration chỉ nên chạy sau khi đã sao lưu hoặc khi chắc chắn các kết quả `cleaning` cũ có thể được tạo lại từ pipeline.

Lệnh chạy migration bằng `psql` là:

```
psql -v ON_ERROR_STOP=1 -U postgres -d doantn -f sql/littlemigration.sql
```

Khi chạy trong pgAdmin, thao tác mở file và thực thi tương tự schema đầy đủ, nhưng người dùng cần kiểm tra chắc chắn đang kết nối đúng database, đồng thời xem lại dữ liệu cần bảo toàn trước khi chọn **Execute Script**. Với môi trường có dữ liệu quan trọng, nên bọc quy trình migration trong kế hoạch sao lưu, kiểm thử ở database thử nghiệm và xác nhận sau migration.

4.3.4. So sánh hai file SQL

Bảng 4.3. So sánh postgresql_schema.sql và littlemigration.sql

Tiêu chí	postgresql_schema.sql	littlemigration.sql
Mục đích	Khởi tạo toàn bộ schema của hệ thống trong một database đã tồn tại	Cập nhật hoặc reset một nhóm bảng data cleaning
Phạm vi tác động	Bảng nghiệp vụ, staging, quản lý phiên chạy, data cleaning và các index được định nghĩa	Chỉ nhóm detected_issues, fix_recommendations, cleaning_actions, dataset_profile; xóa thêm tên cũ recommendations nếu có
Các bảng được xử lý	customers, products, shopping_malls, transactions, staging_customer_shopping_raw, cleaning_runs, detected_issues, fix_recommendations, cleaning_actions, dataset_profile, fixed_transactions	Xóa rồi tạo lại detected_issues, fix_recommendations, cleaning_actions, dataset_profile; chỉ xóa recommendations cũ
Có xóa dữ liệu hay không	Không có lệnh DROP; script sẽ báo lỗi nếu bảng cùng tên đã tồn tại	Có; dữ liệu trong các bảng bị DROP TABLE sẽ mất hoàn toàn
Điều kiện chạy	Database đích phải được tạo và người chạy phải có quyền tạo bảng/index	Schema nền phải tồn tại, đặc biệt phải có cleaning_runs để tạo khóa ngoại
Thời điểm sử dụng	Khi thiết lập môi trường PostgreSQL mới hoặc khởi tạo database thử nghiệm sạch	Khi cần đồng bộ/reset cấu trúc bảng cleaning trong quá trình phát triển và đã chấp nhận mất dữ liệu liên quan

Sự phân biệt trên gắn trực tiếp với pipeline. Schema đầy đủ tạo nền tảng lưu trữ xuyên suốt từ dữ liệu thô đến dữ liệu sau sửa. Migration chỉ tác động đến khu vực lưu kết quả phân tích và review, không thay đổi dữ liệu nghiệp vụ trong transactions hoặc dữ liệu sau sửa trong fixed_transactions. Nhờ đó, người phát triển có thể điều chỉnh issue schema mà không phải tạo lại toàn bộ database, nhưng phải chấp nhận mất lịch sử cleaning thuộc các bảng bị reset.

Trong cài đặt hiện tại, `PostgresDataSource` xem năm bảng `detected_issues`, `fix_recommendations`, `cleaning_actions`, `dataset_profile` và `fixed_transactions` là các bảng có schema cố định. `final_report` không được định nghĩa trong `postgresql_schema.sql`; khi chạy với PostgreSQL, nhánh ghi tổng quát có thể tạo hoặc thay thế bảng này động bằng `pandas`. Do đó, báo cáo chỉ xem `final_report.csv` là đầu ra được bảo đảm của pipeline và không liệt kê `final_report` như một thành phần bắt buộc của schema PostgreSQL. Nếu hệ thống được phát triển tiếp theo hướng lưu báo cáo tổng hợp lâu dài trong database, bảng này cần được bổ sung bằng một migration có cấu trúc và chính sách lưu lịch sử rõ ràng.

4.4. Module import dữ liệu

Module import dữ liệu có nhiệm vụ đọc dataset nguồn, chuẩn hóa tên cột, parse ngày giao dịch, sinh một số feature và tách dữ liệu thành các bảng quan hệ. Dataset ban đầu là một bảng phẳng, nhưng sau khi xử lý được chia thành `customers`, `products`, `shopping_malls` và `transactions`.

Trong quá trình cài đặt, một vấn đề quan trọng là xử lý trường `invoice_date`. Dataset sử dụng định dạng ngày cần parse theo `dayfirst=True`. Nếu parse sai, hệ thống có thể tạo ra năm hoặc tháng không đúng, ảnh hưởng đến feature thời gian và kết quả anomaly detection. Vì vậy, hệ thống đã bổ sung helper parse ngày thống nhất và lưu thêm giá trị ngày thô để hỗ trợ debug.

```
def transform_dataset(input_path: Path) -> dict[str, pd.DataFrame]:
    df = pd.read_excel(input_path)
    df = normalize_columns(df)

    df["invoice_date_raw"] = df["invoice_date"]
    df["invoice_date"] = parse_ecommerce_invoice_date(df["invoice_date_raw"])
    invalid_date_count = int(df["invoice_date"].isna().sum())
    if invalid_date_count > 0:
        sample_bad_dates = df.loc[
            df["invoice_date"].isna(),
            ["invoice_no", "invoice_date_raw"],
        ].head(10)
        raise ValueError(
            f"Found {invalid_date_count} invalid invoice_date values after parsing. "
            f"Sample:\n{sample_bad_dates}"
        )

    df["age"] = pd.to_numeric(df["age"], errors="coerce").astype("Int64")
    df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce").astype("Int64")
    df["price"] = pd.to_numeric(df["price"], errors="coerce")
```

Hình 4.5. Ảnh code module import dữ liệu

Quá trình import dữ liệu được thực hiện theo các bước chính. Đầu tiên, dữ liệu được đọc từ file Excel bằng pandas. Sau đó, tên cột gốc được ánh xạ sang tên cột chuẩn hóa. Việc chuẩn hóa tên cột giúp các module phía sau không phải xử lý nhiều biến thể như INVOICE_NO, invoice no hoặc invoice_no.

Tiếp theo, hệ thống xử lý trường invoice_date. Đây là bước quan trọng vì nếu ngày giao dịch bị parse sai, toàn bộ các feature thời gian như năm, tháng, ngày, thứ trong tuần và cuối tuần sẽ sai theo. Hàm parse_ecommerce_invoice_date được thiết kế để nhận diện trường hợp ngày ở dạng ISO và trường hợp ngày theo định dạng phổ biến có dayfirst=True. Nếu sau khi parse vẫn còn giá trị lỗi, script import sẽ dừng và báo mẫu dữ liệu sai để người dùng kiểm tra.

Sau khi chuẩn hóa kiểu dữ liệu, hệ thống sinh các feature như unit_price, invoice_year, invoice_month, invoice_day, day_of_week, is_weekend, age_group, customer_segment và quantity_band. Các feature này vừa phục vụ phân tích nghiệp vụ, vừa phục vụ pipeline phát hiện lỗi.

Cuối cùng, dữ liệu được tách thành bốn bảng:

1. customers: lấy các thông tin duy nhất theo customer_id.
2. products: tạo từ thống kê theo category.
3. shopping_malls: tạo từ thống kê theo shopping_mall.
4. transactions: bảng giao dịch chính, liên kết với các bảng dimension.

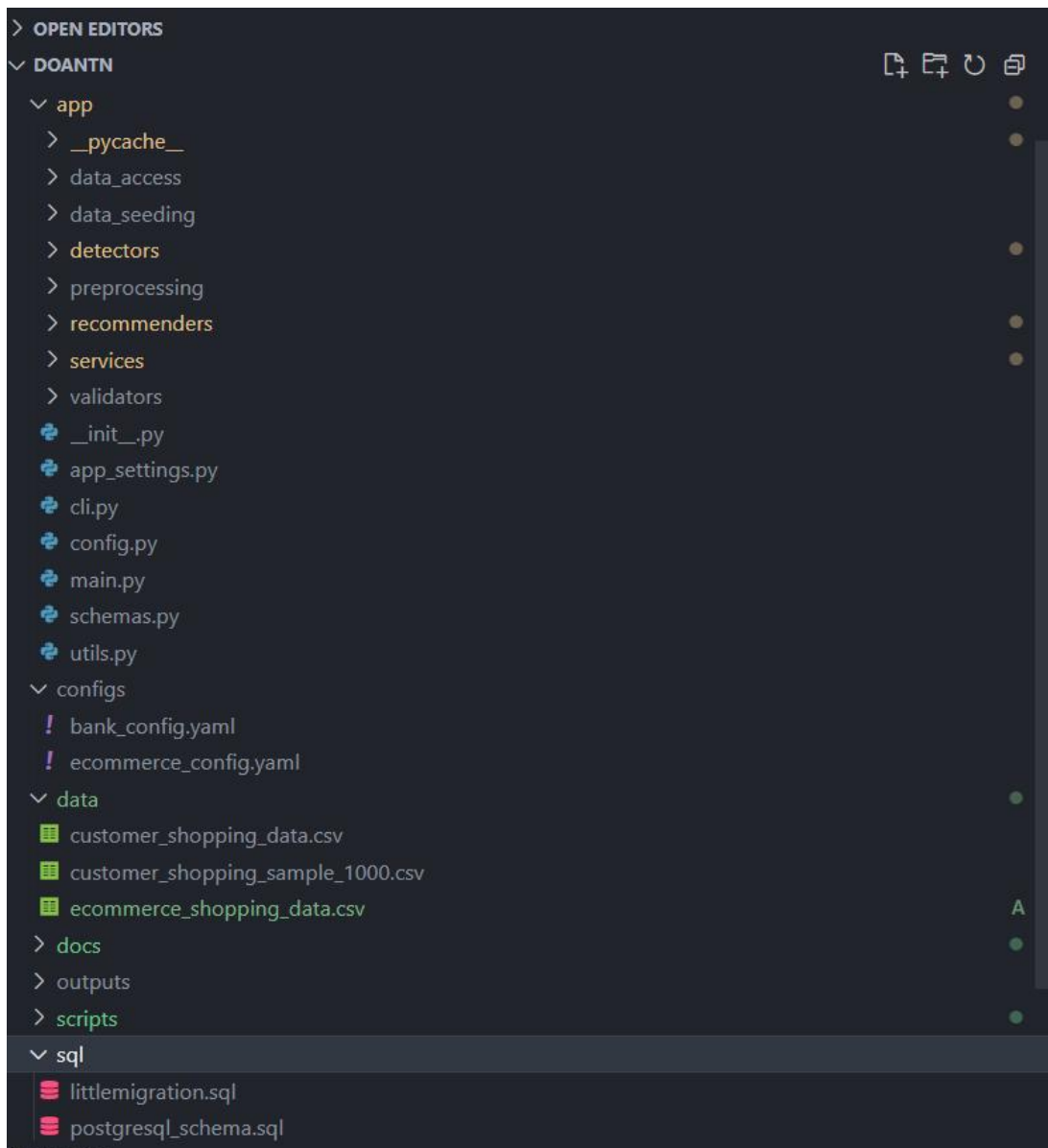
Quá trình này biến dataset ban đầu từ một bảng phẳng thành mô hình quan hệ có cấu trúc. Đây là bước quan trọng để đồ án thể hiện được phần thiết kế cơ sở dữ liệu, không chỉ là xử lý file CSV.

Từ góc nhìn dữ liệu, module import đóng vai trò là cầu nối giữa dataset gốc và database nghiệp vụ. Dataset gốc thường phù hợp cho việc chia sẻ hoặc phân tích nhanh, nhưng chưa chắc phù hợp để lưu trữ trong cơ sở dữ liệu quan hệ. Việc tách bảng giúp giảm lặp dữ liệu và làm rõ quan hệ giữa khách hàng, sản phẩm, địa điểm và giao dịch. Đây cũng là cơ sở để sau này có thể mở rộng thêm các bảng như sản phẩm chi tiết, chương trình khuyến mãi hoặc lịch sử khách hàng.

4.5. Module truy cập dữ liệu

Hệ thống hỗ trợ hai nguồn dữ liệu chính là CSV và PostgreSQL. Với CSV, hệ thống đọc trực tiếp file dữ liệu bằng pandas. Với PostgreSQL, hệ thống sử dụng `source_query` trong file YAML để join các bảng nghiệp vụ và tạo dataframe đầu vào cho pipeline.

Việc hỗ trợ cả CSV và PostgreSQL giúp hệ thống linh hoạt hơn trong quá trình demo và kiểm thử. CSV phù hợp để chạy nhanh end-to-end, còn PostgreSQL phù hợp để trình bày vai trò của cơ sở dữ liệu trong lưu trữ dữ liệu chuẩn và kết quả làm sạch.



Hình 4.6. Cấu trúc thư mục project

Với PostgreSQL, module PostgresDataSource có hai nhiệm vụ: đọc dữ liệu đầu vào và ghi dữ liệu đầu ra. Khi đọc, module sử dụng source_query nếu câu hình có khai báo; nếu không, module đọc trực tiếp từ table_name. Trong dataset e-commerce, source_query join các bảng transactions, customers, products và shopping_malls để tạo dataframe có đầy đủ các cột cần thiết cho pipeline.

Khi ghi dữ liệu, PostgresDataSource xử lý một số vấn đề kỹ thuật quan trọng. Thứ nhất, module ánh xạ tên output cũ sang tên bảng đúng, ví dụ recommendations sang fix_recommendations, fixed_data sang fixed_transactions. Thứ hai, với các bảng đã có schema cố định, module không dùng replace vì thao tác này có thể làm mất ràng buộc khóa chính, khóa ngoại và kiểu dữ liệu. Thay vào đó, module xóa dữ liệu cũ trong bảng đích rồi append dữ liệu mới.

Thứ ba, module sanitize dữ liệu trước khi ghi SQL. Một số kiểu dữ liệu như `numpy.int64`, `numpy.float64`, `numpy.bool_` hoặc `pd.Timestamp` có thể gây lỗi khi ghi trực tiếp vào PostgreSQL. Do đó, hệ thống chuyển các kiểu này về kiểu Python chuẩn như `int`, `float`, `bool` hoặc `datetime`.

4.6. Module tiền xử lý và feature engineering

Module tiền xử lý thực hiện các công việc như chuẩn hóa tên cột, ép kiểu dữ liệu, xử lý ngày tháng và sinh feature. Các feature như `unit_price`, `age_group`, `invoice_year`, `invoice_month`, `day_of_week`, `is_weekend`, `quantity_band` và `price_deviation_from_category` được tạo ra để hỗ trợ phát hiện bất thường.

Đặc biệt, `unit_price` và `price_deviation_from_category` là hai feature có ý nghĩa trong dữ liệu giao dịch. `unit_price` giúp phân tích đơn giá thay vì chỉ nhìn tổng giá. `price_deviation_from_category` giúp đánh giá mức lệch của giao dịch so với giá trị điển hình trong cùng danh mục sản phẩm.

```
if "invoice_date" in out.columns:
    parsed = parse_ecommerce_invoice_date(out["invoice_date"])
    out["invoice_date"] = parsed

if "invoice_year" not in out.columns:
    out["invoice_year"] = parsed.dt.year
if "invoice_month" not in out.columns:
    out["invoice_month"] = parsed.dt.month
if "invoice_day" not in out.columns:
    out["invoice_day"] = parsed.dt.day
if "day_of_week" not in out.columns:
    out["day_of_week"] = parsed.dt.dayofweek
if "is_weekend" not in out.columns:
    out["is_weekend"] = parsed.dt.dayofweek.isin([5, 6]).astype(int)

if "unit_price" not in out.columns and {"price", "quantity"}.issubset(out.columns):
    denom = out["quantity"].replace(0, np.nan)
    out["unit_price"] = out["price"] / denom

if {"category", "unit_price"}.issubset(out.columns):
    category_base_price = (
        out.groupby("category", dropna=False)["unit_price"]
        .median()
        .rename("base_unit_price_by_category")
    )
    out = out.join(category_base_price, on="category")

if "price_deviation_from_category" not in out.columns:
    out["price_deviation_from_category"] = (
        out["unit_price"] - out["base_unit_price_by_category"]
    )
```

Hình 4.7. Ảnh code feature engineering

Module tiền xử lý nằm chủ yếu trong `app/utils.py`. Hàm `prepare_input_dataframe` đóng vai trò cửa ngõ chuẩn hóa dữ liệu trước khi pipeline chính chạy. Hàm này xử lý empty string, xác định loại dataset dựa trên `id_column` và gọi

hàm tạo feature tương ứng. Với e-commerce, `id_column` là `invoice_no`, do đó hệ thống gọi `add_ecommerce_derived_features`.

Trong `add_ecommerce_derived_features`, hệ thống thực hiện nhiều bước:

1. Đổi tên cột từ định dạng gốc sang `snake_case` nếu cần.
2. Chuẩn hóa chuỗi như `gender`, `category`, `payment_method`, `shopping_mall`.
3. Ép kiểu các cột số như `age`, `quantity`, `price`.
4. Parse `invoice_date` và sinh các feature thời gian.
5. Tính `unit_price`.
6. Tính `median unit_price` theo `category` để tạo `base_unit_price_by_category`.
7. Tính `price_deviation_from_category`.
8. Tạo `age_group` và `quantity_band`.

Điểm quan trọng là quá trình feature engineering được thực hiện cả trong script import và trong luồng pipeline từ CSV. Điều này giúp hệ thống có thể xử lý dữ liệu raw CSV mà không bắt buộc người dùng phải import qua PostgreSQL trước.

4.7. Module phát hiện missing data

Missing detector kiểm tra các cột bắt buộc dựa trên cấu hình YAML. Nếu một cột được đánh dấu `required: true` và giá trị tại một bản ghi bị rỗng, hệ thống tạo một issue có loại `missing`. Issue này chứa thông tin dòng, cột, giá trị hiện tại, mức độ nghiêm trọng và hành động khuyến nghị.

Các lỗi missing thường có khả năng giải thích rõ ràng. Ví dụ, nếu `payment_method` bị thiếu, hệ thống có thể ghi nhận rằng giao dịch thiếu phương thức thanh toán. Nếu `quantity` hoặc `price` bị thiếu, lỗi được xem là nghiêm trọng hơn vì ảnh hưởng đến tính toán giao dịch.

Trong phần cài đặt, `MissingDetector` duyệt qua toàn bộ rule trong config. Nếu một cột được đánh dấu `required: true`, detector kiểm tra hai trường hợp: giá trị NaN và chuỗi rỗng sau khi strip khoảng trắng. Việc kiểm tra chuỗi rỗng là cần thiết vì dữ liệu CSV có thể biểu diễn missing value dưới dạng ô trống hoặc chuỗi chỉ chứa khoảng trắng.

Khi phát hiện lỗi, detector tạo một `IssueRecord` với các thông tin:

1. `issue_type = missing`.
2. `confidence = 1.0`, vì missing là lỗi xác định bằng rule.
3. `severity` lấy từ `severity_if_missing` trong cấu hình.
4. `source_method = rule_missing_detector`.

5. `can_auto_fix = True`, vì missing có thể được recommender đề xuất giá trị.

Tuy nhiên, `can_auto_fix = True` không có nghĩa là chắc chắn sẽ sửa. FixService vẫn kiểm tra suggested value và confidence trước khi áp dụng.

4.8. Module phát hiện invalid data

Invalid detector kiểm tra dữ liệu dựa trên miền giá trị và điều kiện ràng buộc. Với dữ liệu categorical, hệ thống kiểm tra giá trị có thuộc danh sách allowed values hay không. Với dữ liệu numeric, hệ thống kiểm tra min/max nếu được cấu hình. Với dữ liệu thời gian, hệ thống kiểm tra khả năng parse và miền giá trị liên quan.

Ví dụ, trường `payment_method` chỉ chấp nhận Cash, Credit Card và Debit Card. Nếu xuất hiện giá trị khác, hệ thống sẽ tạo issue loại invalid. Tương tự, nếu `quantity` nhỏ hơn 1 hoặc `invoice_month` nằm ngoài khoảng 1 đến 12, hệ thống cũng đánh dấu lỗi.

```
gender:
  dtype: categorical
  required: true
  allowed_values:
    - Female
    - Male
  severity_if_missing: medium
  severity_if_invalid: medium

age:
  dtype: numeric
  required: true
  min_value: 0
  max_value: 120
  severity_if_missing: high
  severity_if_invalid: high
```

Hình 4.8. Ảnh cấu hình rule YAML

InvalidDetector xử lý ba nhóm kiểu dữ liệu: numeric, categorical và date.

Với numeric, detector dùng `pd.to_numeric(..., errors="coerce")` để chuyển dữ liệu sang số. Nếu một giá trị không rỗng nhưng không chuyển được sang số, hệ thống đánh dấu invalid type. Sau đó, nếu rule có `min_value` hoặc `max_value`, detector tiếp tục kiểm tra giá trị có vượt miền cho phép hay không.

Với categorical, detector chuẩn hóa chuỗi bằng cách strip khoảng trắng và casefold để so sánh không phân biệt hoa thường. Sau đó, giá trị được kiểm tra với tập `allowed_values`. Ví dụ, `cash` và `Cash` có thể được chuẩn hóa để tránh sai khác không

đáng kể, nhưng một giá trị như Paypal vẫn bị xem là invalid nếu không nằm trong danh sách cho phép.

Với date, detector dùng `pd.to_datetime` để kiểm tra khả năng parse. Nếu giá trị không rỗng nhưng parse thất bại, hệ thống tạo invalid issue. Riêng với e-commerce, việc parse ngày còn được xử lý kỹ hơn ở bước preprocessing bằng hàm `parse_ecommerce_invoice_date`.

Ngoài InvalidDetector, hệ thống còn có CrossFieldValidator. Module này kiểm tra các quan hệ liên trường như:

1. price, quantity, unit_price.
2. age và age_group.
3. invoice_date với invoice_year, invoice_month, invoice_day, day_of_week, is_weekend.

Nhờ đó, hệ thống phát hiện được những lỗi không thể thấy nếu chỉ kiểm tra từng cột độc lập.

4.9. Module phát hiện anomaly bằng Isolation Forest

Module AnomalyDetector hiện thực hóa Isolation Forest như lớp phát hiện bất thường ở cấp bản ghi. Dữ liệu dùng để fit không được lấy trực tiếp từ toàn bộ dữ liệu thô, mà đi qua bước chọn cột theo cấu hình, tiền xử lý và xây dựng clean reference set. Cách tổ chức này giúp mô hình học chủ yếu từ các bản ghi đã vượt qua các kiểm tra lỗi rõ ràng, qua đó giảm nguy cơ để missing hoặc invalid value chi phối phân phối điểm số.

4.9.1. Chuẩn bị ma trận đặc trưng

FeatureBuilder duyệt các rule trong cấu hình và chỉ lựa chọn những cột có `use_for_detection=true`, đồng thời loại id_column khỏi tập đặc trưng. Các cột được chia thành hai nhóm theo dtype. Với nhóm numeric, pipeline thực hiện median imputation rồi StandardScaler. Với nhóm categorical, pipeline thực hiện most-frequent imputation rồi OneHotEncoder với `handle_unknown="ignore"`. Hai nhánh được ghép bằng ColumnTransformer để tạo ma trận đầu vào cho scikit-learn.

Numeric: median imputation → StandardScaler

Categorical: most-frequent imputation → OneHotEncoder

Kết quả: ColumnTransformer → ma trận đặc trưng cho Isolation Forest

Cách xử lý `handle_unknown="ignore"` giúp mô hình vẫn biến đổi được bản ghi có category chưa xuất hiện khi fit. Tuy nhiên, giá trị mới sẽ tạo một vector không có cột

one-hot tương ứng, nên kết quả cần được diễn giải thận trọng. Ngoài ra, StandardScaler được giữ để thống nhất pipeline; đây không phải yêu cầu bắt buộc về mặt toán học đối với Isolation Forest.

4.9.2. Khởi tạo và huấn luyện mô hình

Hàm `_build_model` khởi tạo `IsolationForest` với `n_estimators=200`, `contamination=0.08` và `random_state=42` theo cấu hình hiện tại. Số cây 200 được lựa chọn nhằm tăng độ ổn định của điểm số so với một ensemble quá nhỏ. `random_state=42` giúp tái lập kết quả trong cùng điều kiện dữ liệu và phiên bản thư viện; giá trị này không được xem là tham số làm tăng độ chính xác.

Mã nguồn đặt `max_samples=n_rows`, nghĩa là mỗi cây nhận toàn bộ số dòng của tập fit tương ứng thay vì giá trị mặc định nhỏ hơn của scikit-learn. Lựa chọn này bảo đảm mỗi cây có thể quan sát đầy đủ tập tham chiếu trong phiên bản triển khai hiện tại. Tuy nhiên, xét theo lý thuyết Isolation Forest, đây là một đánh đổi chứ không phải cấu hình luôn tốt hơn: sử dụng toàn bộ dữ liệu làm cây sâu hơn, tăng chi phí và làm giảm tính đa dạng do subsampling; trong một số phân phối còn có thể tăng masking. Vì vậy, báo cáo ghi nhận đây là quyết định cài đặt cần được đánh giá bằng thực nghiệm độ nhạy.

`contamination=0.08` được sử dụng để xác định `offset_` và ngưỡng nhận. Tham số này thể hiện tỷ lệ cảnh báo giả định khoảng 8% trong từng mô hình, không có nghĩa mô hình đã biết trước 8% dữ liệu là sai. Khi áp dụng vào dữ liệu mới, số lượng anomaly thực tế còn phụ thuộc phân phối điểm số, các giá trị trùng tại ngưỡng và việc bản ghi được đánh giá bởi mô hình global hay mô hình segment.

```

def _build_model(self, n_rows: int) -> IsolationForest:
    return IsolationForest(
        n_estimators=self.config.n_estimators,
        contamination=self.config.contamination,
        # Override sklearn's 256-row default so every tree uses the full fit set.
        max_samples=n_rows,
        random_state=self.config.random_state,
    )

def _fit_one_model(self, df: pd.DataFrame) -> tuple[FeatureBuilder, IsolationForest]:
    feature_builder = FeatureBuilder(self.config)
    feature_builder.fit(df)
    X = feature_builder.transform(df)

    model = self._build_model(len(df))
    model.fit(X)

    return feature_builder, model

```

Hình 4.9. Ảnh code Isolation Forest

4.9.3. Mô hình global và các mô hình theo category

Phương thức fit trước hết gọi `_fit_one_model` trên toàn bộ clean reference set để tạo FeatureBuilder và mô hình global. Sau đó, nếu `anomaly_segment_column` tồn tại, dữ liệu được nhóm theo category. Mỗi nhóm có ít nhất 500 bản ghi được fit một FeatureBuilder và một IsolationForest riêng. Việc fit bộ tiền xử lý riêng cho từng nhóm giúp median, mode và không gian one-hot phản ánh đúng phân phối nội bộ của category đó.

Khi chấm điểm, các bản ghi thuộc category đã có segment model được xử lý trước bởi mô hình tương ứng. Các bản ghi còn lại, bao gồm nhóm nhỏ hoặc category chưa có khi fit, được chuyển cho mô hình global. Cơ chế fallback bảo đảm mọi dòng đều nhận được kết quả mà không buộc phải huấn luyện mô hình thiếu ổn định trên nhóm quá nhỏ.

4.9.4. Sinh và lưu kết quả phát hiện

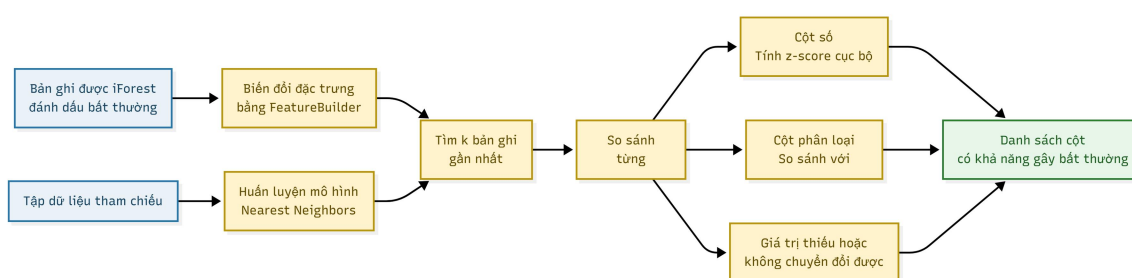
Với mỗi bản ghi, module lưu `raw_score` từ `score_samples`, `decision_score` từ `decision_function`, `is_anomaly` và `anomaly_model_segment`. Theo quy ước của scikit-learn, `raw_score` càng thấp thì quan sát càng bất thường; `decision_score` âm nằm phía outlier của ngưỡng quyết định. `is_anomaly` được gán bằng 1 khi `predict` trả về -1. Việc lưu đồng thời điểm số và nhãn giúp hệ thống vừa lọc các bản ghi đã vượt ngưỡng, vừa xếp hạng mức độ ưu tiên trong cùng một mô hình [1].

Kết quả của AnomalyDetector chưa được dùng để sửa dữ liệu trực tiếp. Các dòng có `is_anomaly=1` được chuyển sang bước phân tích feature contribution để xác định cột nghi ngờ, sau đó mới tạo `detected_issues` và `recommendation`. Ranh giới trách nhiệm này bảo đảm mô hình phát hiện không tự suy diễn nguyên nhân hoặc tự thay đổi giá trị dữ liệu.

4.10. Module feature contribution

Sau khi một bản ghi bị đánh dấu anomaly, hệ thống cần đưa ra thông tin giải thích cột nào có thể đóng góp vào bất thường đó. Module feature contribution phân tích độ lệch của các feature so với dữ liệu tham chiếu, từ đó xác định các cột nghi ngờ.

Kết quả của module này giúp issue không chỉ dừng ở mức “bản ghi bất thường”, mà còn cho biết bất thường có thể liên quan đến `shopping_mall`, `payment_method`, `category`, `age_group`, `quantity_band` hoặc các feature số khác. Điều này làm tăng tính giải thích của hệ thống.



Hình 4.10. Sơ đồ giải thích anomaly

Isolation Forest chỉ cho biết một dòng có bất thường hay không, nhưng không trực tiếp chỉ ra cột nào gây bất thường. Vì vậy, hệ thống bổ sung FeatureContributionAnalyzer. Module này fit một mô hình nearest neighbors trên clean reference set, sau đó với mỗi anomaly row sẽ tìm các dòng bình thường gần nhất.

Sau khi có nhóm láng giềng, hệ thống kiểm tra từng candidate column. Với cột numeric, hệ thống tính trung bình và độ lệch chuẩn của giá trị cột đó trong nhóm láng giềng. Nếu giá trị hiện tại có z-score lớn hơn hoặc bằng `feature_z_threshold`, cột được xem là nghi ngờ. Với cột categorical, hệ thống lấy mode của nhóm láng giềng; nếu giá trị hiện tại khác mode, cột được xem là nghi ngờ.

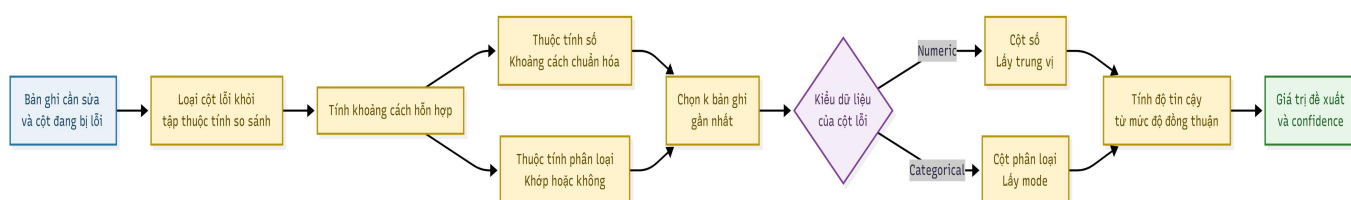
Cách làm này không phải là giải thích mô hình theo nghĩa đầy đủ như SHAP, nhưng phù hợp với phạm vi đề án vì dễ hiểu, dễ cài đặt và gắn trực tiếp với dữ liệu tham

chiều. Khi trình bày kết quả, người dùng có thể hiểu rằng cột bị nghi ngờ vì nó khác đáng kể so với các bản ghi bình thường tương tự.

4.11. Module gợi ý sửa bằng nearest neighbors

Module recommender sử dụng nearest neighbors để tìm các bản ghi tương tự trong tập tham chiếu. Khi một bản ghi bị lỗi ở một cột categorical/text, hệ thống có thể loại cột đó khỏi quá trình so sánh, tìm những bản ghi gần nhất theo các cột còn lại và đề xuất giá trị phổ biến hoặc phù hợp từ nhóm láng giềng.

Trong quá trình tối ưu, hệ thống đã bổ sung cơ chế cache dữ liệu numeric/categorical đã chuẩn hóa và cache neighbor theo row_id cùng danh sách cột exclude. Điều này giúp tránh tính toán lặp lại khi một giao dịch có nhiều issue.



Hình 4.11. Luồng gợi ý sửa lỗi bằng nearest neighbors

Hệ thống có ba recommender chính: MissingRecommender, InvalidRecommender và AnomalyRecommender.

MissingRecommender dùng MixedTypeNearestNeighbors để tìm các dòng tương tự, loại bỏ cột đang thiếu khỏi quá trình tính khoảng cách. Nếu cột thiếu là numeric, hệ thống lấy median của giá trị cột đó trong các láng giềng. Median được chọn thay vì mean vì ít bị ảnh hưởng bởi giá trị cực đoan. Nếu cột thiếu là categorical, hệ thống lấy mode của các láng giềng.

InvalidRecommender xử lý theo rule. Nếu cột numeric nhỏ hơn min_value, gợi ý là min_value. Nếu lớn hơn max_value, gợi ý là max_value. Nếu cột categorical có allowed_values, hệ thống dùng difflib.get_close_matches để tìm giá trị hợp lệ gần nhất. Nếu không tìm được giá trị gần đúng, hệ thống trả về giá trị đầu tiên trong allowed values với confidence thấp hơn.

AnomalyRecommender cũng dùng nearest neighbors, nhưng có nguyên tắc thận trọng hơn. Với anomaly numeric, hệ thống không tự đề xuất giá trị thay thế vì giá trị số

bất thường có thể là giao dịch thật. Với anomaly categorical, hệ thống có thể đề xuất mode từ các láng giềng tương tự.

MixedTypeNearestNeighbors là thành phần dùng chung. Module này chuẩn hóa khoảng cách numeric theo range của từng cột và tính khoảng cách categorical theo match/mismatch. Ngoài ra, module có cache neighbor theo row_id và danh sách cột exclude để tránh tính lại nhiều lần khi một dòng có nhiều issue.

4.12. Module report và output

Sau khi chạy pipeline, hệ thống sinh ra các output chính:

Bảng 4.4. Các tệp đầu ra của hệ thống

Output	Ý nghĩa
detected_issues.csv	Danh sách lỗi và bất thường phát hiện được
fix_recommendations.csv	Gợi ý sửa lỗi
dataset_profile.csv	Thống kê chất lượng dữ liệu theo cột
fixed_transactions.csv	Dữ liệu sau khi áp dụng sửa lỗi phù hợp
final_report.csv	Báo cáo tổng hợp

Khi thực hiện bằng PostgreSQL, detected_issues, fix_recommendations, dataset_profile và fixed_transactions được ánh xạ đến các bảng có cấu trúc cố định trong schema. Trong quá trình triển khai, hệ thống đã xử lý mapping tên output cũ để không ghi nhầm vào recommendations hoặc fixed_data. Riêng final_report chưa có định nghĩa trong postgresql_schema.sql; output này tồn tại ở dạng CSV, còn việc lưu thành bảng PostgreSQL hiện đi qua nhánh ghi động của PostgresDataSource và cần được xác nhận như đã ghi chú ở Mục 4.3.

dataset_profile.csv	✓	Comma Separated Values Source File	1 KB
detected_issues.csv	✓	Comma Separated Values Source File	1,937 KB
final_report.csv	✓	Comma Separated Values Source File	1 KB
fix_recommendations.csv	✓	Comma Separated Values Source File	1,361 KB
fixed_transactions.csv	✓	Comma Separated Values Source File	12,351 KB

Hình 4.12. Ảnh output sau khi chạy pipeline

ReportService gồm ba chức năng chính.

Chức năng thứ nhất là `build_summary`, dùng để tạo `final_report`. Báo cáo này gồm tổng số issue, số issue theo loại, số issue theo severity, confidence trung bình và fixable rate. Đây là output ngắn gọn nhất để trình bày kết quả tổng quan sau mỗi lần chạy.

Chức năng thứ hai là `build_recommendations`, dùng để tạo bảng gợi ý sửa lỗi. Bảng này lấy các cột quan trọng từ issue như `issue_id`, `row_id`, `column_name`, `issue_type`, `current_value`, `suggested_value`, `confidence`, `severity`, `reason` và `can_auto_fix`.

Chức năng thứ ba là `build_dataset_profile`, dùng để thống kê chất lượng dữ liệu theo cột. Với mỗi cột, hệ thống tính `missing_count`, `missing_rate`, `unique_count`, `min`, `max`, `issue_count`, `anomaly_count` và `invalid_count`. Bảng này rất phù hợp để đưa vào dashboard hoặc báo cáo chất lượng dữ liệu.

Khi chạy từ CSV, các output được ghi ra file `.csv`. Khi chạy từ PostgreSQL, các output có schema cố định được ghi vào bảng tương ứng, còn output không có trong schema cần được xem xét riêng để tránh tạo bảng động ngoài thiết kế. Việc giữ cấu trúc dữ liệu logic nhất quán giữa hai chế độ giúp pipeline dễ kiểm tra, nhưng tăng lưu trữ vẫn phải tuân theo các bảng thực sự được định nghĩa trong SQL.

4.13. Giao diện dòng lệnh CLI

Hệ thống được chạy thông qua CLI. Ví dụ chạy pipeline từ CSV:

```
python -m app.cli `
--source csv `
--input data\customer_shopping_data.csv `
--config configs\ecommerce_config.yaml `
--output outputs\teacher_csv_run `
--report `
--profile-runtime `
--apply-fixes
```

Ví dụ chạy pipeline từ PostgreSQL:

```
python -m app.cli `
--source postgres `
--connection-uri "postgres://postgres:password@localhost:5432/doantn" `
--config configs\ecommerce_config.yaml `
--output outputs\teacher_pg_run `
--report `
--profile-runtime `
--apply-fixes
```

CLI giúp việc demo hệ thống rõ ràng hơn vì có thể chạy từng chế độ, chỉ định nguồn dữ liệu, output folder, bật report, bật apply fixes và bật profile runtime.


```

=== REPORT ===
metric      value
0    total_issues 1965.00
1    issue_type_anomaly 1964.00
2    issue_type_invalid 1.00
3    severity_high 1955.00
4    severity_low 6.00
5    severity_medium 4.00
6    average_confidence 0.06
7    fixable_rate 1.00

=== DATASET PROFILE ===
column_name  missing_count  missing_rate  ...  issue_count  anomaly_count  invalid_count
0    invoice_no 0 0.0 ... 0 0 0
1    customer_id 0 0.0 ... 0 0 0
2    gender 0 0.0 ... 33 33 0
3    age 0 0.0 ... 0 0 0
4    category 0 0.0 ... 158 158 0
5    quantity 0 0.0 ... 0 0 0
6    price 0 0.0 ... 0 0 0
7    payment_method 0 0.0 ... 247 247 0
8    invoice_date 0 0.0 ... 0 0 0
9    shopping_mall 0 0.0 ... 1515 1514 1
10    invoice_year 0 0.0 ... 0 0 0
11    invoice_month 0 0.0 ... 0 0 0
12    invoice_day 0 0.0 ... 0 0 0
13    day_of_week 0 0.0 ... 0 0 0
14    is_weekend 0 0.0 ... 0 0 0
15    unit_price 0 0.0 ... 0 0 0
16    base_unit_price_by_category 0 0.0 ... 0 0 0
17    price_deviation_from_category 0 0.0 ... 0 0 0
18    age_group 0 0.0 ... 11 11 0
19    quantity_band 0 0.0 ... 1 1 0

[20 rows x 9 columns]

```

(a) Kết quả báo cáo và hồ sơ dữ liệu

```

=== SAMPLE ISSUES ===
issue_id      row_id  ...  can_auto_fix  created_at
0    19c49ea6-da24-4a24-a626-66c81dec80ff I196704 ... True 2026-07-25T19:03:59.797848
1    d38829b4-6e9f-4517-aa4e-aa4fec8e07a6 I317333 ... True 2026-07-25T19:04:29.820688
2    9d9bf0ad-64e0-472a-85f9-13f834f5f275 I317333 ... True 2026-07-25T19:04:29.820688
3    b18ea74c-c9d5-43d9-a243-88272c2354c2 I688768 ... True 2026-07-25T19:04:29.842293
4    21aa8ab2-4998-4d50-be73-2f5c017f0d7d I412481 ... True 2026-07-25T19:04:29.881961
5    8722481b-bdb3-4732-8830-7483a54e7bd2 I170160 ... True 2026-07-25T19:04:30.467015
6    ef3edba1-b52d-4ae1-80f0-36adefbf4234 I402966 ... True 2026-07-25T19:04:30.499088
7    5fa6c3b0-25df-4526-8494-349b4b659780 I291540 ... True 2026-07-25T19:04:30.555824
8    cb026a10-194b-40b2-8327-adb6f2033f57 I941223 ... True 2026-07-25T19:04:30.796003
9    5ccca36a-f2fd-40bd-95d6-056bc14d301d I236520 ... True 2026-07-25T19:04:30.816345
10   63c87494-8a9a-49fb-8375-c95a0d3a5bee I283774 ... True 2026-07-25T19:04:30.889463
11   8c41563e-4264-4c78-af6a-88e64ab4b757 I251779 ... True 2026-07-25T19:04:31.039787
12   e4103752-71f6-47cc-a765-baa4b9e300c6 I251307 ... True 2026-07-25T19:04:31.113644
13   0d8f0886-bd76-4848-9d2c-26d4a98dffcd I188382 ... True 2026-07-25T19:04:31.339985
14   b3ff260e-154b-44b1-9aea-cfa01911ab9b I713320 ... True 2026-07-25T19:04:31.577943
15   61bc2262-f76b-4ab1-8fe4-b01f3bfc1e57 I125636 ... True 2026-07-25T19:04:31.632226
16   dab3800d-9c51-40a0-bd6a-61a47a3e0041 I846448 ... True 2026-07-25T19:04:31.666662
17   93d5a21d-0433-4fc0-8cfe-2e847de69b83 I255891 ... True 2026-07-25T19:04:31.989951
18   cd02c2be-cd59-453a-b7fa-8a83256764b7 I420393 ... True 2026-07-25T19:04:32.056651
19   caa81b05-f2da-41fe-bd5b-bffc8fb888a1 I912553 ... True 2026-07-25T19:04:32.153619

[20 rows x 13 columns]
=== PIPELINE COMPLETED ===
Total issues: 1965
Output directory: outputs\Danh_gia_hieu_nang

=== RUNTIME PROFILE ===
load_config: 0.01s
read_source: 0.18s
prepare_input: 0.23s
validate_columns: 0.00s
run_pipeline: 215.78s
build_reports: 0.09s
write_outputs: 0.04s
apply_fixes: 0.04s
write_fixed_output: 0.44s

```

(b) Kết quả hoàn tất pipeline và runtime profile

Hình 4.13. Kết quả thực thi full pipeline qua giao diện dòng lệnh

CLI được thiết kế để hỗ trợ nhiều chế độ chạy trong cùng một entry point. Các tham số quan trọng gồm:

Bảng 4.5. Các tham số CLI chính

Tham số	Ý nghĩa
--source	Chọn nguồn dữ liệu: csv hoặc postgres
--input	Đường dẫn file CSV khi chạy từ CSV
--connection-uri	Chuỗi kết nối PostgreSQL
--config	Đường dẫn file YAML
--output	Thư mục output
--report	In báo cáo ra terminal
--profile-runtime	In thời gian chạy từng stage
--apply-fixes	Bật bước áp dụng sửa lỗi
--mode	Chế độ fix: conservative, auto hoặc interactive
--seed-dirty	Tạo dữ liệu lỗi để kiểm thử
--seed-rate	Tỷ lệ dữ liệu lỗi khi seed

Luồng chính trong CLI gồm: load config, đọc source, prepare input, validate columns, run pipeline, build reports, write outputs, apply fixes và write fixed output. Khi bật --profile-runtime, CLI ghi nhận thời gian của từng stage bằng perf_counter. Đây là cơ sở để xác định bottleneck trong Chương 5.

Chức năng --seed-dirty giúp tạo dữ liệu lỗi từ dữ liệu sạch. Khi bật tham số này, CLI tạo seeded_dirty_data.csv và seeded_dirty_labels.csv, sau đó dừng. Người dùng có thể dùng file dirty vừa tạo để chạy pipeline và kiểm tra detector.

4.14. Một số vấn đề kỹ thuật đã xử lý

Trong quá trình triển khai, hệ thống đã xử lý một số vấn đề kỹ thuật đáng chú ý:

1. Parse sai invoice_date do định dạng ngày cần dayfirst=True.
2. Ghi PostgreSQL bằng replace có thể làm mất schema và khóa ngoại, nên được điều chỉnh để clear dữ liệu và append vào bảng cố định.
3. Một số kiểu dữ liệu NumPy scalar không ghi trực tiếp được vào PostgreSQL, nên cần sanitize trước khi ghi.
4. Tên bảng output trong code và schema từng bị lệch, đã được map lại đúng thành fix_recommendations và fixed_transactions.

-
5. Trường `source_method` trong `detected_issues` cần tăng độ dài để lưu tên phương pháp anomaly detection.
 6. Full pipeline có bottleneck ở phần `run_pipeline`, nên hệ thống đã tối ưu lookup và cache trong các module xử lý.

Ngoài các vấn đề trên, một điểm kỹ thuật đáng chú ý là cách hệ thống xây dựng `row_lookup` trong pipeline. Ban đầu, nếu mỗi issue cần tìm dòng tương ứng bằng cách filter dataframe, thời gian xử lý có thể tăng đáng kể khi số issue lớn. Hệ thống đã tạo lookup theo `id_column` một lần bằng `groupby`, sau đó truy cập trực tiếp theo `row_id`. Đây là tối ưu nhỏ nhưng có ý nghĩa trong full pipeline.

Một vấn đề khác là tránh tính lại nearest neighbors nhiều lần. Khi một giao dịch có nhiều issue, recommender có thể cần tìm cùng một nhóm láng giềng cho cùng `row_id` nhưng loại trừ các cột khác nhau. `MixedTypeNearestNeighbors` đã bổ sung cache theo khóa gồm `row_id` và danh sách `exclude columns`. Điều này giúp giảm lặp tính toán trong quá trình recommendation.

Về ghi PostgreSQL, hệ thống phải đảm bảo thứ tự xóa dữ liệu giữa các bảng có khóa ngoại. Ví dụ, trước khi ghi lại `detected_issues`, cần xóa `cleaning_actions` và `fix_recommendations` liên quan để tránh vi phạm ràng buộc khóa ngoại. Module `PostgresDataSource` định nghĩa `DEPENDENT_CLEAR_ORDER` để xử lý vấn đề này.

4.15. Luồng vận hành end-to-end của hệ thống

Sau khi các module được cài đặt, luồng vận hành end-to-end của hệ thống có thể mô tả theo một kịch bản cụ thể. Người dùng chuẩn bị dữ liệu đầu vào và file cấu hình. Nếu chạy từ CSV, người dùng truyền đường dẫn file CSV vào tham số `--input`. Nếu chạy từ PostgreSQL, người dùng truyền connection URI và hệ thống sử dụng `source_query` để đọc dữ liệu. Sau đó CLI load config, đọc source, chuẩn hóa dataframe và kiểm tra cột bắt buộc.

Khi dữ liệu đã sẵn sàng, `DataCleaningPipelineService` được khởi tạo. Service này lần lượt gọi `MissingDetector`, `InvalidDetector` và `CrossFieldValidator` để phát hiện các lỗi rõ ràng. Các dòng có lỗi rõ ràng được loại khỏi clean reference set. Sau đó `AnomalyDetector` fit mô hình trên clean reference set và predict anomaly trên toàn bộ dữ liệu. Với các dòng bị đánh dấu anomaly, `FeatureContributionAnalyzer` tìm cột nghi ngờ và tạo issue tương ứng.

Khi toàn bộ issue đã được tạo, các recommender được fit trên clean reference set. Mỗi issue được bổ sung suggested value và confidence nếu có thể. Sau đó ReportService tạo final report, recommendation table và dataset profile. Nếu người dùng bật --apply-fixes, FixService áp dụng các gợi ý đủ điều kiện và tạo fixed dataframe. Cuối cùng, hệ thống ghi tất cả output ra CSV hoặc PostgreSQL.

Luồng này cho thấy hệ thống đã được tổ chức thành một pipeline hoàn chỉnh, không chỉ là các hàm xử lý rời rạc. Mỗi module có vai trò cụ thể, nhưng kết quả cuối cùng chỉ hình thành khi các module phối hợp theo đúng thứ tự. Đây là điểm quan trọng để chứng minh hệ thống có thể vận hành end-to-end trong thực nghiệm ở Chương 5.

4.16. Kết luận chương

Chương 4 đã trình bày quá trình cài đặt hệ thống và các module chính. Hệ thống được tổ chức theo cấu trúc rõ ràng, mỗi nhóm module đảm nhiệm một phần trong pipeline data cleaning. Các module import, preprocessing, validation, anomaly detection, recommendation, report và fixing được triển khai theo đúng thiết kế ở Chương 3. Việc hỗ trợ cả CSV và PostgreSQL giúp hệ thống linh hoạt trong thực nghiệm, còn các bảng output giúp kết quả có thể kiểm tra và trình bày trong báo cáo.

Điểm quan trọng của chương này là làm rõ sự tương ứng giữa thiết kế và mã nguồn. Những khái niệm như clean reference set, issue schema, feature contribution, confidence, conservative fixing và runtime profiling không chỉ được nêu ở mức ý tưởng, mà đã được hiện thực hóa thành các module cụ thể. Chương tiếp theo sẽ sử dụng chính hệ thống đã cài đặt này để chạy thực nghiệm, ghi nhận kết quả và đánh giá mức độ đáp ứng mục tiêu đề tài.

CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương 5 trình bày quá trình thực nghiệm hệ thống trên dữ liệu mẫu và dữ liệu đầy đủ. Nội dung bao gồm mục tiêu thực nghiệm, môi trường thực nghiệm, kịch bản chạy, kết quả phát hiện lỗi, kết quả gợi ý sửa, đánh giá hiệu năng, nhận xét và hạn chế của hệ thống.

Chương này là bước kiểm chứng cho các nội dung đã trình bày ở các chương trước. Nếu Chương 3 trả lời hệ thống được thiết kế như thế nào và Chương 4 trả lời hệ thống được cài đặt ra sao, thì Chương 5 trả lời hệ thống hoạt động như thế nào khi chạy trên dữ liệu thực nghiệm. Các kết quả trong chương không chỉ dùng để chứng minh pipeline chạy được, mà còn để đánh giá xem hướng kết hợp rule-based validation và machine learning có phù hợp với bài toán làm sạch dữ liệu hay không.

5.1. Mục tiêu thực nghiệm

Mục tiêu của thực nghiệm là kiểm tra hệ thống có thể chạy end-to-end và sinh đúng các output cần thiết hay không. Cụ thể, thực nghiệm cần xác nhận các điểm sau:

1. Hệ thống đọc được dữ liệu từ CSV và PostgreSQL.
2. Hệ thống phát hiện được các lỗi missing, invalid và anomaly.
3. Hệ thống sinh được gợi ý sửa lỗi.
4. Hệ thống tạo được dữ liệu sau khi apply fixes.
5. Hệ thống ghi được các output cần thiết.
6. Hệ thống chạy được trên dữ liệu đầy đủ gần 100.000 dòng.
7. Kết quả có thể dùng để trình bày và đánh giá trong báo cáo.

Ngoài các mục tiêu trên, thực nghiệm còn nhằm kiểm tra tính phù hợp của thiết kế hybrid. Cụ thể, hệ thống cần chứng minh rằng rule-based validation phát hiện tốt các lỗi rõ ràng, còn Isolation Forest bổ sung khả năng phát hiện các điểm bất thường mà rule không mô tả trực tiếp. Nếu kết quả chỉ có missing và invalid, phần AI sẽ chưa thể hiện rõ vai trò. Ngược lại, nếu kết quả chỉ có anomaly mà không có rule, hệ thống sẽ thiếu tính chắc chắn và khó giải thích. Vì vậy, thực nghiệm cần quan sát cả ba nhóm lỗi.

Thực nghiệm cũng nhằm kiểm tra output có đủ thông tin cho người dùng hay không. Một hệ thống data cleaning không chỉ cần phát hiện lỗi, mà còn cần trả lời các câu hỏi: lỗi nằm ở đâu, vì sao bị xem là lỗi, hệ thống đề xuất sửa gì, mức độ tin cậy bao nhiêu, có được phép auto-fix hay không và dữ liệu sau fix thay đổi thế nào.

5.2. Môi trường thực nghiệm

Thực nghiệm được thực hiện trên môi trường Windows, sử dụng Python 3.12 và PostgreSQL cho phần chạy với cơ sở dữ liệu. Các thư viện chính gồm pandas, NumPy, scikit-learn, SQLAlchemy và psycopg2. Hệ thống được chạy thông qua PowerShell bằng lệnh `python -m app.cli`.

Các output hiện có ghi nhận hệ điều hành và phiên bản Python nhưng chưa lưu đầy đủ thông số phần cứng của máy thực nghiệm. Vì runtime phụ thuộc trực tiếp vào cấu hình máy, các trường chưa có bằng chứng được giữ ở trạng thái “chưa cung cấp” và phải được cập nhật từ máy chạy cuối cùng trước khi sử dụng số liệu thời gian như một kết quả có khả năng tái lập.

Các thông tin môi trường được tổng hợp như sau:

Bảng 5.1. Môi trường thực nghiệm

Thành phần	Giá trị
Hệ điều hành	Windows
CPU	Intel Core i7-12700H
RAM	8GB DDR4
Python	Python 3.12
PostgreSQL	18
Thư viện chính	pandas, NumPy, scikit-learn, SQLAlchemy

Việc ghi rõ môi trường là cần thiết vì runtime của pipeline phụ thuộc vào cấu hình máy. Cùng một dataset và cùng một thuật toán, thời gian chạy có thể khác nhau nếu CPU, RAM hoặc ổ đĩa khác nhau. Do đó, kết quả 216,81 giây trong báo cáo nên được hiểu là kết quả trên môi trường thử nghiệm cụ thể, không phải cam kết hiệu năng tuyệt đối cho mọi môi trường.

5.3. Dataset thực nghiệm

Hệ thống được kiểm tra trên hai nhóm dữ liệu:

8. Dữ liệu sample dirty: dữ liệu có lỗi được tạo/chỉnh để kiểm tra khả năng bắt lỗi của pipeline.
9. Dữ liệu full CSV: dữ liệu đầy đủ gồm 99.457 dòng, dùng để kiểm tra khả năng chạy trên tập dữ liệu lớn hơn.

Với dữ liệu đầy đủ, hệ thống đã xác nhận số lượng bảng nghiệp vụ sau transform như sau:

Bảng 5.2. Quy mô các bảng dữ liệu thực nghiệm

Bảng	Số dòng
customers	99.457
products	8
shopping_malls	11
transactions	99.457

```
1  SELECT table_name, row_count
2  FROM (
3      SELECT 1 AS order_no, 'customers' AS table_name,
4             COUNT(*) AS row_count FROM public.customers
5      UNION ALL
6      SELECT 2, 'products',
7             COUNT(*) FROM public.products
8      UNION ALL
9      SELECT 3, 'shopping_malls',
10             COUNT(*) FROM public.shopping_malls
11     UNION ALL
12     SELECT 4, 'transactions',
13            COUNT(*) FROM public.transactions
14 ) AS dataset_statistics
15 ORDER BY order_no;
```

Data Output Messages Notifications

	table_name text	row_count bigint
1	customers	99457
2	products	8
3	shopping_malls	11
4	transactions	99457

Hình 5.1. Thống kê dataset sau import

Dataset full được dùng để kiểm tra khả năng vận hành của pipeline trên dữ liệu có kích thước tương đối lớn trong phạm vi đề án. Dataset sample dirty được dùng để kiểm tra khả năng bắt lỗi khi dữ liệu có lỗi được tạo có chủ đích. Hai nhóm dữ liệu này bổ sung

cho nhau. Full dataset giúp đánh giá hiệu năng và anomaly trên dữ liệu gần sạch; dirty sample giúp đánh giá missing/invalid và khả năng recommendation.

Khi sử dụng dữ liệu đã inject lỗi, cần phân biệt giữa lỗi được seed tự động, lỗi được thêm thủ công và anomaly phát sinh do mô hình. Labels chỉ phản ánh một phần sự thật nếu chúng không ghi nhận toàn bộ lỗi có trong file dirty. Đây là lý do báo cáo trình bày rõ hạn chế của ground truth hiện tại.

5.3.1. Phương pháp tạo dữ liệu lỗi

Hệ thống hỗ trợ tạo dữ liệu lỗi bằng tùy chọn `--seed-dirty`. Khi bật tùy chọn này, pipeline tạo ra hai file:

1. `seeded_dirty_data.csv`: dữ liệu đã được thêm lỗi.
2. `seeded_dirty_labels.csv`: nhãn mô tả các lỗi được seed tự động.

Các lỗi có thể bao gồm missing value, invalid value và anomaly value. Mục đích của bước này là tạo kịch bản kiểm thử có kiểm soát. Trong thực tế, nếu chỉ dùng dữ liệu gốc, rất khó biết chắc dữ liệu nào là lỗi. Khi chủ động inject lỗi, người thực hiện có thể đánh giá detector trên những lỗi đã biết.

Tuy nhiên, việc inject lỗi cũng có hạn chế. Lỗi được tạo nhân tạo có thể chưa phản ánh đầy đủ lỗi thực tế trong hệ thống vận hành. Ví dụ, lỗi thực tế có thể đến từ tích hợp nhiều nguồn dữ liệu, trùng khóa, sai encoding, lỗi timezone hoặc thay đổi schema. Vì vậy, kết quả trên dirty sample chỉ là một phần trong đánh giá hệ thống.

5.4. Kịch bản thực nghiệm

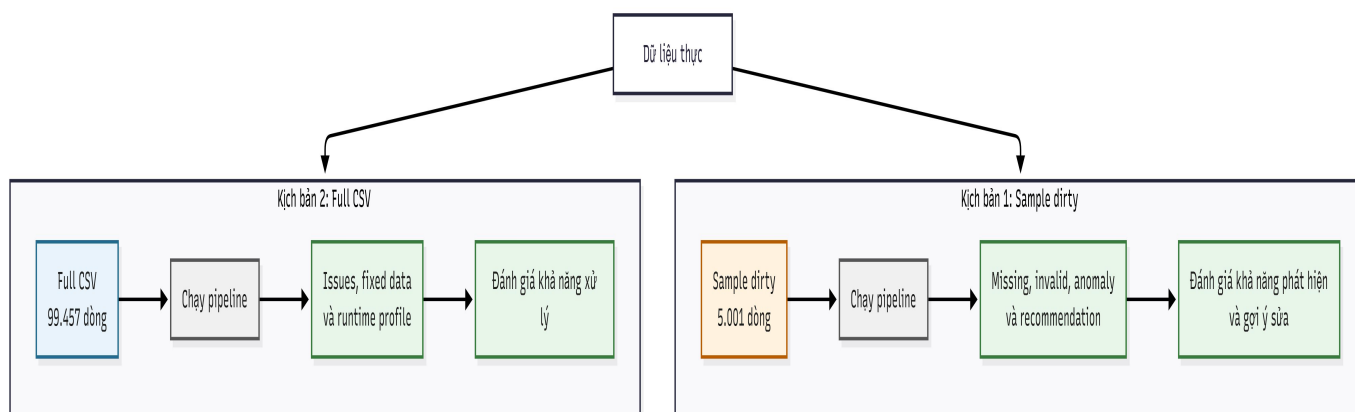
Kịch bản đầu tiên là chạy pipeline trên sample dirty. Lệnh thực nghiệm:

```
python -m app.cli --source csv --input
outputs/e2e_dirty/ecommerce_dirty_sample.csv --config
configs/ecommerce_config.yaml --output outputs/e2e_dirty/run_20260630_9h15 --
apply-fixes --report
```

Kết quả sinh ra các file:

1. `dataset_profile.csv`
2. `detected_issues.csv`
3. `final_report.csv`
4. `fixed_transactions.csv`
5. `fix_recommendations.csv`

Kịch bản thứ hai là chạy pipeline trên full CSV 99.457 dòng. Kết quả được lưu tại `outputs/csv_full_optimized`, gồm cùng các file output như trên. Kịch bản này dùng để kiểm tra khả năng xử lý trên toàn bộ dataset và đo runtime.



Hình 5.2. Kịch bản thực nghiệm

Kịch bản thực nghiệm được thiết kế theo thứ tự từ đơn giản đến đầy đủ. Trước hết, hệ thống chạy với CSV để kiểm tra pipeline độc lập với database. Sau đó, hệ thống có thể chạy với PostgreSQL để kiểm tra khả năng đọc dữ liệu đã chuẩn hóa và ghi output vào các bảng cố định. Cách chia này giúp tách lỗi xử lý dữ liệu khỏi lỗi cấu hình database.

Trong mỗi lần chạy, các output cần kiểm tra gồm:

Bảng 5.3. Cách kiểm tra các tệp đầu ra

Output	Cách kiểm tra
<code>final_report.csv</code>	Kiểm tra tổng số issue, issue theo type, severity và fixable rate
<code>detected_issues.csv</code>	Kiểm tra từng lỗi, cột lỗi, reason và source method
<code>fix_recommendations.csv</code>	Kiểm tra suggested value và confidence
<code>dataset_profile.csv</code>	Kiểm tra missing rate, unique count, issue count theo cột
<code>fixed_transactions.csv</code>	Kiểm tra số dòng và giá trị sau fix

Ngoài kiểm tra file, khi chạy với `--profile-runtime`, cần ghi nhận thời gian từng stage. Đây là căn cứ để đánh giá phần nào của hệ thống cần tối ưu.

Các kịch bản thực nghiệm được sắp xếp theo mức độ tăng dần. Kịch bản sample dirty tập trung vào việc kiểm tra chức năng phát hiện lỗi vì dữ liệu có lỗi rõ ràng. Kịch bản full CSV tập trung vào khả năng vận hành trên toàn bộ dataset và quan sát anomaly

trong dữ liệu gần sạch. Kịch bản PostgreSQL, nếu được chạy bổ sung, dùng để kiểm tra khả năng tích hợp database. Cách sắp xếp này giúp quá trình đánh giá có tính hệ thống thay vì chỉ chạy một lệnh duy nhất rồi đọc kết quả.

5.5. Kết quả trên sample dirty

Trên sample dirty gồm 2.000 dòng, hệ thống phát hiện tổng cộng 269 issue. Kết quả tổng hợp:

Bảng 5.4. Kết quả tổng hợp trên sample dirty

Metric	Giá trị
Tổng số issue	269
Anomaly	229
Invalid	32
Missing	8
Severity high	224
Severity medium	35
Severity low	10
Average confidence	0.1676
Fixable rate	0.9591

Kết quả theo cột có số issue cao nhất:

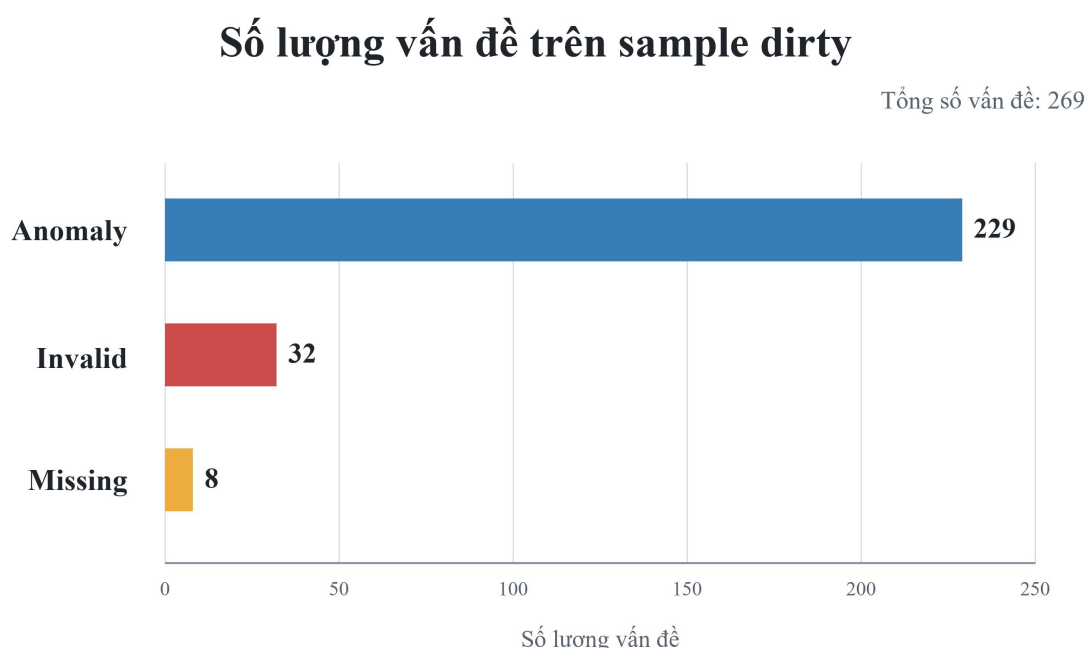
Bảng 5.5. Số issue theo cột trên sample dirty

Cột	Số issue
shopping_mall	99
payment_method	57
gender	27
age_group	26
category	25
quantity	10
unit_price	7
price	6

Cột	Số issue
age	5
quantity_band	3

Kết quả cho thấy pipeline phát hiện được cả ba nhóm lỗi missing, invalid và anomaly. Số lượng anomaly lớn hơn số lỗi được inject là điều có thể xảy ra vì anomaly detection đánh dấu các điểm bất thường theo phân phối dữ liệu, không chỉ những dòng được gắn nhãn lỗi thủ công.

File labels của lần thử nghiệm mới ghi nhận 36 thay đổi được chèn trên 33 dòng, gồm 8 missing, 24 invalid và 4 anomaly. Khi đối chiếu theo row_id, column_name và issue_type, hệ thống phát hiện đủ 36/36 nhãn đã chèn. Các issue phát sinh thêm chủ yếu là điểm bất thường do Isolation Forest và các vi phạm liên trường được suy ra từ feature dẫn xuất.



Hình 5.3. Phân bố số issue theo loại trên sample dirty

Kết quả sample dirty cho thấy hệ thống phát hiện được cả ba nhóm missing, invalid và anomaly. Trong 269 issue, nhóm missing và invalid phản ánh các vi phạm quy tắc cụ thể; nhóm anomaly chiếm số lượng lớn do mô hình đánh dấu những bản ghi có đặc trưng khác biệt so với tập tham chiếu sạch.

Việc anomaly chiếm 229 issue không có nghĩa sample chắc chắn có 229 lỗi. Đây là các điểm bất thường cần kiểm tra do mô hình không giám sát phát hiện theo

contamination và không gian feature hiện tại. Cách diễn giải này phù hợp với bản chất của anomaly detection và tránh kết luận quá mức.

5.5.1. Phân tích dòng lỗi tiêu biểu

Dòng I139441 là trường hợp tiêu biểu trong lần chạy mới. Giá trị age bị thay thành 150 trong khi age_group vẫn là 18_25, từ đó làm phát sinh hai issue invalid và bốn issue anomaly liên quan:

Bảng 5.6. Phân tích một số lỗi tiêu biểu trên sample dirty

Cột	Loại lỗi	Diễn giải
age	invalid	Giá trị 150 vượt ngưỡng tối đa 120; đề xuất 120, confidence 1.0000.
age_group	invalid	Nhóm 18_25 không còn phù hợp với age = 150 theo quy tắc liên trường.
age	anomaly	Tuổi 150 khác biệt rõ rệt so với phân phối tuổi trong tập tham chiếu.
age_group	anomaly	Mô hình ghi nhận 18_25 là feature bất thường và gợi ý 56_plus, confidence 0.0687.
invoice_year	anomaly	Giá trị 2023 tham gia vào tổ hợp feature làm dòng bị cô lập.
shopping_mall	anomaly	Giá trị Istinye Park khác với nhóm láng giềng; gợi ý Mall of Istanbul, confidence 0.0687.

Trường hợp I139441 thể hiện sự kết hợp giữa hai cơ chế phát hiện. Rule-based validation xác định trực tiếp age = 150 vượt miền hợp lệ và age_group không nhất quán; Isolation Forest bổ sung ngữ cảnh khi đánh dấu các feature có đóng góp vào tính bất thường của toàn dòng.

Với issue invalid trên age, hệ thống đề xuất giá trị 120 với confidence 1.0000 nên có thể áp dụng trong mode conservative. Ngược lại, các gợi ý gắn với anomaly chỉ có confidence 0.0687 và cần được xem như thông tin hỗ trợ kiểm tra. Trên toàn sample, có 28 issue vừa can_auto_fix, vừa đạt confidence từ 0.8 trở lên, gồm 24 invalid và 4 missing.

5.6. Kết quả trên full CSV

Với full CSV, hệ thống phát hiện tổng cộng 8.512 issue. Kết quả tổng hợp:

Bảng 5.7. Kết quả tổng hợp trên full CSV

Metric	Giá trị
Tổng số issue	8.512
Anomaly	8.511
Invalid	1
Severity critical	4
Severity high	7.254
Severity medium	1.084
Severity low	170
Average confidence	0.0532
Fixable rate	0.887

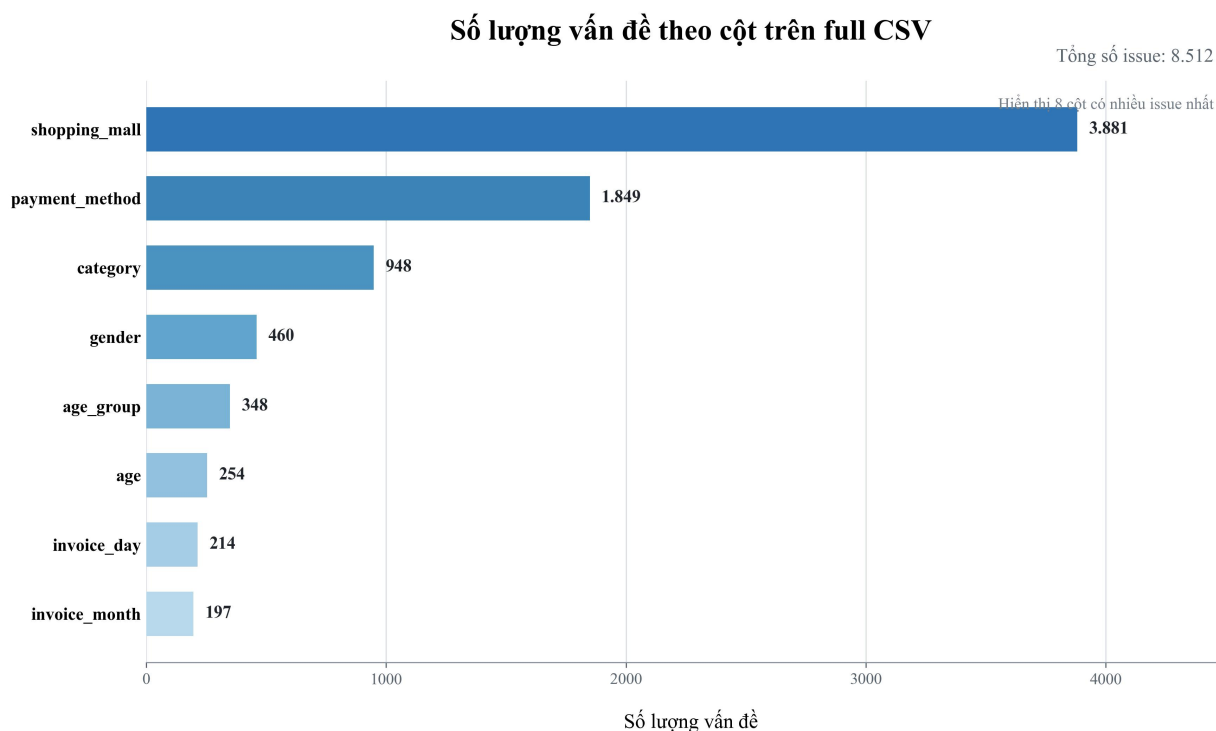
Kết quả theo cột có số issue cao nhất:

Bảng 5.8. Số issue theo cột trên full CSV

Cột	Số issue
shopping_mall	3.881
payment_method	1.849
category	948
gender	460
age_group	348
age	254
invoice_day	214
invoice_month	197
day_of_week	177
price_deviation_from_category	74
quantity_band	64
quantity	23

Kết quả này cho thấy dữ liệu gốc gần như không có missing data, chỉ có một lỗi invalid được ghi nhận ở cột shopping_mall, còn phần lớn issue là anomaly. Điều này phù

hợp với đặc điểm của full dataset: dữ liệu tương đối sạch về mặt rule, nhưng vẫn có các điểm bất thường theo phân phối hoặc theo tổ hợp đặc trưng.



Hình 5.4. Biểu đồ issue theo cột trên full CSV

Khi chạy trên full CSV, kết quả cho thấy dữ liệu gốc tương đối sạch theo rule. Không có missing issue trong final_report, chỉ có một invalid issue, còn phần lớn là anomaly. Điều này phù hợp với kỳ vọng vì dataset Kaggle thường đã được làm sạch ở mức cơ bản trước khi công bố. Tuy nhiên, ngay cả dữ liệu tương đối sạch vẫn có thể có các tổ hợp feature hiếm hoặc khác biệt.

Các cột có nhiều issue nhất là shopping_mall, payment_method, category, gender và age_group. Đây đều là các cột categorical có vai trò mô tả ngữ cảnh giao dịch. Số issue cao ở các cột này có thể đến từ việc feature contribution so sánh giá trị hiện tại với mode của các bản ghi láng giềng. Nếu một bản ghi có tổ hợp giá/quantity/thời gian giống một nhóm giao dịch, nhưng lại khác về shopping mall hoặc payment method, cột đó có thể bị đánh dấu nghi ngờ.

Cần lưu ý rằng các issue ở full CSV không nhất thiết là lỗi cần sửa. Ví dụ, một phương thức thanh toán khác với mode của nhóm láng giềng không có nghĩa là sai. Nó có thể chỉ là hành vi mua hàng ít phổ biến. Vì vậy, trong ứng dụng thực tế, các anomaly nên được ưu tiên đưa vào danh sách review thay vì auto-fix toàn bộ.

5.6.1. Nhận xét về tỷ lệ anomaly

Trên full CSV, hệ thống đánh dấu 8.511 bản ghi anomaly trong tổng số 99.457 dòng, tương ứng khoảng 8,56%. Kết quả này gần với $\text{contamination}=0.08$, nhưng không nên được diễn giải rằng mô hình đã xác nhận 8,56% dữ liệu là sai. contamination chủ yếu xác định ngưỡng tách outlier trên phân phối điểm số của từng mô hình; nó không phải tỷ lệ lỗi thực tế và cũng không phải chỉ số độ chính xác.

Tỷ lệ tổng thể có thể chênh lệch so với 8% vì dữ liệu được chấm bởi nhiều mô hình: các category đủ lớn sử dụng segment model, còn các nhóm khác dùng global model. Mỗi mô hình có phân phối `score_samples`, `offset` và kích thước tập fit riêng. Ngoài ra, các giá trị điểm số nằm gần hoặc trùng ngưỡng có thể làm số nhãn thực tế không khớp tuyệt đối với tỷ lệ cấu hình.

Số bản ghi anomaly cần được phân biệt với số `detected_issues`. Một bản ghi có thể tạo nhiều issue nếu FeatureContributionAnalyzer xác định nhiều cột nghi ngờ. Vì vậy, tỷ lệ anomaly phản ánh phạm vi bản ghi cần review, trong khi số issue phản ánh khối lượng vấn đề ở cấp cột. Hai đại lượng phục vụ hai mục đích quản lý khác nhau và không nên cộng hoặc so sánh trực tiếp.

Việc lựa chọn contamination là một bài toán cân bằng giữa khả năng phát hiện và chi phí kiểm tra. contamination thấp làm danh sách cảnh báo ngắn hơn nhưng có thể bỏ sót những trường hợp nằm sát ngưỡng; contamination cao tăng độ bao phủ nhưng làm người dùng phải xem xét nhiều bản ghi hợp lệ hiếm gặp. Trong phạm vi đề án, giá trị 0.08 được xem là cấu hình thực nghiệm để tạo danh sách ưu tiên kiểm tra, không phải kết luận về tỷ lệ dữ liệu bản của bộ dữ liệu.

Khi đánh giá sâu hơn, cần quan sát phân phối `decision_score` thay vì chỉ đếm nhãn nhị phân. Các bản ghi có điểm âm rất xa 0 thường đáng được ưu tiên hơn các bản ghi chỉ vừa vượt ngưỡng. Với mô hình segment, việc xếp hạng nên thực hiện trong từng category; nếu cần xếp hạng toàn cục, phải chuẩn hóa điểm theo phân phối của từng mô hình trước khi so sánh.

Do bộ dữ liệu chưa có ground truth đầy đủ cho anomaly, báo cáo không sử dụng số lượng 8.511 để tính precision, recall hoặc F1. Đánh giá phù hợp hơn là kiểm tra thủ công một mẫu phân tầng theo mức điểm, phân tích tính hợp lệ nghiệp vụ của các bản ghi đúng đầu và đo mức ổn định của thứ hạng khi thay đổi seed hoặc tham số.

5.7. Kết quả gợi ý sửa lỗi

Trên sample dirty, hệ thống sinh 417 recommendation tương ứng với 417 issue. Trên full CSV, hệ thống sinh 8.512 recommendation tương ứng với 8.512 issue. Điều này cho thấy hệ thống có khả năng đưa ra hành động khuyến nghị cho tất cả issue được phát hiện.

Tuy nhiên, không phải mọi recommendation đều nên được hiểu là sửa tự động chắc chắn. Với missing và invalid rõ ràng, gợi ý sửa có thể được dùng trực tiếp nếu đủ điều kiện. Với anomaly, đặc biệt là anomaly numeric, recommendation nên được xem là thông tin hỗ trợ review. Đây là điểm cần nhấn mạnh khi trình bày với giảng viên, vì hệ thống hướng đến AI-assisted data cleaning chứ không phải tự động thay thế mọi giá trị.

row_id	column_name	issue_type	current_value	suggested_value	confidence	severity	can_auto_fix
I127801	age	missing		24	0.7576	high	TRUE
I317333	category	missing		Shoes	0.6	medium	TRUE
I127711	quantity	missing		2	0.6536	high	TRUE
I127711	price	missing		60.6	0.5263	high	TRUE
I138884	payment_method	missing		Credit Card	0.6	medium	TRUE
I293112	invoice_date	missing			0	high	TRUE
I127711	invoice_date	missing			0	high	TRUE
I127711	gender	invalid	Woman	Female	0.5	medium	TRUE
I121056	age	invalid	150	120	1	high	TRUE
I337046	category	invalid	Tech	Books	0.5	medium	TRUE
I127711	category	invalid	Cloth	Clothing	0.7692	medium	TRUE
I227836	quantity	invalid	-5	1	1	high	TRUE
I173702	payment_method	invalid	Paypal	Cash	0.5	medium	TRUE
I127711	payment_method	invalid	Cre Card	Credit Card	0.8421	medium	TRUE
I196704	shopping_mall	invalid	MALL	Cevahir AVM	0.5	medium	TRUE
I127711	shopping_mall	invalid	Mall of	Mall of Istanbul	0.6087	medium	TRUE
I227836	unit_price	invalid	300.08000000000004		0	high	TRUE
I293455	unit_price	invalid	999999		0	high	TRUE
I139207	unit_price	invalid	15.15		0	high	TRUE
I640508	unit_price	invalid	3584.0000000000005		0	high	TRUE
I121056	age_group	invalid	46_55		0	medium	TRUE

Hình 5.5. Ví dụ bảng fix_recommendations

Kết quả recommendation cần được phân tích theo loại issue. Với missing numeric, gợi ý dựa trên median của các bản ghi tương tự. Với missing categorical, gợi ý dựa trên mode. Với invalid numeric, gợi ý thường là giá trị min/max hợp lệ. Với invalid categorical, gợi ý dựa trên giá trị hợp lệ gần nhất hoặc giá trị mặc định trong allowed values. Với anomaly categorical, gợi ý dựa trên mode của các láng giềng. Với anomaly numeric, hệ thống không đưa ra suggested value để tránh sửa sai.

Thiết kế này cho thấy hệ thống không áp dụng một công thức sửa duy nhất cho mọi lỗi. Mỗi loại lỗi có chiến lược recommendation riêng. Đây là điểm quan trọng để trình bày tính hợp lý của hệ thống.

Một số recommendation có confidence thấp. Điều này không phải lỗi của hệ thống mà là tín hiệu cho thấy dữ liệu tham chiếu không đủ đồng thuận để tự động sửa. Trong chế độ conservative, các recommendation có confidence thấp sẽ không được áp dụng vào fixed_transactions. Nhờ vậy, hệ thống vẫn lưu được gợi ý để người dùng tham khảo nhưng không làm thay đổi dữ liệu một cách thiếu chắc chắn.

5.8. Kết quả apply fixes

Khi bật tùy chọn --apply-fixes, hệ thống sinh file fixed_transactions.csv. Với sample dirty, file fixed có 5.001 dòng. Với full CSV, file fixed có 99.457 dòng, tương ứng với số giao dịch đầu vào.

Kết quả này cho thấy pipeline có thể tạo ra phiên bản dữ liệu sau xử lý mà không làm mất số lượng bản ghi. Đây là yếu tố quan trọng vì hệ thống làm sạch dữ liệu cần bảo toàn khả năng đối chiếu giữa dữ liệu gốc và dữ liệu sau xử lý.

row_id	column_name	issue_type	current_value	suggested_value	confidence	can_auto_fix
I127801	age	missing		24	0.7576	TRUE
I317333	category	missing		Shoes	0.6	TRUE
I127711	quantity	missing		2	0.6536	TRUE
I127711	price	missing		60.6	0.5263	TRUE
I138884	payment_method	missing		Credit Card	0.6	TRUE
I293112	invoice_date	missing			0	TRUE
I127711	invoice_date	missing			0	TRUE
I127711	gender	invalid	Woman	Female	0.5	TRUE
I121056	age	invalid	150	120	1	TRUE
I337046	category	invalid	Tech	Books	0.5	TRUE
I127711	category	invalid	Cloth	Clothing	0.7692	TRUE
I227836	quantity	invalid	-5	1	1	TRUE
I173702	payment_method	invalid	Paypal	Cash	0.5	TRUE
I127711	payment_method	invalid	Cre Card	Credit Card	0.8421	TRUE
I196704	shopping_mall	invalid	MALL	Cevahir AVM	0.5	TRUE
I127711	shopping_mall	invalid	Mall of	Mall of Istanbul	0.6087	TRUE
I227836	unit_price	invalid	300.08000000000004		0	TRUE
I293455	unit_price	invalid	999999		0	TRUE
I139207	unit_price	invalid	15.15		0	TRUE
I640508	unit_price	invalid	3584.0000000000005		0	TRUE
I121056	age_group	invalid	46_55		0	TRUE
I306368	invoice_month	invalid	1		0	TRUE
I317333	category	anomaly		Shoes	0.0152	TRUE
I317333	price	anomaly	1800.51		0.0152	FALSE
I317333	payment_method	anomaly	Debit Card	Cash	0.0152	TRUE
I317333	shopping_mall	anomaly	Forum Istanbul	Kanyon	0.0152	TRUE
I317333	unit price	anomaly	600.17		0.0152	FALSE

Hình 5.6. So sánh dữ liệu trước và sau fix

Quá trình apply fixes trong hệ thống được thực hiện sau khi đã có `issues_df` hoàn chỉnh. `FixService` duyệt qua từng `issue`, kiểm tra `can_auto_fix`, kiểm tra `suggested_value`, kiểm tra `confidence` theo mode và sau đó mới cập nhật giá trị trong `dataframe` đầu ra.

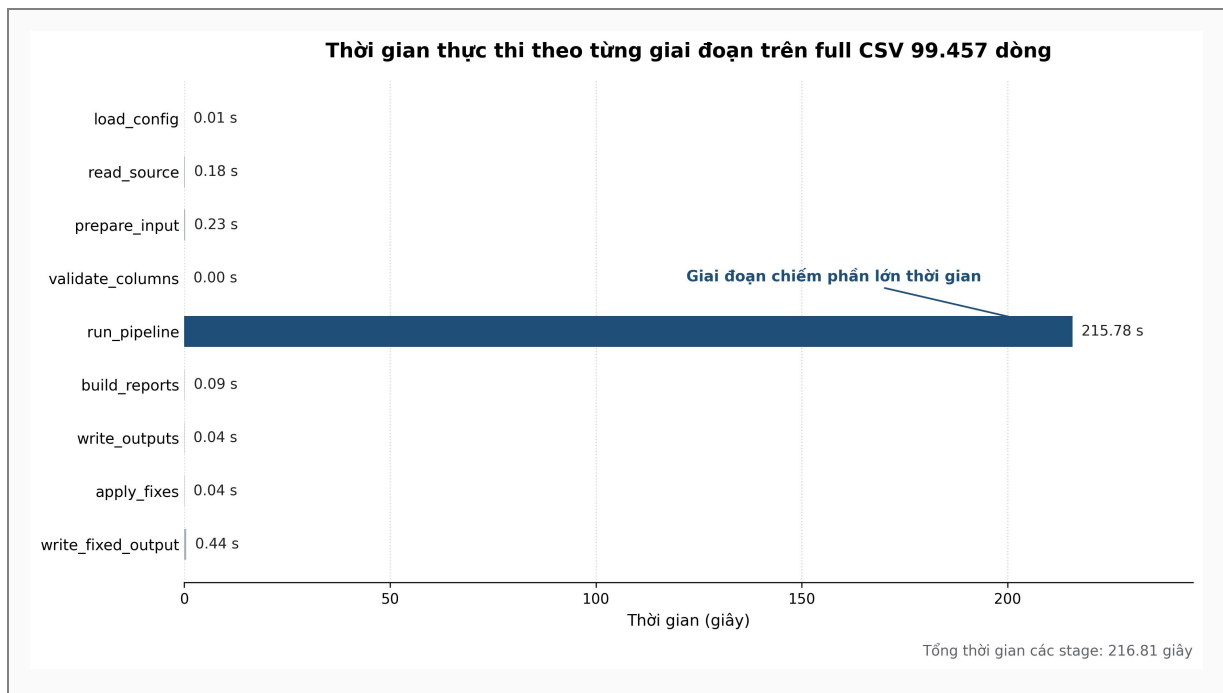
Trong mode `conservative`, chỉ các gợi ý có `confidence` lớn hơn hoặc bằng `conservative_confidence_threshold` mới được áp dụng. Điều này giúp giảm rủi ro trong các trường hợp recommender không đủ chắc chắn. Trong mode `interactive`, hệ thống hỏi người dùng từng fix, phù hợp nếu muốn kiểm tra thủ công. Trong mode `auto`, hệ thống áp dụng các fix có thể auto-fix mà không hỏi thêm, nhưng chế độ này cần thận trọng khi dùng với dữ liệu thật.

Với full CSV, `fixed_transactions.csv` có 99.457 dòng, bằng số dòng giao dịch đầu vào. Điều này cho thấy hệ thống không làm mất bản ghi trong quá trình apply fixes. Việc bảo toàn số dòng là yêu cầu quan trọng vì dữ liệu sau làm sạch vẫn cần đối chiếu được với dữ liệu gốc qua `invoice_no`.

5.9. Đánh giá hiệu năng

Trên full CSV 99.457 dòng, hệ thống chạy thành công với tổng thời gian khoảng 216,81 giây. Runtime profile cho thấy bottleneck chính nằm ở stage `run_pipeline`, tức phần `detect`, `recommend` và `explain`. Phần đọc/ghi dữ liệu không còn là nút thắt chính.

Kết quả này cho thấy hệ thống có thể chạy được trên dataset gần 100.000 dòng trong phạm vi đồ án. Tuy nhiên, nếu mở rộng lên dữ liệu lớn hơn hoặc yêu cầu xử lý gần thời gian thực, cần tiếp tục tối ưu thuật toán `anomaly detection`, `feature contribution` và `nearest-neighbor recommendation`.



Hình 5.7. Thời gian thực thi theo từng giai đoạn trên full CSV 99.457 dòng

Runtime full CSV khoảng 216,81 giây cho thấy hệ thống có thể xử lý tập dữ liệu gần 100.000 dòng trong phạm vi đề án, nhưng chưa tối ưu cho xử lý thời gian thực. Stage `run_pipeline` là bottleneck chính vì bao gồm nhiều bước tính toán nặng: chạy detector, fit/predict Isolation Forest, tìm feature contribution, tìm nearest neighbors và sinh recommendation.

Có ba nguyên nhân chính khiến `run_pipeline` tốn thời gian. Thứ nhất, Isolation Forest và feature preprocessing phải xử lý nhiều feature sau khi one-hot encoding. Thứ hai, feature contribution cần tìm láng giềng và so sánh từng anomaly row với nhóm tham chiếu. Thứ ba, recommendation cũng sử dụng nearest-neighbor search trên dữ liệu hỗn hợp, có thể tốn thời gian khi số issue lớn.

Hệ thống đã có một số tối ưu như build `row_lookup` một lần, cache neighbor và cache dữ liệu chuẩn hóa trong nearest neighbors. Tuy nhiên, nếu muốn mở rộng lên dataset lớn hơn, cần cân nhắc thêm các hướng tối ưu:

1. Dùng approximate nearest neighbors.
2. Batch hóa việc tính khoảng cách.
3. Giới hạn recommendation cho các issue có severity cao.
4. Tối ưu feature set để giảm số chiều sau one-hot encoding.
5. Lưu kết quả trung gian hoặc xử lý incremental theo dữ liệu mới.

Trong phạm vi đề án, kết quả runtime hiện tại đủ để chứng minh hệ thống chạy được end-to-end. Tuy nhiên, đây vẫn là một hạn chế cần nêu trong phần hướng phát triển.

5.10. Nhận xét kết quả thực nghiệm

Kết quả thực nghiệm cho thấy hệ thống đáp ứng được các yêu cầu chức năng chính. Pipeline có thể đọc dữ liệu, chuẩn hóa, phát hiện lỗi, sinh gợi ý sửa, tạo báo cáo và xuất dữ liệu sau xử lý. Các output được tổ chức rõ ràng, phù hợp để kiểm tra và trình bày.

Đối với sample dirty, hệ thống phát hiện được cả missing, invalid và anomaly. Điều này chứng minh pipeline có khả năng xử lý dữ liệu lỗi được tạo ra có chủ đích. Tuy nhiên, vì file labels chưa hoàn chỉnh cho toàn bộ sample, việc tính precision/recall cần được thực hiện cẩn trọng hoặc cần cập nhật lại ground truth.

Đối với full CSV, số lượng anomaly chiếm phần lớn tổng issue. Đây là đặc điểm thường gặp khi áp dụng anomaly detection không giám sát. Các kết quả anomaly nên được xem là cảnh báo để người dùng kiểm tra thêm, đặc biệt với các cột như shopping_mall, payment_method, category và age_group.

Một điểm tích cực là hệ thống tạo ra output nhất quán giữa sample dirty và full CSV. Dù số lượng lỗi khác nhau, các file output vẫn có cùng cấu trúc. Điều này cho thấy thiết kế pipeline có tính ổn định.

Một điểm cần thận trọng là confidence trung bình của full CSV tương đối thấp. Điều này phản ánh bản chất của anomaly/recommendation trong dữ liệu gán sạch: nhiều điểm bị xem là bất thường nhưng không có bằng chứng đủ mạnh để tự động sửa. Vì vậy, việc sử dụng chế độ conservative là hợp lý.

Khi trình bày với giảng viên, nên nhấn mạnh rằng hệ thống không khẳng định mọi anomaly đều là lỗi. Hệ thống đóng vai trò hỗ trợ phát hiện và ưu tiên kiểm tra. Đây là cách diễn giải chính xác của AI-assisted data cleaning.

5.10.1. So sánh rule-based only và hybrid approach

Nếu chỉ sử dụng rule-based validation, hệ thống có thể phát hiện tốt missing, invalid và cross-field inconsistency vì các lỗi này có điều kiện nghiệp vụ rõ ràng. Ưu điểm của lớp luật là kết quả dễ giải thích và có thể xác định trực tiếp cột vi phạm. Hạn chế là luật không bao quát được mọi tổ hợp hiếm hoặc quan hệ phân phối phức tạp giữa nhiều đặc trưng.

Khi bổ sung Isolation Forest, hệ thống phát hiện thêm các bản ghi có cấu trúc khác biệt so với clean reference set. Phần học máy vì vậy mở rộng phạm vi quan sát vượt ra ngoài các miền giá trị cố định. Đổi lại, anomaly chỉ là bằng chứng thống kê và cần được

review; mô hình cũng không tự xác định nguyên nhân ở cấp cột. Đây là lý do đồ án bổ sung feature contribution và không cho phép anomaly numeric tự động sinh giá trị sửa.

Hai hướng tiếp cận không thay thế lẫn nhau mà tạo thành một pipeline phân tầng. Rule-based validation xử lý trước những lỗi chắc chắn; tập tham chiếu sạch hơn được dùng để fit mô hình; Isolation Forest tạo danh sách bản ghi nghi ngờ; module giải thích và recommender hỗ trợ bước xem xét tiếp theo. Cấu trúc này vừa giữ được tính kiểm soát của hệ thống quy tắc, vừa tận dụng khả năng phát hiện mẫu hiếm của học máy.

Bảng 5.9. So sánh rule-based only và hybrid approach

Tiêu chí	Rule-based only	Hybrid approach
Phát hiện missing	Tốt	Tốt
Phát hiện invalid	Tốt	Tốt
Phát hiện cross-field	Tốt nếu có rule	Tốt nếu có rule
Phát hiện anomaly theo ngữ cảnh	Hạn chế	Có
Tính giải thích	Cao	Trung bình đến cao nếu có feature contribution
Nguy cơ false positive	Thấp	Cao hơn
Cần review thủ công	Ít	Nhiều hơn với anomaly

Từ bảng so sánh có thể kết luận rằng giá trị của hybrid approach không nằm ở việc tạo ra nhiều cảnh báo nhất, mà ở việc phân chia đúng vai trò: luật đưa ra kết luận xác định đối với vi phạm rõ ràng, còn Isolation Forest ưu tiên các trường hợp cần con người hoặc chuyên gia nghiệp vụ kiểm tra. Cách diễn giải này phù hợp với định hướng AI-assisted data cleaning của đề tài.

5.10.2. Phân tích độ nhạy của cấu hình Isolation Forest

Kết quả hiện tại được tạo từ một cấu hình chính gồm $n_estimators=200$, $contamination=0.08$, $max_samples$ bằng số dòng của tập fit và $random_state=42$. Vì Isolation Forest có thành phần ngẫu nhiên, một cấu hình đơn lẻ chưa đủ để khẳng định kết quả ổn định trong mọi điều kiện. Phân tích độ nhạy cần xem xét cả số lượng anomaly, thứ hạng điểm số và thời gian chạy.

Đối với $contamination$, nên thử một dải giá trị thấp hơn và cao hơn 0.08, sau đó so sánh mức chồng lấp giữa các nhóm bản ghi đứng đầu. Nếu các anomaly có điểm mạnh vẫn xuất hiện ổn định khi ngưỡng thay đổi, đó là bằng chứng tốt hơn so với việc chỉ quan

sát tổng số nhãn. Các bản ghi chỉ xuất hiện ở một ngưỡng cao nên được xếp vào nhóm cần xem xét thận trọng.

Đối với `max_samples`, cần so sánh cấu hình lấy mẫu con, chẳng hạn 256 hoặc một tỷ lệ của tập fit, với cấu hình dùng toàn bộ `n_rows`. Tiêu chí so sánh gồm runtime, độ sâu trung bình, độ ổn định của top-k anomaly và hiện tượng masking trong các category lớn. Thử nghiệm này đặc biệt quan trọng vì subsampling là thành phần của thiết kế thuật toán gốc [8].

Đối với `n_estimators` và `random_state`, có thể chạy nhiều seed ở các mức số cây khác nhau rồi đo tỷ lệ giao nhau của top-k anomaly. Khi số cây tăng, mức biến động giữa các seed được kỳ vọng giảm, nhưng chi phí xử lý tăng. Báo cáo hiện tại mới ghi nhận cấu hình triển khai; các so sánh trên được đề xuất như bước kiểm chứng bổ sung, không được trình bày như kết quả đã thực hiện.

5.10.3. Các yếu tố ảnh hưởng đến độ tin cậy

Kết quả thực nghiệm của đồ án có một số yếu tố ảnh hưởng đến độ tin cậy.

Thứ nhất, ground truth của sample dirty chưa hoàn chỉnh. Việc dòng **I127711** có lỗi nhưng không nằm trong labels cho thấy không thể tính precision/recall/F1 cho toàn bộ sample nếu chưa cập nhật nhãn đầy đủ.

Thứ hai, anomaly detection không có nhãn đúng/sai tuyệt đối. Một điểm bất thường có thể là lỗi hoặc là dữ liệu hợp lệ hiếm gặp. Vì vậy, số lượng anomaly không nên được diễn giải như số lượng lỗi chắc chắn.

Thứ ba, dataset chỉ thuộc một lĩnh vực e-commerce, chưa đánh giá trên nhiều loại dữ liệu khác nhau. Nếu áp dụng sang dữ liệu y tế, tài chính hoặc giáo dục, rule và feature cần thay đổi đáng kể.

Thứ tư, runtime phụ thuộc vào môi trường máy chạy. Kết quả 216,81 giây cần được hiểu trong bối cảnh cấu hình máy cụ thể và cách chạy hiện tại.

Thứ năm, hệ thống chưa có đánh giá người dùng cuối. Các recommendation được sinh tự động nhưng chưa có quy trình để chuyên gia nghiệp vụ xác nhận đúng/sai trên quy mô lớn.

Thứ sáu, thí nghiệm Isolation Forest hiện sử dụng một `random_state` và một cấu hình `max_samples`. Do chưa có đánh giá nhiều seed và chưa so sánh lấy mẫu con với toàn bộ tập fit, mức ổn định của thứ hạng anomaly chưa được định lượng đầy đủ.

Thứ bảy, one-hot encoding có thể làm các cột phân loại nhiều mức tạo ra số lượng đặc trưng lớn hơn các cột khác. Nếu không kiểm soát cardinality và ý nghĩa của từng cột, cấu trúc không gian đặc trưng có thể ảnh hưởng đến xác suất lựa chọn phép chia và làm thay đổi điểm bất thường.

5.11. Ưu điểm của hệ thống

Hệ thống có một số ưu điểm chính:

1. Kết hợp được rule-based validation và học máy trong cùng một pipeline.
2. Có khả năng chạy từ CSV hoặc PostgreSQL.
3. Thiết kế database rõ ràng, tách dữ liệu nghiệp vụ, dữ liệu thô và dữ liệu phục vụ cleaning.
4. Output đầy đủ, bao gồm issue, recommendation, profile, report và fixed data.
5. Có cơ chế cấu hình bằng YAML, giúp dễ thay đổi rule và tham số.
6. Có runtime profile, giúp xác định bottleneck khi tối ưu.
 - Có cơ chế apply fixes có kiểm soát, tránh sửa dữ liệu số bất thường một cách tùy tiện.

5.12. Hạn chế của hệ thống

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế:

- Dataset chỉ thuộc lĩnh vực e-commerce, chưa kiểm chứng trên nhiều lĩnh vực dữ liệu khác.
- Bảng products mới đại diện theo category, chưa có SKU-level product.
- Ground truth cho sample dirty chưa hoàn chỉnh, gây khó khăn khi tính precision/recall chính xác.
- Anomaly detection có thể sinh false positive vì bất thường không luôn đồng nghĩa với lỗi.
- Full pipeline còn mất thời gian khi chạy trên toàn bộ dữ liệu.
- Hệ thống hiện chủ yếu vận hành qua CLI, chưa có giao diện dashboard cho người dùng cuối.

5.13. Đánh giá tổng hợp theo mục tiêu đề tài

Đối chiếu với các mục tiêu đã đặt ra ở Chương 1, hệ thống đã đạt được phần lớn các mục tiêu chính. Thứ nhất, hệ thống đã chuẩn hóa được dữ liệu giao dịch và sinh các

feature phục vụ phát hiện lỗi. Thứ hai, hệ thống đã thiết kế được cơ sở dữ liệu PostgreSQL gồm các bảng nghiệp vụ và các bảng phục vụ data cleaning. Thứ ba, pipeline đã phát hiện được missing, invalid và anomaly. Thứ tư, hệ thống đã sinh được recommendation và fixed transactions. Thứ năm, hệ thống đã chạy được trên full CSV 99.457 dòng và ghi nhận runtime để đánh giá hiệu năng.

Tuy nhiên, một số mục tiêu vẫn cần hoàn thiện thêm để đạt mức tốt hơn. Phần đánh giá precision/recall/F1 chưa thể trình bày như kết quả cuối cùng vì ground truth của sample dirty chưa hoàn chỉnh. Phần anomaly detection đã phát hiện được nhiều điểm nghi vấn nhưng vẫn cần cơ chế review để phân biệt lỗi thật và dữ liệu hợp lệ hiếm gặp. Phần giao diện người dùng chưa được triển khai, nên hiện tại hệ thống phù hợp hơn với người dùng kỹ thuật thông qua CLI.

Bảng đối chiếu tổng hợp:

Bảng 5.10. Đánh giá mức độ đáp ứng mục tiêu đề tài

Mục tiêu	Trạng thái	Nhận xét
Chuẩn hóa data e-commerce	Đạt	Đã chuẩn hóa cột, kiểu dữ liệu, ngày tháng và feature
Thiết kế PostgreSQL	Đạt	Có bảng nghiệp vụ, staging, cleaning và output
Phát hiện missing/invalid	Đạt	Dựa trên rule YAML và cross-field validator
Phát hiện anomaly	Đạt một phần tốt	Có Isolation Forest và feature contribution, nhưng cần review false positive
Gợi ý sửa lỗi	Đạt	Có recommender cho missing, invalid, anomaly categorical
Apply fixes có kiểm soát	Đạt	Có conservative/auto/interactive mode
Đánh giá định lượng đầy đủ	Chưa hoàn chỉnh	Cần hoàn thiện labels nếu muốn precision/recall/F1 chắc chắn
Giao diện người dùng	Chưa thực hiện	Hướng phát triển tiếp theo

Từ bảng trên, có thể kết luận rằng hệ thống đã hoàn thành vai trò của một pipeline AI-assisted data cleaning trong phạm vi đề án. Các hạn chế còn lại chủ yếu nằm ở phần

mở rộng đánh giá, giao diện và tối ưu hiệu năng, không làm thay đổi cấu trúc chính của hệ thống.

5.14. Kết luận chương

Chương 5 đã trình bày quá trình thực nghiệm và đánh giá hệ thống. Kết quả cho thấy hệ thống đã chạy được end-to-end, phát hiện được nhiều nhóm lỗi và sinh đầy đủ các output cần thiết. Hệ thống cũng đã được kiểm tra trên dữ liệu đầy đủ gần 100.000 dòng, qua đó chứng minh pipeline có thể vận hành trên tập dữ liệu có quy mô tương đối lớn trong phạm vi đồ án.

Từ kết quả thực nghiệm, có thể thấy hướng tiếp cận hybrid là phù hợp. Rule-based validation giúp phát hiện các lỗi rõ ràng và dễ giải thích. Isolation Forest giúp mở rộng khả năng phát hiện các điểm bất thường theo ngữ cảnh. Nearest-neighbor recommendation giúp hệ thống không chỉ dừng ở phát hiện lỗi mà còn hỗ trợ đề xuất sửa. Tuy nhiên, anomaly detection vẫn cần được diễn giải thận trọng vì không phải mọi anomaly đều là lỗi thật.

Những kết quả và hạn chế trong chương này là cơ sở cho phần kết luận chung. Phần tiếp theo sẽ tổng hợp các kết quả đạt được, nêu rõ các hạn chế còn tồn tại và đề xuất hướng phát triển cho hệ thống trong tương lai.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Đồ án đã xây dựng được một hệ thống hỗ trợ làm sạch dữ liệu giao dịch thương mại điện tử bằng cách kết hợp rule-based validation và học máy. Hệ thống có thể đọc dữ liệu từ CSV hoặc PostgreSQL, chuẩn hóa dữ liệu, sinh feature, phát hiện lỗi missing, invalid và anomaly, đưa ra gợi ý sửa lỗi, tạo dữ liệu sau xử lý và sinh báo cáo tổng hợp.

Về thiết kế dữ liệu, đồ án đã chuyển dataset ban đầu từ một bảng phẳng sang mô hình quan hệ gồm customers, products, shopping_malls và transactions. Ngoài ra, schema còn có bảng staging, bảng quản lý phiên chạy và các bảng phục vụ data cleaning gồm detected_issues, fix_recommendations, cleaning_actions, dataset_profile và fixed_transactions. final_report là output tổng hợp do tầng Python sinh ra, không phải bảng được tạo trong postgresql_schema.sql.

Về mặt xử lý, hệ thống đã triển khai được pipeline rõ ràng, có khả năng cấu hình bằng YAML và có thể đo runtime từng stage. Kết quả thực nghiệm cho thấy hệ thống chạy được trên sample dirty và full CSV 99.457 dòng, phát hiện được lỗi và tạo đầy đủ output phục vụ kiểm tra.

Kết quả quan trọng nhất của đồ án là xây dựng được một quy trình có tính liên kết từ dữ liệu đầu vào đến dữ liệu sau xử lý. Dữ liệu không chỉ được đọc và kiểm tra trong bộ nhớ, mà còn được chuẩn hóa, mô hình hóa trong database, phát hiện lỗi, gợi ý sửa và ghi lại kết quả. Cách tiếp cận này làm cho hệ thống có khả năng truy vết, giải thích và mở rộng tốt hơn so với một chương trình kiểm tra CSV đơn giản.

Về mặt chuyên môn, đồ án đã thể hiện được sự kết hợp giữa ba nhóm kỹ thuật. Nhóm thứ nhất là kỹ thuật cơ sở dữ liệu và ETL, thể hiện qua việc tách bảng, thiết kế schema, import dữ liệu và đọc dữ liệu bằng source_query. Nhóm thứ hai là kỹ thuật kiểm tra chất lượng dữ liệu bằng rule, thể hiện qua missing detector, invalid detector và cross-field validator. Nhóm thứ ba là kỹ thuật học máy hỗ trợ data cleaning, thể hiện qua Isolation Forest, feature contribution và nearest-neighbor recommendation.

Về mặt thực nghiệm, hệ thống đã sinh được các output có ý nghĩa như detected_issues, fix_recommendations, dataset_profile, fixed_transactions và final_report. Các output này không chỉ phục vụ kiểm tra kết quả mà còn có thể dùng làm cơ sở cho dashboard hoặc quy trình phê duyệt sửa lỗi trong tương lai.

2. Hạn chế

Mặc dù hệ thống đã đáp ứng được mục tiêu chính, đề tài vẫn còn một số hạn chế. Thứ nhất, dữ liệu thực nghiệm chỉ tập trung vào một dataset e-commerce, nên chưa thể khẳng định khả năng tổng quát hóa sang mọi loại dữ liệu khác. Thứ hai, phần ground truth cho dữ liệu dirty chưa hoàn chỉnh, làm cho việc đánh giá precision, recall và F1-score cần được hoàn thiện thêm. Thứ ba, anomaly detection có thể tạo ra false positive, do đó kết quả anomaly cần có bước review của người dùng. Thứ tư, pipeline còn bottleneck ở phần detect/recommend/explain khi chạy trên dữ liệu lớn. Cuối cùng, hệ thống hiện chưa có giao diện trực quan để người dùng nghiệp vụ duyệt lỗi và xác nhận sửa lỗi.

Các hạn chế này phản ánh đúng phạm vi của một đề án tốt nghiệp. Hệ thống hiện đã chứng minh được ý tưởng và khả năng vận hành cơ bản, nhưng để trở thành một sản phẩm hoàn chỉnh cần có thêm thời gian để tối ưu, kiểm thử trên nhiều nguồn dữ liệu và bổ sung trải nghiệm người dùng. Đặc biệt, phần anomaly detection cần được kết hợp với đánh giá của con người để phân biệt dữ liệu sai thật và dữ liệu hiếm nhưng hợp lệ.

3. Hướng phát triển

Trong tương lai, đề tài có thể được phát triển theo các hướng sau:

1. Xây dựng dashboard web để hiển thị issue, recommendation, dataset profile và trạng thái apply fixes.
2. Bổ sung cơ chế human-in-the-loop để người dùng xác nhận hoặc từ chối gợi ý sửa lỗi.
3. Hoàn thiện ground truth cho dữ liệu dirty để tính precision, recall và F1-score chính xác hơn.
4. Tối ưu hiệu năng của anomaly detection, feature contribution và nearest-neighbor recommendation.
5. Mở rộng hệ thống để hỗ trợ nhiều loại cơ sở dữ liệu và nhiều dạng dataset khác nhau.
6. Bổ sung thêm thuật toán như Local Outlier Factor, One-Class SVM hoặc clustering-based anomaly detection để so sánh với Isolation Forest.
7. Lưu lịch sử nhiều lần chạy pipeline để theo dõi xu hướng chất lượng dữ liệu theo thời gian.

Ngoài các hướng trên, hệ thống cũng có thể mở rộng theo hướng tích hợp với quy trình quản trị dữ liệu trong doanh nghiệp. Ví dụ, mỗi issue có thể được gán trạng thái như new, reviewed, approved hoặc rejected. Người dùng nghiệp vụ có thể xác nhận gợi ý sửa lỗi, còn hệ thống học lại từ các quyết định đó để cải thiện recommendation. Đây là hướng human-in-the-loop phù hợp với bản chất của data cleaning, vì không phải mọi quyết định sửa dữ liệu đều nên tự động hóa hoàn toàn.

Một hướng phát triển khác là tăng khả năng đánh giá và giám sát theo thời gian. Khi dữ liệu mới được nạp định kỳ, hệ thống có thể chạy lại pipeline và so sánh số lỗi giữa các lần chạy. Nếu một cột đột ngột tăng tỷ lệ missing hoặc anomaly, hệ thống có thể cảnh báo người quản trị. Khi đó, đề án có thể phát triển thành một hệ thống data quality monitoring thay vì chỉ là pipeline làm sạch dữ liệu một lần.

Tổng kết lại, đề án đã xây dựng được một nền tảng data cleaning có tính thực tiễn, kết hợp giữa thiết kế cơ sở dữ liệu, xử lý dữ liệu và AI/ML. Hệ thống không chỉ hỗ trợ phát hiện lỗi mà còn tạo ra các output có thể kiểm tra, giải thích và sử dụng cho các bước phân tích dữ liệu tiếp theo. Đây là nền tảng phù hợp để tiếp tục phát triển thành một công cụ hỗ trợ quản lý chất lượng dữ liệu trong các hệ thống thông tin thực tế.

Các nguồn tham khảo

- [1] scikit-learn developers, "IsolationForest", scikit-learn documentation. Truy cập: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- [2] scikit-learn developers, "NearestNeighbors", scikit-learn documentation. Truy cập: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.NearestNeighbors.html>
- [3] PostgreSQL Global Development Group, "What Is PostgreSQL?", PostgreSQL Documentation. Truy cập: <https://www.postgresql.org/docs/current/intro-what-is.html>
- [4] pandas development team, "Package overview", pandas documentation. Truy cập: https://pandas.pydata.org/docs/getting_started/overview.html
- [5] NumPy developers, "NumPy: the absolute basics for beginners", NumPy documentation. Truy cập: https://numpy.org/doc/stable/user/absolute_beginners.html
- [6] SQLAlchemy authors and contributors, "SQLAlchemy 2.0 Documentation - Overview". Truy cập: <https://docs.sqlalchemy.org/en/20/intro.html>
- [7] Vasileios Papastergios, Lisa Ehrlinger, Anastasios Gounaris, "Unfolding Data Quality Dimensions in Practice: A Survey", arXiv, 2025. Truy cập: <https://arxiv.org/abs/2507.17507>
- [8] Fei Tony Liu, Kai Ming Ting, Zhi-Hua Zhou, "Isolation Forest", 2008 Eighth IEEE International Conference on Data Mining. DOI: <https://doi.org/10.1109/ICDM.2008.17>
- [9] Mehmet Tahir Aslan, "Customer Shopping Dataset - Retail Sales Data", Kaggle. Truy cập: <https://www.kaggle.com/datasets/mehmettahiraslan/customer-shopping-dataset>
- [10] Varun Chandola, Arindam Banerjee, Vipin Kumar, "Anomaly Detection: A Survey", ACM Computing Surveys, 2009. DOI: <https://doi.org/10.1145/1541880.1541882>
- [11] Stuart J. Russell, Peter Norvig, "Artificial Intelligence: A Modern Approach", Third Edition, Global Edition, Pearson Education, 2016.
- [12] Vũ Hữu Tiệp, "Machine Learning cơ bản", phiên bản cập nhật ngày 27 tháng 3 năm 2018, machinelearningcoban.com.
- [13] Peter Bruce, Andrew Bruce, "Practical Statistics for Data Scientists: 50 Essential Concepts", O'Reilly Media, 2017.
- [14] Võ Duy Tuấn, "Python rất là cơ bản", 2015.

[15] Afshine Amidi, Shervine Amidi, "CS 229 - Super VIP Cheatsheet: Học máy", bản tiếng Việt, 2020.

